

MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING
MÓDULO 9. TÉCNICAS DE ANALISIS DE MALWARE

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*



Campus Internacional
CIBERSEGURIDAD

ENIIT
INNOVA IT BUSINESS SCHOOL



UCAM
UNIVERSIDAD
CATÓLICA DE MURCIA

INDICE DE CONTENIDOS

1.	INTRODUCCIÓN	4
2.	CONCEPTOS BÁSICOS	4
2.1.	Niveles de abstracción	4
2.2.	¿Qué es el Malware?	5
2.3.	Tipos de Malware	5
3.	INTRODUCCIÓN AL ANÁLISIS DE MALWARE	8
3.1.	¿Qué es el análisis Malware?	8
3.2.	Reglas generales en el análisis de Malware	9
4.	ANÁLISIS DE MALWARE	10
4.1.	Técnicas Estáticas Básicas	11
4.2.	Técnicas Dinámicas Básicas	15
4.3.	Técnicas Estáticas Avanzadas	16
4.4.	Técnicas Dinámicas Avanzadas	17
4.5.	Metodología de análisis	18
5.	ANÁLISIS DE MALWARE EN LA PRÁCTICA	19
5.1.	Análisis de un binario X86	19
5.1.1.	Análisis estático BASICO	20
5.1.2.	Análisis dinámico BASICO	30
5.1.3.	Análisis estático AVANZADO	38
5.1.4.	Análisis dinámico AVANZADO	47
	Ejercicio Práctico	54
5.2.	Droppers y Downloaders	54
5.3.	Empaquetado y otras técnicas de ofuscación	55
5.4.	Análisis de Documentos maliciosos	65
5.5.	Análisis DLL	70
5.6.	Análisis .NET	72
5.7.	Técnicas de Process injection	75
5.7.1.	PE Injection	75
5.7.2.	Dll Injection	77
5.7.3.	Process hollowing	78
5.7.4.	Thread Execution Hijacking (A.K.A Suspend, Inject, y Resume (SIR))	83

5.7.5.	Hook Injection via Setwindowshookex	84
5.7.6.	Inyección y persistencia mediante registro (Aooinit_Dlls, Appcertdlls, ifeo)	86
5.7.7.	APC Injection y Atombombing	88
5.7.8.	Extra windows memory injection (EWMI) via Setwindowlong	90
5.7.9.	Process Injection utilizando Shims	91
5.7.10.	IAT Hooking e Inline Hooking (A.K.A Userland Rootkits)	92
5.7.11.	Process Doppelganging	93
6.	ANÁLISIS DE BINARIOS X64.	95
7.	ANÁLISIS DE MALWARE EN DISPOSITIVOS MÓVILES.	96
8.	TÉCNICAS ANTI-DEBIGGIN.	98

1. INTRODUCCIÓN

Realizar reversing de malware es sin duda una de las disciplinas más complejas del mundo de la ciberseguridad, esto se debe a que los desarrolladores de malware son expertos en codificación a bajo nivel y expertos también en reversing. Incluirán multitud de “triquiñuelas” para hacer la tarea de reversing tediosa, compleja y en ocasiones insufrible.

Por otra parte, descubrir las funcionalidades ocultas, desempacar o desofuscar el código es altamente gratificante una vez que se ha conseguido.

2. CONCEPTOS BÁSICOS

A lo largo de este módulo se estudiará todo lo necesario para poder llevar a cabo el análisis de malware. Antes de comenzar es necesario tener en cuenta algunos conceptos básicos, con lo cuáles es necesario familiarizarse y de esta forma entender todo aquello que se va a explicar.

2.1. NIVELES DE ABSTRACCIÓN

En la arquitectura de computadores, un sistema de computación puede ser representado con diversos niveles de abstracción que son creados para ocultar los detalles de implementación. Generalmente son seis niveles de abstracción, los listaremos a continuación desde el más profundo hasta el más visible.

- **Hardware:** Este nivel es el nivel físico, consiste en circuitos electrónicos que implementan una combinación de operadores lógicos (XOR, AND, OR, NOT) conocida como lógica digital. Debido a su naturaleza física, el hardware no puede ser fácilmente por el software.
- **Micro código:** Este nivel es conocido como firmware. El microcódigo opera sobre el circuito exacto para el que está diseñado. Contiene microinstrucciones que traduce a código máquina de alto nivel para proveer una interfaz junto al hardware. Cuando se realiza análisis de malware habitualmente no se tiene en cuenta el microcódigo dado que es específico para el hardware del sistema para el que se escribió.
- **Código Máquina:** Este código consiste en opcodes, que son código hexadecimal que dicen al microprocesador que debe hacer. Este código es habitualmente implementado con microinstrucciones de tal forma que el hardware que se encuentra por debajo pueda realizar la función que se ha diseñado. Este código se crea cuando se compila el código escrito en aplicaciones de alto nivel.
- **Lenguajes de Bajo Nivel:** Este lenguaje es una versión leíble por humanos y que pertenece al set de instrucciones de la máquina. El más común es el conocido

como lenguaje ensamblador. Los analistas de malware realizan sus actividades a este nivel dado que el código máquina es demasiado difícil.

- **Lenguajes de Alto Nivel:** La mayoría de los desarrolladores realizan código en este nivel. Los lenguajes de alto nivel proveen una abstracción enorme del nivel máquina y hacen fácil programar funciones lógicas y de control de flujo.
- **Lenguajes Interpretados:** Estos lenguajes son el nivel más alto, son por ejemplo, Java, .NET, C, C++, etc. El código de este nivel no es traducido a código máquina, es su lugar se traduce a bytecode. Bytecode es una representación intermedia para un lenguaje de programación específico y se ejecuta junto con un intérprete de tal forma que lo traduce a código máquina en tiempo de ejecución.

Cuando el malware es almacenado en el disco, es habitualmente en forma de archivo binario en nivel de código máquina. Cuando se realiza el desensamblado de este, se obtiene como salida código ensamblador.

El código ensamblador es un conjunto de lenguajes y es usado para programar de forma única en una familia de procesadores, por ejemplo, x86, x64, SPARC, PowerPC, MIPS y ARM, siendo x86 el más usado.

2.2. ¿QUÉ ES EL MALWARE?

Se define malware como todo aquel software cuyo fin es causar daño a un sistema o perjudicar de alguna forma al usuario del mismo. Esta palabra es un acrónimo que proviene del inglés "malicious software" y que se conoce como código malicioso.

2.3. TIPOS DE MALWARE

El malware no es un sólo tipo de software, este puede clasificarse de formas distintas según su funcionamiento. Es importante aprender a diferenciar con qué tipo de malware se está trabajando a la hora de realizar un análisis del mismo. Entre estos tipos se pueden diferenciar:

- **Virus:** Este tipo de malware es el más antiguo que existe. Su funcionamiento se basa en sustituir parte del código fuente de un ejecutable, dañando este o provocando un funcionamiento malicioso en el sistema. Este tipo de malware no tiene la capacidad de replicarse, es decir, es necesario que sea ejecutado por un usuario del sistema.
- **Gusanos (Worms):** Es capaz de auto replicarse, es decir, son capaces de infectar varios sistemas sin necesidad de que intervenga un usuario del sistema. Suelen infectar la memoria de un sistema, causando problemas en la red y usando diversos protocolos para distribuirse por ésta de forma autónoma.
- **Troyanos:** Su funcionamiento se basa en el control remoto de sistemas, es decir, que cuando un usuario es infectado por este, el usuario malicioso que lo

controla es capaz de acceder a los distintos recursos que posea el sistema infectado. El vector de infección habitual es a partir de un fichero que contenga el código malicioso. No existe un sólo tipo de troyano, ya que suelen incorporar otras funcionalidades adicionales. Algunas de ellas son:

- Backdoors: Proporciona acceso remoto al sistema.
 - Keyloggers: Registra las pulsaciones de teclado y las envía remotamente.
 - Proxy: Genera un proxy en el sistema infectado para poder filtrar el tráfico malicioso a través de este.
 - Password Stealer: Robo de credenciales de correo, bancarias...
 - Bancarios: Robo de datos bancarios.
-
- Botnets: El sistema infectado se convierte en una máquina "zombie" dentro de una gran red de máquinas infectadas que controla un usuario malicioso desde un punto centralizado.
 - Hijackers: Secuestran las funcionalidades del navegador modificando los ajustes del mismo y evitando que el usuario pueda cambiarlo.
 - Spyware: La funcionalidad de este malware es el espionaje a los usuarios que se infectan con él. Su finalidad es el robo de datos de los usuarios, los cuales, son posteriormente usados para acceso a sus cuentas y robo de identidad.
 - Adware: Su funcionalidad es mostrar publicidad intrusiva en el sistema del usuario, con ésta el usuario malicioso suele obtener algún tipo de beneficio económico por su distribución. Por otro lado este tipo de malware consume de forma notable los recursos del sistema que ha infectado llegando a provocar que funcione de forma lenta. Otra funcionalidad es la modificación DNS del sistema, provocando que el usuario sea redirigido a páginas con publicidad.
 - Ransomware: Se centra en el secuestro de los datos del usuario y petición de un rescate por los mismos. Para hacer esto, cifra todos los ficheros del sistema con una clave única, la cuál es el reclamo del usuario malicioso al usuario del sistema para recuperar sus datos. Este tipo de malware esta creciendo en los últimos años.
 - Rogue: Se trata de "software falso", es decir, que muestra ser o hacer algo que no es cierto. Al ejecutarlos suelen mostrar un falso problema o infección en el sistema que puede ser arreglado proporcionando un pago por su versión "premium".
 - Rootkit: Son un conjunto de herramientas que son usadas para acceder ilícitamente a un sistema. Estas herramientas sirven para esconder los procesos y archivos que permiten al intruso mantener el acceso al sistema, a menudo con fines maliciosos, con lo cual si un usuario intenta rastrearlo en su sistema

mostrará información falsa. Este tipo de malware normalmente suele instalarse en zonas internas del Sistema Operativo, es decir, lo más cerca del Kernel sustituyendo archivos críticos e interceptando llamadas de sistema para manipular la información que éstas ofrecen.

- PUP (Potentially Unwanted Program): Se trata de programas que se instalan sin consentimiento del usuario y realiza acciones para amenazar la privacidad, confidencialidad e integridad de los recursos del sistema.
- Spam: Se trata de correo no solicitado por el usuario que lo recibe y que se envía de forma masiva. Habitualmente suele ser un vector de entrada de otro tipo de malware más nocivo.
- Dropper: Generalmente el malware no suele mostrarse como un elemento único, es decir, se trata de varias piezas contenidas unas dentro de otras, las cuales se van generando de forma dinámica. Las nuevas partes que lo componen pueden volcarse a memoria o a disco. Al elemento inicial que genera la siguiente parte se la conoce como Dropper y al acto de generación de ese siguiente elemento "Dropeo". Un ejemplo puede ser un documento ofimático que contiene embebido un binario.
- Downloaders: Siguen la misma filosofía que los Droppers, la única diferencia es que el siguiente elemento malicioso será descargado de una ubicación remota utilizando una conexión a internet. Es decir, el código malicioso básicamente descargará el malware real.
- Fileless: Malware que se ejecuta directamente en memoria sin tocar disco. Ha de tenerse en cuenta que siempre hay un estado o fichero inicial que ejecutará el código malicioso en memoria.

De forma esquemática podemos explicar el ciclo de vida del malware de la siguiente forma:

1. Desarrollo Malware
 - Diferentes Lenguajes
 - Framework explotación
 - Reaprovechamiento código
 - Ficheros iniciales (Word, Excel, exe, dll, java, ps, js, zip, vbs, lnk,...)
2. Vector de Entrada
 - Email
 - USB
 - Exploit
 - Recurso compartido
 - Descarga de Internet

3. Stage 1
 - Fichero ofimático
 - Macro maliciosa
 - Binario Downloader
 - Binario Dropper
 - Fileless
4. Stage 2
 - C2C comunicación inicial
 - Persistencia
 - Registro Windows
 - Servicios
 - Tareas Programadas
 - Menu Inicio
 - Infección otros ficheros
 - Elevación privilegios
 - Bypass UAC
5. Stage Final
 - Acciones Maliciosas
 - Cifrado ficheros (ransomware)
 - Info Stealer (keylogger, clipboard, capturas pantalla, credenciales, ficheros)
 - Control total victima (Troyanos)
 - Seguir infectando (gusanos, file infector, etc)
 - Etc.....
 - Comunicaciones C2C

3. INTRODUCCIÓN AL ANÁLISIS DE MALWARE

3.1. ¿QUÉ ES EL ANÁLISIS MALWARE?

El análisis de malware es el arte de la disección del software malicioso para entender cómo funciona, cómo identificarlo y cómo derrotarlo o eliminarlo. Su objetivo, es poder proporcionar la información necesaria para responder a una intrusión en un endpoint, para ello analiza los supuestos elementos de software malicioso.

Cuando se está analizando malware, el objetivo fundamental es detectar que binario o binarios son los que se encuentran realizando actividades maliciosas, para saber cómo detectarlos en la red o endpoint y como contener el daño que estén causando.

Una vez identificado, se desarrollan firmas y se extraen artefactos para detectar el malware de este tipo que se encuentre en la red o endpoint. Se pueden desarrollar dos tipos:

- Host-based signatures: Usadas para detectar malware en los sistemas víctima.
- Network-based signatures: Usadas para detectar malware en la monitorización de red.

Una vez realizado este paso, finalmente se analiza el funcionamiento del software malicioso.

Los objetivos que se persiguen con el análisis de malware se resumen en:

- Entender los TTP que los atacantes utilizan.
- Fortificar nuestras defensas.
- Analizar el alcance de un incidente.
- Atribuir las campañas de malware a sus autores.
- Entender sus motivaciones y los verticales objetivo.
- Identificar artefactos, campañas, autores (IOCs)
- Compartir la información para diseminar el conocimiento en la comunidad.

3.2. REGLAS GENERALES EN EL ANÁLISIS DE MALWARE

Algunas reglas generales que se deben de tener en cuenta a la hora de realizar un análisis de malware de forma correcta son las siguientes:

Es habitual no encontrar todos los detalles del funcionamiento desde un principio. Todo malware suele ser un programa largo y complejo. Lo primordial es centrarse en los rasgos más característicos del mismo e ir localizando los detalles poco a poco. Debe recordarse que no debe usarse una sola herramienta para el análisis, cada herramienta tiene un propósito distinto y cada una sirve para una situación distinta. Con lo cual para realizar un análisis adecuado deben probarse múltiples herramientas hasta encontrar el resultado esperado.

En análisis de malware se asemeja un poco al juego del ratón y el gato, cuando se descubre una nueva técnica de análisis se lanza una nueva forma de crear malware. Con lo cual este campo está siempre en continua expansión, el analista debe adaptarse y aprender las nuevas técnicas que vayan conociéndose para poder analizar todas las amenazas que sean posibles.

En función de la naturaleza de la investigación del incidente, debemos tener en cuenta que la muestra con la que estamos trabajando no debe ser expuesta ni compartida utilizando herramientas de terceros que puedan comprometer la confidencialidad del análisis que se está realizando, por lo tanto, antes de enviar la muestra maliciosa a una sandbox pública, subirla a un motor de antivirus público o cualquier otra herramienta en cloud de terceros, habrá que evaluar si es posible o no hacerlo. En los casos en los que se quiere mantener la privacidad del análisis, nunca enviaremos el fichero malicioso, y únicamente preguntaremos por su hash en dichas herramientas para verificar si existe conocimiento previo de las mismas.

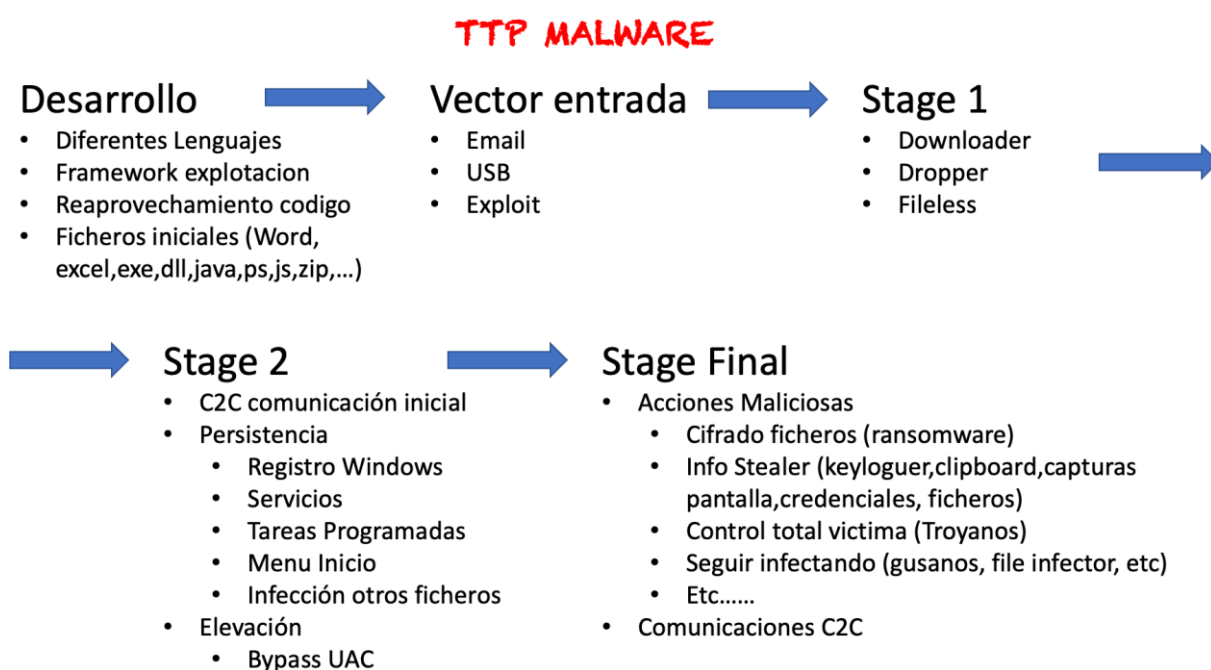
Se trata de un proceso en muchos casos de prueba y error, en el que los puntos de control (Breakpoints y Spanapshot VM), la paciencia y una metodología adecuada son la clave del éxito.

4. ANÁLISIS DE MALWARE

En este tema se explicarán las distintas técnicas que se usan dentro del análisis de malware. Se estudiará el análisis estático y dinámico básico y avanzado. Es muy importante entender las técnicas que se usan para realizar este proceso, ya que, sin comprenderlas, no se realizarían los análisis de forma adecuada, dejándose información importante en el proceso.

Generalmente, al comienzo del análisis de una muestra maliciosa, será posible inferir si deberán utilizarse las diferentes técnicas que veremos a continuación de forma parcial o total.

Nuestro objetivo inicial es entender las tácticas, técnicas y procedimientos que incluye la amenaza en su totalidad:



Captura 1: TTPs del Malware

Tras la comprensión del TTP general de la amenaza, se profundizará en el análisis detallado, para obtener todas las funcionalidades de la muestra, conocimientos detallados sobre las técnicas de evasión y anti-análisis que implementa e indicadores de compromiso específicos de la amenaza que nos ayuden a clasificarla y eliminarla.

Veremos a continuación las diferentes técnicas y herramientas que nos permiten realizar el análisis detallado de una muestra de malware, así como los objetivos que perseguimos.

4.1. TÉCNICAS ESTÁTICAS BÁSICAS

El análisis estático básico consiste en examinar el archivo amenaza sin ejecutar el fichero malicioso.

Los objetivos que se persiguen son:

- Hacer una clasificación inicial del espécimen **sin ejecutarlo**.
- Obtener una visión general de la muestra y sus capacidades, a través del análisis de sus propiedades estáticas.
- Buscar información interesante que nos de pistas sobre cómo continuar.
- Anotar la información relevante para poder validarla posteriormente por otros medios

Obtendremos:

- Packers y crypters.
- Hashes
- Secciones del fichero PE.
- Recursos incluidos.
- Imports y exports de las API de Sistema utilizadas por el malware.
- Información cifrada.
- Strings interesantes (IOC, Claves de registro, comentarios de código, etc)

Algunas herramientas útiles en este proceso:

- CFF Explorer
- Exeinfo PE
- ResourceHacker
- Pestudio
- BinText
- Etc.....

Las principales técnicas de análisis estático básico son:

1. **Análisis con antivirus:** Un primer acercamiento a la hora de comenzar a analizar malware es hacerlo usando múltiples antivirus, donde quizás alguno de estos ya lo tenga identificado.

No es una prueba fiable, por los posibles falsos positivos, es decir identificar una muestra maliciosa cuando no lo es y los falsos negativos del antivirus cuando no es capaz de detectar la muestra por firmas. Para comprender por qué ocurre esto se debe conocer de qué forma funcionan estos, siendo:

- **Firma:** Realiza una función hash sobre las piezas del malware y las compara con la base de datos.
- **Heurística:** Usa patrones de búsqueda sobre el código del malware hasta encontrar alguna coincidencia.

En cuanto a la firma, se puede modificar el código del malware para que parezca distinto al que ya conoce el antivirus, de forma que no sea capaz de reconocerlo en la base de datos. Y la heurística se puede traspasar cuando el código que se desarrolla no coincide con los patrones que ya conoce.

Un servicio web muy usado en este punto es el conocido como Virus Total (<https://www.virustotal.com>).

Es importante tener en cuenta que si se manda una muestra a Virustotal para su análisis, asumimos que va a ser compartida con el resto del mundo y por lo tanto esto debe tenerse en cuenta antes de utilizar este servicio. En caso de que se trate de una muestra que no debe hacerse pública, lo que haremos es preguntar por su hash para ver si ya hay información relativa a la misma.

Cuando se encuentra nuevo malware puede subirse a este servicio que mirará cuantos antivirus son capaces de detectarlo, además creará una firma específica para el mismo junto con otras funcionalidades.

Las características principales de VirusTotal son:

- Servicio libre e independiente.
- Ejecuta múltiples motores antivirus.
- Actualizaciones automáticas de firmas de virus en tiempo real.
- Resultados detallados de cada motor de antivirus.
- Estadísticas globales en tiempo real.
- Automatización de análisis API.
- Comunidad de investigación de malware en línea.
- Aplicaciones de escritorio (VTUploader, VTzilla) para interactuar con el servicio.

Se pueden realizar los análisis de distintas formas usando el servicio, bien cargando un archivo sospechoso o url que pueden ser maliciosas.



VirusTotal es un servicio gratuito que [analiza archivos y URLs sospechosas](#) facilitando la rápida detección de virus, gusanos, troyanos y todo tipo de malware.

No hay archivo seleccionado **Seleccionar**

Tamaño máximo: 64MB

Al hacer click en 'Analizar', acepta nuestros [Términos del servicio](#) y permite que VirusTotal comparta este fichero con la comunidad de seguridad. Vea nuestra [Política de privacidad](#) para más detalles.

Analizar

Quizás prefiera analizar una URL o buscar en VirusTotal

Captura 2: Análisis en Virus Total

Una vez cargada la muestra el motor realizará un análisis usando diversos motores antivirus para ello y mostrará el resultado de ello.

Antivirus	Result	Update
ALYac	Gen:Heur Ransom:HiddenTears.1	20170309
AVG	Atros5.NTQ	20170309
AVware	Trojan.Win32.GenericBT	20170309
Ad-Aware	Gen:Heur Ransom:HiddenTears.1	20170309
AegisLab	Troj.W32.GenericIdc	20170309
Antiy-AVL	Trojan.Win32.AGeneric	20170309
Arcabit	Trojan.Ransom.HiddenTears.1	20170309
Avira (no cloud)	TR/Spy.Gen	20170309

Captura 3: Resultados análisis en Virus Total

Esto permite a los motores antivirus y analistas de malware tener muestras frescas para poder realizar análisis y actualizar sus bases de datos ante nuevas amenazas.

Es posible registrarse en Virustotal de forma gratuita para tener la posibilidad de descargar muestras, así como de acceder a la zona de "Intelligence", en la cual no solamente veremos información de si una muestra es maliciosa o no, sino que también será posible tener datos de la frescura de una muestra, de cuantas ocasiones ha sido enviada a virustotal, entre otras informaciones de valor. Todo este tipo de información ayuda a contextualizar la amenaza, sabiendo así por ejemplo si se trata de una muestra muy nueva o es un tipo de malware antiguo, si ha sido detectado por muchos analistas o no, etc.

Hashing - Firmas de malware: Este es un método común usado únicamente para identificar malware. Se ejecuta este sobre un programa que crea un hash único para el mismo. El algoritmo que se usa para crear este es MD5 (Message-Digest Algorithm5) ó SHA1 (Secure Hash Algorithm1).

INDICATOR	TYPE	CONTEXT
0a77a39285d6bc816791320bb13408e5	MD5 hash	Dyre trojan
c3980a6228b68f88a0718de7a0362116	MD5 hash	Dyre trojan
b5b3af636f545da62f87c2773aa99016	MD5 hash	Dyre trojan
ec525c578d14a15d8d913e83ec5c557b	MD5 hash	Dyre trojan
32d32802a97b9c24e1eafceaf52440	MD5 hash	Dyre trojan

Captura 4: Hashes

Una vez realizado el proceso anterior, esta firma puede usarse de distintas formas:

- Como etiqueta para el malware.
- Como forma de compartición entre analistas de malware.
- Como búsqueda para comprobar si ha sido identificado.

Existen otro tipo de hashes que nos permitirán clasificar las muestras por hashes parciales del fichero como puede ser IMPHASH, que realiza un hash sobre la tabla de importaciones. También hay otros tipos de hashes utilizados en el ámbito de la similitud de un grupo de ficheros, para ello se utilizan los denominados Fuzzy hashing, algunos de ellos son sdhash o ssdeep. Según algunos estudios el algoritmo sdhash ofrece mejores resultados al comparar muestras de malware, al realizar el análisis de similitud mediante al usar análisis estadístico del mismo en vez de utilizar un análisis secuencial de los ficheros comparados.

2. **Análisis de Empacado:** Mediante técnicas de empaquetado y ofuscado, los desarrolladores de malware consiguen hacer más difícil la detección de malware por los antivirus y adicionalmente complican su análisis al realizar el ofuscado o empaquetando del código. Estos dos tipos de técnicas pueden definirse como:

- **Ofuscación:** Son programas cuyo autor ha intentado ocultar las funcionalidades de su ejecución.
- **Empaquetado:** Es un subconjunto de los programas ofuscados, los cuales se han comprimido y no pueden ser analizados.

Este tipo de técnicas limitarán en gran medida el análisis estático del malware al no poder ver las cadenas en texto claro o la totalidad de funciones importadas.

3. **Análisis de estructura del PE:** Mediante este tipo de análisis entenderemos como está formado el fichero, su tipo y las diferentes secciones que lo componen. Podremos comprender si incluyen payloads adicionales u otras partes del

malware que serán extraídas del mismo en tiempo de ejecución. Generalmente se asocia a muestras empaçadas.

4. **Análisis de importaciones de librerías:** Al analizar las diferentes funciones de sistema o de librerías propias del malware, podemos inferir rápidamente muchas de las funcionalidades que incorpora la muestra que se está analizando.
5. **Buscando cadenas de texto:** El tipo de cadenas de texto que suelen buscarse son mensajes que muestra el programa malicioso, conexiones a C2C, copias del archivo en ubicaciones específicas o cualquier otro detalle que lo identifique. Esta técnica puede ser una forma sencilla de conocer el funcionamiento del malware, pero no será útil si la muestra está empaçada u ofuscada, tampoco si implementa medidas avanzadas de evasión como la inyección de procesos.

Estas cadenas pueden encontrarse en formato ASCII o UNICODE, en ambos casos el almacenamiento de cadenas de texto siempre termina con el carácter NULL (0x00).

4.2. TÉCNICAS DINÁMICAS BÁSICAS

El análisis dinámico básico consiste en ejecutar el malware para observar su comportamiento en el sistema mediante el análisis de su telemetría. El objetivo es averiguar algunos aspectos de éste para poder eliminar la infección y producir firmas eficaces.

Los objetivos que se persiguen son:

- Hacer una clasificación inicial del espécimen ejecutándolo.
- Confirmar las hipótesis iniciales del análisis estático.
- Intentar montar un entorno en el que la muestra revele todo su funcionamiento teniendo en cuenta que podría ser necesario incluir servicios falsos de red.

Trataremos de monitorizar:

- Procesos
- Sistema de Archivos
- Registro de Windows.
- Recursos incluidos.
- Comunicaciones de Red.

Algunas herramientas útiles en este proceso:

- Sandbox
- Process Hacker
- Wireshark
- Process Monitor
- Etc.....

Estas técnicas son:

1. **Uso de Sandbox de Malware:** Existen una gran variedad de sandbox gratuitas que realizan análisis de malware de forma dinámica. Una sandbox es un mecanismo de seguridad para realizar la ejecución de programas en un entorno seguro que no dañe los sistemas no virtualizados.

A pesar de todo las sandbox tienen algunos problemas, como, por ejemplo, si el malware que se va a ejecutar necesita interacción del usuario y la sandbox no lo implementa, entonces no puede producirse la ejecución del malware y por tanto se produciría un análisis fallido.

Entre los problemas que pueden producir podemos destacar:

- Existe malware capaz de detectar cuando se está ejecutando en una máquina virtual y es capaz de detenerse o modificar su comportamiento.
 - Algunos malwares requieren que se encuentren algunos registros o archivos y puede que el sandbox no lo implemente.
 - Si el malware es una DLL habrá ciertas funciones que no exporten correctamente y el malware no se ejecutará.
 - El SO que emula el sandbox puede no ser correcto para el malware y por tanto dar problemas en la ejecución.
 - El sandbox no muestra todo lo que el malware hace. Reporta las funcionalidades básicas de este tras su ejecución.
2. **Ejecución en entorno controlado y obtención de telemetría:** Utilizaremos estas técnicas en el caso de que la muestra sea sensible y no se disponga de Sandbox propia, o bien si la muestra no se ejecuta en entornos de sandbox propios o de terceros por alguna de los factores comentados en el punto anterior.

Para la realización de este tipo de análisis debemos disponer de un entorno de análisis adecuado en MV que incorpore herramientas de telemetría de sistema y de red como pueden ser procmon para obtener telemetría relativa a procesos, registro o comunicaciones entre otras (sysinternals), Wireshark para examinar en profundidad las comunicaciones de red y finalmente Procesos Hacker que nos ayudará a observar a nivel de proceso, los posibles procesos hijos, hilos de ejecución e incluso los contenidos de la memoria de cada proceso de nuestro interés.

4.3. TÉCNICAS ESTÁTICAS AVANZADAS

Consiste en realizar ingeniería inversa cargando el binario en un **programa de desensamblado**, observando sus instrucciones en desensambladas con el objetivo descubrir cuáles son todas sus funcionalidades. Según el tipo de fichero analizado

deberemos realizar el análisis leyendo en diferentes lenguajes como asm, .NET o lenguajes de Scripts como Vbs o javascript.

Los objetivos que se persiguen son:

- Comprender mejor aquellas partes no reveladas por el análisis estático y dinámico básico.
- Descubrir funcionalidades e loC aún no vistos.
- Comprender las técnicas de auto-defensa e intentar hacer bypass de ellas.
- Parchear el malware para modificar su funcionalidad.
- Normalmente solo se hace reversing de código de algunas partes clave y no de todo el código.
- Servirá de ayuda en la realización del análisis dinámico Avanzado.

Algunas herramientas útiles en este proceso:

- IDA
- DnSpy

4.4. TÉCNICAS DINÁMICAS AVANZADAS

Consiste en el uso de una aplicación de debug para conocer paso a paso el estado de la ejecución interna del malware. El uso de estas técnicas permite seguir el flujo de ejecución del binario malicioso paso a paso, utilizar breackpoints para detener la ejecución, observar el estado de la memoria, los registros y la pila del sistema en cada llamada a sistema para llegar a conocer en profundidad todas sus funcionalidades.

El empleo de esta técnica facilita el análisis de funcionalidades complejas, las cuales requieren de más tiempo y esfuerzo si se realizan mediante el análisis estático avanzado.

Los objetivos que se persiguen son:

- Comprender mejor aquellas partes no reveladas por el análisis estático y dinámico básico.
- Descubrir funcionalidades e loC aún no vistos.
- Comprender las técnicas de auto-defensa e intentar hacer bypass de ellas.
- Parchear el malware para modificar su funcionalidad.

Normalmente solo se hace reversing de código de algunas partes clave y no de todo el código.

Algunas herramientas útiles en este proceso:

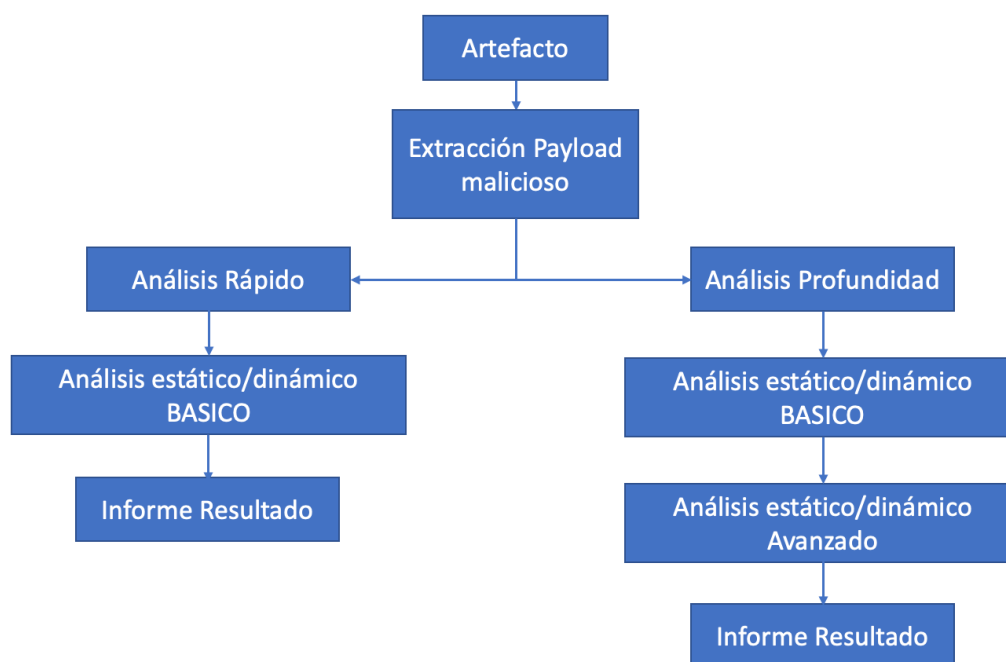
- OllyDbg y plugins de interés
- x64dbg

- Process Hacker
- dnSpy

4.5. METODOLOGÍA DE ANÁLISIS

Una vez que se comprenden las diferentes técnicas de análisis, debemos utilizar una metodología que nos permita aplicarlas de forma secuencial y estructurada. No existen como tal metodologías regladas y completas que aborden las diferentes técnicas de análisis que se han introducido en puntos anteriores. Tampoco existe una forma única de analizar una muestra o bypassar una medida de protección específica. La experiencia del reverser, sus conocimientos específicos y el manejo de las herramientas disponibles le ayudarán a construir su propia metodología. A lo largo de los siguientes módulos, se explicará una metodología general y basada en la experiencia, la cual ayudará al alumno a realizar las diferentes técnicas de análisis de una forma lógica e iterativa, que sin duda le será útil para enfrentarse al análisis de malware de una forma efectiva.

De forma muy esquemática el flujo sería:



Captura 5: Esquema alto nivel metodología de análisis

Artefacto: El análisis se iniciará con el análisis de un artefacto, ya que la pieza original en la que este contenida la amenaza puede ir desde un fichero ofimático a y binario, pasando por una imagen virtual de un disco.

Extracción de Payload: En función del tipo de artefacto se utilizarán una serie de técnicas y herramientas para la extracción del payload inicial, en caso de encontrarse contenido en otro elemento malicioso. Un ejemplo puede ser la

extracción de una macro de un documento ofimático. En otras ocasiones la pieza original será directamente un binario con el que podremos comenzar su análisis.

Tipos de Análisis: En función de la sofisticación de la amenaza o de la profundidad deseada de un análisis, se realizará un análisis básico o un análisis avanzado. Por ejemplo, si el malware implementa técnicas anti sandbox o de evasión, será necesario su análisis en profundidad.

Informe Resultado: Una vez finalizado el análisis, debemos generar un informe que contenga la información obtenida, un ejemplo de reporte de alto nivel puede ser el siguiente:

Análisis muestra: malware.exe

1. Hash:
82B5DD6165DC0883F703C3D11BB30C12CF5F6E4BCEF8C8063F0260B0D862E2E8
2. Nombre muestra: MASTER_Malware_Lab1.exe
3. Funcionalidad de la muestra:
 - 3.1. La muestra no se encuentra empacada.
 - 3.2. Crea carpetas, ficheros y se copia a si misma:
 - 3.2.1. C:\Users\Public\Public\keylogs.txt
 - 3.2.2. C:\Users\Public\Public\Screens\Screenshot.bmp
 - 3.2.3. C:\Users\Public\Public\svchost.exe
 - 3.3. Ejecuta comando de sistema en una ventana oculta.
 - 3.4. Persistencia mediante inserción de clave en el registro
 - 3.4.1. REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V
 - 3.5. Info stealer
 - 3.5.1. Keylogger
 - 3.5.2. Screenshots
4. No dispone de funcionalidades de comunicación
5. La muestra no se encuentra en VT

5. ANÁLISIS DE MALWARE EN LA PRÁCTICA

5.1. ANÁLISIS DE UN BINARIO X86

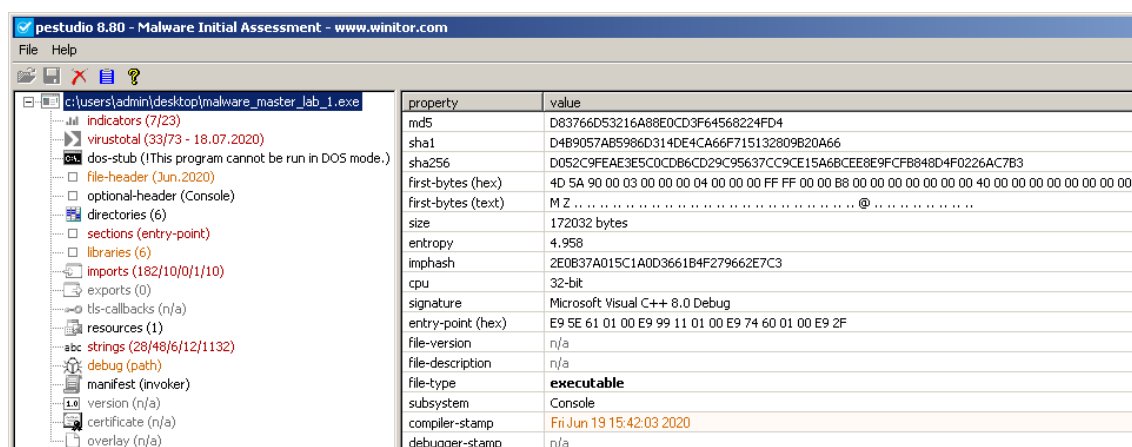
Comenzaremos el primer análisis realizando un **análisis rápido**:

Hash	muestra	maliciosa:
82B5DD6165DC0883F703C3D11BB30C12CF5F6E4BCEF8C8063F0260B0D862E2E8		

5.1.1. ANÁLISIS ESTÁTICO BASICO

Existen múltiples herramientas para realizar esta modalidad de análisis. Se utilizarán varias, pero lo importante no son las herramientas sino la interpretación que hagamos de las mismas.

Comenzaremos con la herramienta PeStudio para tener una visión global. Obtendremos sus hashes, veremos si está presente y las detecciones de Virustotal, indicadores de maliciosidad, estructura del binario, cadenas de texto, importaciones de librerías entre otra información interesante para el análisis.



Captura 6: pestudio Initial Assesment

En la captura anterior vemos un resumen de diferentes parámetros del análisis que detallamos a continuación:

- Hashes de la muestra que identifican el fichero de forma unívoca:

property	value
md5	D83766D53216A88E0CD3F64568224FD4
sha1	D4B9057AB5986D314DE4CA66F715132809B20A66
sha256	D052C9FEAE3E5C0CDB6CD29C95637CC9CE15A6BCEE8E9FCFB848D4F0226AC7B3

- Los primeros bytes del fichero, que nos aportan información sobre su cabecera. También se muestra el tamaño del archivo.

first-bytes (hex)	4D 5A 90 00 03 00 00 04 00 00 FF FF 00 00 B8 00 00 00 00 00 00 40 00 00 00 00 00 00 00
first-bytes (text)	M Z @
size	172032 bytes

- **Entropía e “Imphash” del fichero.** La entropía se define como una medida del “desorden”. Y en el ámbito del malware nos ayudará a identificar malware que utiliza técnicas de empaquetado u ofuscación, las cuales inciden en este grado de desorden de los datos contenidos por el fichero malicioso. Cuanto más alta es la entropía mayor grado de desorden se infiere y por lo tanto hay mayor probabilidad de que este empaquetado u ofuscado. En el caso concreto de nuestro análisis la entropía es baja por lo que el malware probablemente

no está empacado, no obstante, se profundizará en capítulos posteriores al respecto de la detección de packers y cómo enfrentarnos ante malware que utilizan estas técnicas.

El término de **imphash**, hace referencia a un hash basado en la tabla de importaciones que contienen las llamadas a API y su orden específico dentro del ejecutable. Este valor nos ayudará a buscar malware similar, o que al menos utilice las mismas importaciones de funciones.

entropy	4.958
imphash	2E0B37A015C1A0D3661B4F279662E7C3

- Los siguientes datos muestran información relativa a la compilación del fichero. Veremos la arquitectura que soportan, su "Entry point", es decir, donde comienza la sección ejecutable del binario, así como otros datos relativos a fechas o descripciones que pueden ser útiles para determinar frescura de la muestra, pero que deben tratarse con cautela puesto que pueden ser modificados.

cpu	32-bit
signature	Microsoft Visual C++ 8.0 Debug
entry-point (hex)	E9 5E 61 01 00 E9 99 11 01 00 E9 74 60 01 00 E9 2F
file-version	n/a
file-description	n/a
file-type	executable
subsystem	Console
compiler-stamp	Fri Jun 19 15:42:03 2020
debugger-stamp	n/a

Antes de continuar con Pestudio en detalle, analizaremos si la muestra esta empacada u ofuscada.

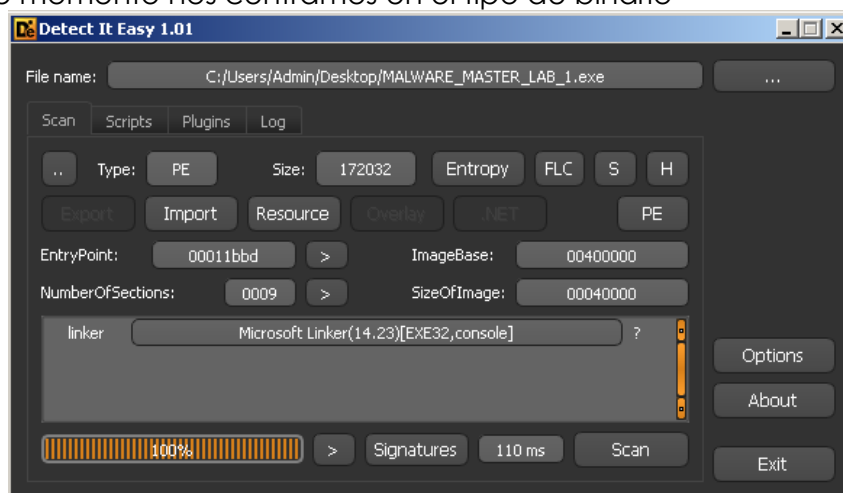
Existen múltiples herramientas, en este caso utilizaremos dos de ellas: Exeinfo PE y Detect it Easy. Si bien, veremos que hay muchas formas de detectar si la muestra esta empacada o no. Este es un paso muy importante, dado que, si la muestra esta empacada, las técnicas de análisis estáticas básicas no nos ofrecerán mucha información.



Captura 7: Exeinfo PE

En la captura anterior, vemos que tras abrir Exeinfo PE y arrastrar la muestra sobre la ventana, nos indica el tipo de fichero que es y si esta empacada o no. En el caso de la muestra que estamos analizando vemos que se trata de un fichero de 32 bits, compilado con Microsoft Visual C++ y que no se encuentra empacado.

En la siguiente captura de pantalla, utilizamos Detect it Easy, una potente herramienta con la que podremos realizar muchas tareas y que nos ofrece mucha información, y en la que de momento nos centramos en el tipo de binario



Captura 8: DetectIt Easy

En la siguiente captura vemos la información relativa a Entropía del fichero, que como ya sabemos está directamente relacionada con el empacado. En el caso que nos ocupa, la entropía es baja y el veredicto es de no empacado.



Captura 9: DetectIt Easy

Veamos ahora el detalle de las diferentes secciones más interesantes que encontramos en parte izquierda de “pestudio”

pestudio 8.80 - Malware Initial Assessment - www.winitor.com

File Help

c:\users\admin\desktop\malware_master_lab_1.exe

xml-id	indicator (23)	severity
1120	The file is scored (33/73) by virustotal	1
1245	The section (name:.msvcjmc) is blacklisted	1
1245	The section (name:.00cfg) is blacklisted	1
1223	The first section (name: textbss) is writable	1
1225	The entry-point is outside the first section	1
2215	The file has (1) writable and executable section(s)	1
1631	The file contains self-modifying code	1
1321	The time-stamp (Year:2020) of the compiler is suspicious	2
1323	The time-stamp (Year:2020) of the debugger is suspicious	2
1266	The file imports (10) blacklisted function(s)	2
2248	The file has (1) blacklisted section name(s)	2
1100	The file opts for Data Execution Prevention (DEP)	3
1633	The file references (22) rtti string(s)	3
1229	The file signature is 'Microsoft Visual C++ 8.0 Debug'	5
1430	The file references (29) blacklisted string(s)	5
1102	The file opts for Address Space Layout Randomization (ASLR)	5
1106	The file opts for cookies on the stack (GS)	5
1264	The file imports (58) decorated function(s)	5
1261	The file imports (5) deprecated function(s)	5
1040	The file does not contain a digital Certificate	7
1268	The file references (6) whitelist strings	9
1152	The file references a debug symbols file (path:'c:\users\data\documents\steal\stealer\debug\steal...)	9
1109	The file ignores Code Integrity	9

Captura 10: pestudio Indicators

En la imagen anterior, vemos los detalles de la sección “Indicators” en la que pestudio nos proporciona información de indicadores de maliciosidad, como puede ser el número de detecciones en Virustotal, nombres de secciones del fichero que están en lista negra por haberse visto anteriormente en malware, secciones escribibles y ejecutables, funciones importadas marcadas como peligrosas, información relativa a la firma del binario y otra serie de detalles iniciales que nos dan una visión general

sobre aspectos que indican la maliciosidad del fichero.

En la siguiente captura de pantalla, se muestran las detecciones de diferentes motores antivirus presentes en VirusTotal. Pestudio en ningún caso subirá la muestra a Virus Total, lo que hará es consultar por su hash. Vemos que en el ejemplo que estamos tratando, la muestra es detectada por 33 de los 73 motores antivirus disponibles en Virus Total.

engine (72)	detection (33)	date (dd.mm.yyyy)	age (days)
MicroWorld-eScan	Trojan.GenericKD.34132724	18.07.2020	79
FireEye	Trojan.GenericKD.34132724	18.07.2020	79
McAfee	RDN/Generic PWS.y	18.07.2020	79
Cylance	Unsafe	18.07.2020	79
VIPRE	Trojan.Win32.Generic!BT	18.07.2020	79
Sangfor	Malware	23.04.2020	165
CrowdStrike	win/malicious_confidence_60% (W)	02.07.2019	461
BitDefender	Trojan.GenericKD.34132724	18.07.2020	79
K7GW	Spyware (0056a4ae1)	18.07.2020	79
K7AntiVirus	Spyware (0056a4ae1)	17.07.2020	80
Symantec	ML.Attribute.HighConfidence	17.07.2020	80
ESET-NOD32	Win32/Spy.KeyLogger.RAG	18.07.2020	79
TrendMicro-HouseCall	TROJ_GEN.R03BC0PGA20	18.07.2020	79
Avast	Win32:Trojan-gen	18.07.2020	79
GData	Trojan.GenericKD.34132724	18.07.2020	79
Alibaba	TrojanSpy:Win32/KeyLogger.67f2943e	27.05.2019	497
Emsisoft	Trojan.GenericKD.34132724 (B)	18.07.2020	79
E-Secure	Trojan.TP/Snu Keylogger.chenn	18.07.2020	79

Captura 11: pestudio detecciones Virus Total

En la siguiente imagen vemos las diferentes secciones de las que se compone el binario. Como ya sabemos, los ficheros se dividen en Headers y Sections. Explorar las secciones del binario nos ayudará a entender si esta empacado o si contiene payload adicionales en los recursos del binario. En capítulos posteriores se tratará con profundidad este aspecto.

property	value	value	value	value	value	value	value	value	value
name	.text	.data	.rsrc	.rsrc	.rsrc	.rsrc	.rsrc	.rsrc	.rsrc
md5	0CF782E3318245F9601...	6F36B6180EFC43A632...	0778FE8EE8E8E74085...	3760C4F1508F2F0E36...	3904C183D18B99438EF...	68E529833F1F46CD15...	390426777B1DAE54AC12...	AR96675A0X3E15ED67A...	AR96675A0X3E15ED67A...
file-ratio (99.40 %)	0.00 %	75.00 %	13.10 %	0.89 %	5.36 %	0.30 %	0.89 %	3.57 %	3.57 %
virtual-size (28336 bytes)	65536 bytes	128722 bytes	2352 bytes	8888 bytes	407 bytes	260 bytes	1094 bytes	5977 bytes	5977 bytes
virtual-address	0x00001000	0x00001000	0x00001000	0x00001000	0x00001000	0x00001000	0x00001000	0x00001000	0x00001000
raw-size (171008 bytes)	0x00000000 (0 bytes)	0x00001F00 (129024 bytes)	0x00000500 (2528 bytes)	0x00000400 (16384 bytes)	0x00000200 (131072 bytes)	0x00000200 (131072 bytes)	0x00000200 (131072 bytes)	0x00000200 (131072 bytes)	0x00000200 (131072 bytes)
raw-address	0x00000000	0x00001F00	0x00000500	0x00000400	0x00000200	0x00000200	0x00000200	0x00000200	0x00000200
case (3214 bytes)	302 bytes	488 bytes	0 bytes	328 bytes	188 bytes	252 bytes	452 bytes	147 bytes	147 bytes
entropy	n/a	4.772	2.317	2.302	4.947	0.805	2.143	6.019	6.019
entry-point (0x00001000)	-	-	-	-	-	-	-	-	-
blacklisted	-	-	-	-	-	-	-	-	-
variable	-	-	-	-	-	-	-	-	-
executable	-	-	-	-	-	-	-	-	-
shareable	-	-	-	-	-	-	-	-	-
discardable	-	-	-	-	-	-	-	-	-
cacheable	-	-	-	-	-	-	-	-	-
pageable	-	-	-	-	-	-	-	-	-
initialized-data	-	-	-	-	-	-	-	-	-
uninitialized-data	-	-	-	-	-	-	-	-	-
readable	-	-	-	-	-	-	-	-	-

Captura 12: pestudio secciones del binario

En la siguiente imagen encontramos las librerías importadas por el binario. Será una información muy útil para determinar qué tipo de funcionalidades puede incorporar el malware. Por ejemplo, en este caso observamos que no hay librerías con funciones relativas a comunicaciones por lo que podemos pensar que no realiza conexiones de red. En cambio, si utiliza la librería gd132.dll relacionada con funciones gráficas las cuales podrían estar relacionadas con captura de pantalla. Estas suposiciones iniciales deben hacerse con cautela, ya que veremos en capítulos posteriores que la carga de librerías puede ser dinámica o que el malware esta empacado, por lo que no estaremos viendo inicialmente los componentes reales del binario malicioso.



library (6)	blacklist (0)	missing (0)	type (1)	imports (182)	file-description
kernel32.dll	-	-	implicit	32	Windows NT BASE API Client DLL
user32.dll	-	-	implicit	6	Multi-User Windows USER API Client DLL
gdi32.dll	-	-	implicit	7	GDI Client DLL
msvcrt140d.dll	-	-	implicit	61	Microsoft® C Runtime Library
vcruntime140d.dll	-	-	implicit	15	Microsoft® C Runtime Library
ucrtbased.dll	-	-	implicit	61	Microsoft® C Runtime Library

Captura 13: pestudio importación de librerías

En la siguiente captura, vemos la sección de imports, en la que se muestran las funciones que se están importando de las librerías dll que hemos visto en la sección anterior. Con estos datos disponemos de una visión más clara de las funcionalidades que el binario utiliza. Pestudio, nos marcará con una x, aquellas que se consideran de riesgo por ser generalmente utilizadas por malware para dotarlo de diferentes capacidades.



name (182)	group (10)	anonymous (0)	type (1)	blacklist (10)	anti-debug (0)	undocumented (0)	deprecated (5)	library (6)
GetAsyncKeyState	22	-	implicit	X	-	-	-	user32.dll
FreeLibrary	21	-	implicit	-	-	-	-	kernel32.dll
GetModuleFileNameW	21	-	implicit	X	-	-	-	kernel32.dll
GetModuleHandleW	21	-	implicit	-	-	-	-	kernel32.dll
GetProcAddress	21	-	implicit	-	-	-	-	kernel32.dll
AllocConsole	20	-	implicit	X	-	-	-	kernel32.dll
QueryPerformanceCounter	19	-	implicit	X	-	-	-	kernel32.dll
IsProcessorFeaturePresent	19	-	implicit	-	-	-	-	kernel32.dll
IsDebuggerPresent	19	-	implicit	-	-	-	-	kernel32.dll
SetUnhandledExceptionFilter	18	-	implicit	-	-	-	-	kernel32.dll
UnhandledExceptionFilter	18	-	implicit	-	-	-	-	kernel32.dll
RaiseException	18	-	implicit	X	-	-	-	kernel32.dll
ShowWindow	12	-	implicit	-	-	-	-	user32.dll
FindWindowA	12	-	implicit	-	-	-	-	user32.dll
InitializeListHead	7	-	implicit	-	-	-	-	kernel32.dll
WriteFile	6	-	implicit	-	-	-	-	kernel32.dll
SetFilePointer	6	-	implicit	-	-	-	-	kernel32.dll
CreateFileA	6	-	implicit	-	-	-	-	kernel32.dll
CopyFileW	6	-	implicit	-	-	-	-	kernel32.dll
GetSystemTimeAsFileTime	6	-	implicit	-	-	-	-	kernel32.dll
fclose	6	-	implicit	-	-	-	X	ucrtbased.dll
VirtualQuery	5	-	implicit	-	-	-	-	kernel32.dll
GetProcessHeap	5	-	implicit	-	-	-	-	kernel32.dll
HeapAlloc	5	-	implicit	-	-	-	-	kernel32.dll
HeapFree	5	-	implicit	-	-	-	-	kernel32.dll
memset	5	-	implicit	-	-	-	-	vcruntime140d.dll
memmove	5	-	implicit	-	-	-	X	vcruntime140d.dll
memcpy	5	-	implicit	-	-	-	X	ucrtbased.dll
malloc	5	-	implicit	-	-	-	-	ucrtbased.dll
GetStartupInfoW	2	-	implicit	-	-	-	-	kernel32.dll
Sleep	2	-	implicit	-	-	-	-	kernel32.dll
GetCurrentProcessId	2	-	implicit	X	-	-	-	kernel32.dll
TerminateProcess	2	-	implicit	X	-	-	-	kernel32.dll
GetCurrentProcess	2	-	implicit	X	-	-	-	kernel32.dll
GetCurrentThreadId	2	-	implicit	X	-	-	-	kernel32.dll
GetLastError	-	-	implicit	-	-	-	-	kernel32.dll
CloseHandle	-	-	implicit	-	-	-	-	kernel32.dll
GetCurrentDirectoryW	-	-	implicit	-	-	-	-	kernel32.dll
WideCharToMultiByte	-	-	implicit	-	-	-	-	kernel32.dll
MultiByteToWideChar	-	-	implicit	-	-	-	-	kernel32.dll
GetSystemMetrics	-	-	implicit	-	-	-	-	user32.dll
GetDC	-	-	implicit	-	-	-	-	user32.dll
ReleaseDC	-	-	implicit	-	-	-	-	user32.dll
GetDBits	-	-	implicit	-	-	-	-	gdi32.dll
DeleteObject	-	-	implicit	-	-	-	-	gdi32.dll
DeleteDC	-	-	implicit	-	-	-	-	gdi32.dll
CreateCompatibleDC	-	-	implicit	-	-	-	-	gdi32.dll
CreateCompatibleBitmap	-	-	implicit	-	-	-	-	gdi32.dll
BitBlt	-	-	implicit	-	-	-	-	gdi32.dll

Captura 14: pestudio Imports

Es importante comentar, que cuando no se dispone de experiencia en análisis de malware, podremos ayudarnos de la propia ayuda que ofrece Microsoft mediante MSDN para saber qué hace cada función. Será tan sencillo como ir a MSDN y buscar la función que nos interesa.

En la muestra que estamos analizando vemos de forma concreta:
La función **GetAsyncKeyState**, importada de la librería user32.dll, vemos que es usada

para determinar cuándo una tecla está pulsada o no, y si la tecla se pulsó después de una llamada anterior a **GetAsyncKeyState**.



Captura 15: MSDN

Si pensamos que funcionalidad de un malware requeriría saber si se ha pulsado una tecla o no, rápidamente podemos pensar que los malware que implementan funcionalidades de **Keylogger** necesitarán importar esta función. De este modo, y sin ser categóricos en este momento sin tener más evidencias, podemos pensar que es una de las funcionalidades que este malware incorpora.

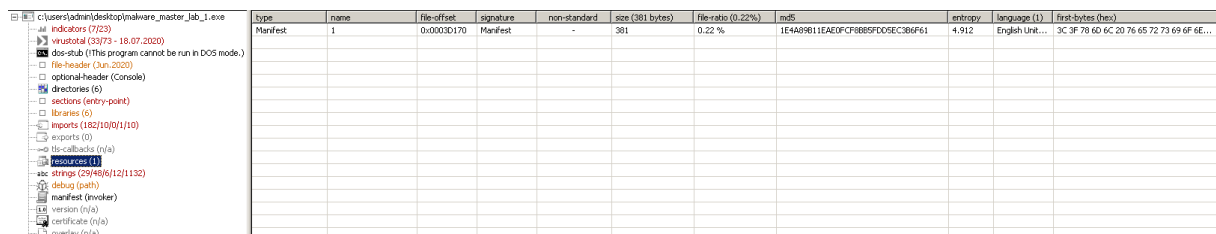
¿Tendremos que revisar una a una todas las funciones importadas?, la respuesta es no, muchas de ellas se usan de forma genérica en casi cualquier programa, por lo que trataremos de poner foco en aquellas de mayor interés. Pestudio nos dará pistas de cuáles son (sin nos fijamos en la columna blacklisted), y posteriormente nuestra propia experiencia nos permitirá de un vistazo entender gran parte del funcionamiento, analizando las funciones importadas. Es importante señalar que la aparición de alguna de ellas puede ser también utilizada de forma legítima, por lo que hasta que no se realice un estudio detallado del funcionamiento del malware, sólo debe considerarse como un indicador de posible acción maliciosa.

Otras funciones interesantes que observamos en la muestra que estamos analizando son:

- **Función relacionada con técnicas Anti debuguer:** Habitualmente las muestras maliciosas tratarán de no ser analizadas mediante el uso de un debuguer, pero los debuguers suelen incorporar opción que permiten ocultarse.
 - IsDebuggerPresent importada de kernel32.dll
- **Funciones relacionada con manejo de directorio y archivos:** El manejo de directorios y ficheros puede ser utilizado de forma lícita, pero debemos prestar atención siempre que nos encontremos este tipo de funciones durante un análisis.
 - CreateFileA importada de kernel32.dll
 - WriteFile importada de kernel32.dll
 - CopyFileW importada de kernel32.dll
 - Fclose importada de ucrtbased.dll

- **Función relacionada con acceso a ventanas de ejecución:** Acceso a ventanas de ejecución y tratar de ocultarlas para que no se vea en tiempo de ejecución que hace el malware es habitualmente utilizado por las muestras maliciosas.
 - FindWindowA importada de user32.dll
 - ShowWindow importada de user32.dll
- **Función relacionada con ejecución de comandos de sistema:** Ejecutar comandos de sistema como si se tratase de comandos ejecutados desde consola es habitual en piezas de malware.
 - System importada de ucrtbased.dll
- **Funciones relacionadas con acceso a funciones gráficas y creación de ficheros de imagen:** Pueden encontrarse asociado a acciones lícitas, pero también cuando el malware incluye funcionalidades de captura de pantalla.
 - BitBlt importada de gdi32.dll
 - CreateCompatibleBitmap importada de gdi32.dll
 - CreateCompatibleDC importada de gdi32.dll

En la siguiente ventana, podremos observar, si en la sección que generalmente es utilizada por los recursos (.rsc) que incluye un PE, por ejemplo, el icono, se está ocultando algún tipo de Payload malicioso. En este caso no encontramos nada sospechoso, únicamente el Manifest, que se trata de metadatos de información del compilado del binario.



type	name	file-offset	signature	non-standard	size (381 bytes)	file-ratio (0.22%)	md5	entropy	language (1)	first-bytes (hex)
Manifest	1	0:0000D170	Manifest	-	381	0.22 %	1E4A89B11EAE0FCB88FDD0EC386F61	4.912	English Unit...	3C 3F 78 6D 6C 20 76 65 72 73 69 6F 6E...

Captura 16: pestudio resources

En la siguiente captura, vemos una parte muy interesante del análisis, las strings que contiene, ya sean en formato ascii o en formato Unicode. Siempre ofrecen información de interés, a no ser que la muestra este empacada u ofuscada, en cuyo caso únicamente veremos caracteres sin sentido, en fragmentos cortos no legibles.

	type	size	blacked (??)	hnt (48)	whitelo (6)	gross (12)	import (0)	value (1138)
ad indicators (7/23)	ascii	103	X	X	-	-	-	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\...
! vliststat (13/73 - 18.07.2020)	ascii	34	X	X	-	-	-	C:\Users\Public\Public\log.txt
! fileheader (3.m.2020)	ascii	41	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
dos-stub (This program cannot be run in DOS mode.)	ascii	22	X	X	-	-	-	C:\Users\Public\Public\...
optional-header (Console)	ascii	30	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
directories (6)	ascii	45	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
sections (entry-point)	ascii	103	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
libraries (6)	ascii	34	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
imports (182/100/110)	ascii	95	X	X	-	-	-	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\...
exports (0)	unicode	103	X	X	-	-	-	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\...
! file-cache (n/a)	ascii	34	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
resources (1)	unicode	103	X	X	-	-	-	C:\Users\Public\Public\Screenshots\...
! string (27/48/6/12/11/32)	ascii	40	-	X	-	-	-	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\...
! string (path)	ascii	6	-	-	-	-	-	This program cannot be run in DOS mode.
manifest (invoker)	ascii	5	-	-	-	-	-	@src
! version (n/a)	ascii	4	-	-	-	-	-	start
certificate (n/a)	ascii	64	-	-	-	-	-	!src
overlay (n/a)	ascii	60	-	-	-	-	-	!src
	ascii	30	-	-	-	-	-	!src
	ascii	19	-	-	-	-	-	!src
	ascii	19	-	-	-	-	-	!src
	ascii	18	-	-	-	-	-	!src
	ascii	49	-	-	-	-	-	!src
	ascii	51	-	-	-	-	-	!src
	ascii	50	-	-	-	-	-	!src
	ascii	45	-	-	-	-	-	!src
	ascii	49	-	-	-	-	-	!src
	ascii	18	-	-	-	-	-	!src
	ascii	19	-	-	-	-	-	!src
	ascii	15	-	-	-	-	-	!src
	ascii	23	-	-	-	-	-	!src
	ascii	22	-	-	-	-	-	!src
	ascii	24	-	-	-	-	-	!src
	ascii	56	-	-	-	-	-	!src
	ascii	55	-	-	-	-	-	!src
	ascii	50	-	-	-	-	-	!src
	ascii	103	-	-	-	-	-	!src
	ascii	54	-	-	-	-	-	!src
	ascii	13	-	-	-	-	-	!src
	ascii	17	-	-	-	-	-	!src
	ascii	13	-	-	-	-	-	!src
	unicode	9	-	-	-	-	-	!src
	unicode	16	-	-	-	-	-	!src
	unicode	17	-	-	-	-	-	!src

Captura 17: pestudio strings

En la muestra que estamos analizando, podemos encontrar las siguientes strings de interés:

type	size	blacklist (29)	hint (46)	whitelist (6)	group (12)	import (0)	value (1138)
ascii	103	x	x	-	-	n/a	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\memory
ascii	34	x	x	-	-	n/a	C:\Users\Public\Public\keylogs.txt
ascii	41	x	x	-	-	n/a	C:\Users\Public\Public\Screens\screenshot
ascii	22	x	x	-	-	n/a	C:\Users\Public\Public
ascii	30	x	x	-	-	n/a	C:\Users\Public\Public\Screens
ascii	45	x	x	-	-	n/a	C:\Users\Public\Public\Screens\screenshot.bmp
ascii	34	x	x	-	-	n/a	C:\Users\Public\Public\vschost.exe
ascii	103	x	x	-	-	n/a	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\cstring
ascii	55	x	x	-	-	n/a	C:\Users\DATA\Documents\stealdealer\debug\stealer_pdb
unicode	103	x	x	-	-	n/a	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\memory
unicode	34	x	x	-	-	n/a	C:\Users\Public\Public\vschost.exe
unicode	103	x	x	-	-	n/a	C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.23.28105\include\cstring
ascii	40	-	x	-	-	n/a	!This program cannot be run in DOS mode.
ascii	6	-	x	-	-	n/a	@.rsrc
ascii	5	-	x	-	-	n/a	start
ascii	4	-	x	-	-	n/a	.bnp
ascii	64	-	x	-	-	n/a	REG ADD "HK\CLAS\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V

Captura 18: pestudio strings interesantes

Vemos en la captura anterior como se encuentran diferentes rutas "hardcodeadas" que ofrecen información de la estructura de directorios y ficheros que maneja la muestra que estamos analizando. Vemos como:

- **C:\Users\Public\Public** -> Es la ruta raíz donde el malware fija las operaciones en diferentes carpetas y archivos
- **C:\Users\Public\Public\keylogs.txt** -> Este nombre de fichero es muy descriptivo, habitualmente no será tan obvio, pero en este primer ejemplo nos ayuda a inferir que en ese fichero se almacenará el log relativo a los datos de un keylogger. Si combinamos esta información con la que hemos visto anteriormente por las funciones importadas, parece bastante claro que este malware incorpora funcionalidades de Keylogger.
- **C:\Users\Public\Public\Screens\Screenshot.bmp** -> Nuevamente encontramos un nombre de fichero muy descriptivo, nos ayuda a inferir que en ese fichero se crearán imágenes con extensión bmp. Si combinamos esta información con la que hemos visto anteriormente por las funciones

importadas, parece bastante claro que también incorpora funcionalidades de captura de pantalla.

- **C:\Users\Public\Public\svchost.exe** -> Parece indicar el nombre del fichero malicioso. Es habitual que la pieza original de malware se copie en alguna otra ruta y se renombre para luego ser ejecutada en acciones posteriores, también para ser invocada mediante algún método de persistencia como puede ser la creación de una clave de registro, creación de un servicio o una tarea programada. Nuevamente encaja con las funciones importadas que vimos anteriormente relativas a manejo de ficheros.
- **REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V ->** Esta es una ruta de registro utilizada para alcanzar persistencia, es decir para que una vez que el sistema se reinicie, la muestra se ejecute de forma automática. Existen funciones nativas para el manejo de claves de registro, pero en este caso utiliza un comando de sistema. Como vemos una vez más, encaja con la función "System" que vimos en la importación, y que será la utilizada para ejecutar este comando y conseguir persistencia-

Generalmente debemos buscar Strings que nos aporten información de posibles url, dominios o Ip que puedan encontrarse "hardcodeados" como string, en el caso de que implementen funciones de comunicaciones de red, y cualquier otra cadena de texto que podamos identificar como interesante. Habitualmente veremos miles de cadenas, por lo que podemos utilizar diferentes herramientas para buscar y filtrar cadenas interesantes. Puede utilizarse des de el comando "Strings" y usar greps para filtrar (Ejemplo: Strings malware.exe | grep http:), o programas de Windows como Bintext, especialmente diseñados para tal efecto.

En la siguiente imagen, están las opciones de debug. Generalmente nos fijaremos en el "path", ya que, si el creador del malware no ha sido cuidadoso, podemos ver en la ruta el nombre de usuario del sistema donde lo compiló. Veremos en temas posteriores, que este es un parámetro muy útil a la hora de clasificar o atribuir el malware que estamos analizando.

Captura 19: pestudio debug

En la siguiente captura vemos que no siempre se ofrece información en cada sección de estudio. En este caso detalles relativos al certificado o la versión de compilación no están presentes. Su ausencia es en si información de interés también.



Captura 20: estudio secciones sin datos

5.1.2. ANÁLISIS DINÁMICO BÁSICO

Dos aproximaciones, SANDBOX pública si la muestra no es sensible o si no se dispone sandbox privada. La otra aproximación consiste en la ejecución en la MV para obtener telemetría que aplicaremos cuando la muestra incorpore medidas anti sandbox o no dispongamos de un entorno de Sandbox privado.

1. Análisis automático (SANDBOX)

En el siguiente enlace se ofrece una lista de recursos para el análisis automático de malware: Enlace

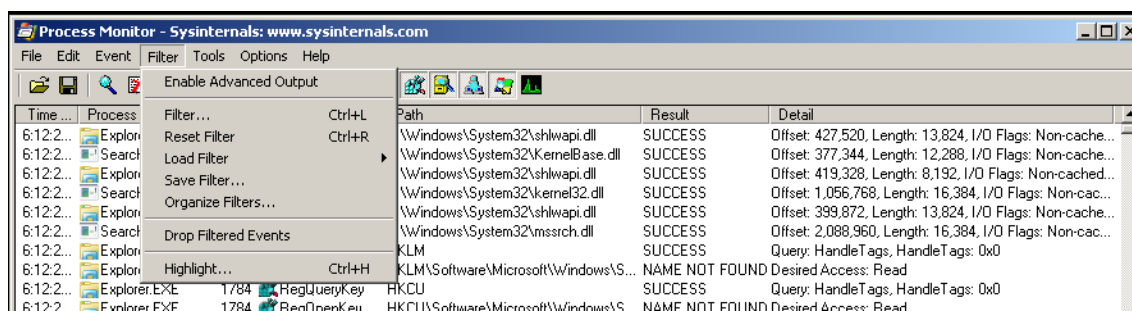
2. Análisis en entorno de MV y herramientas de telemetría

En primer lugar, hay que recordar que siempre debemos tener un snapshot de la MV con todas las herramientas de análisis instaladas, de este modo, siempre podremos volver a un estado inicial con la MV limpia, y revertir así cualquier cambio relativo a persistencia o a acciones dañinas como el cifrado del disco.

Ejecutaremos la muestra en el entorno con las herramientas de telemetría activas **procmon y Procces Hacker**. En este caso no utilizaremos herramientas adicionales de telemetría de red como Wireshark, dado que sabemos por el análisis previo que no incorpora este tipo de funcionalidades. No obstante, procmon incorpora este tipo de funcionalidades, aunque de forma limitada.

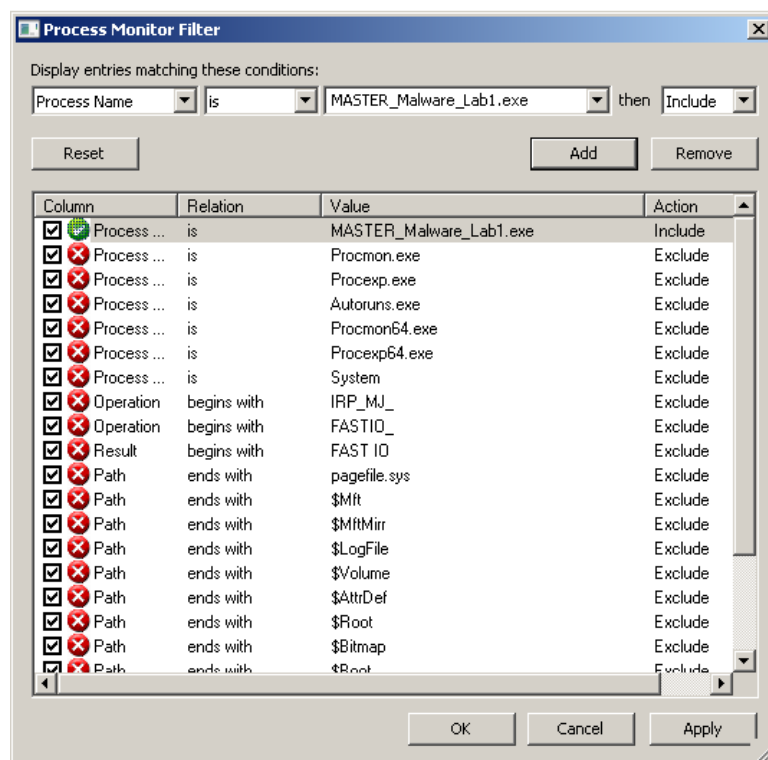
Análisis con procmon:

Una vez abierto procmon, en primer lugar debemos incluir un Filtro para que no se analice la telemetría de todos los procesos en curso. Para ello accedemos al menú a la opción Filter



Captura 21: procmon menú Filter

En la siguiente captura vemos la ventana que aparece para aplicar filtros. En la parte izquierda desplegamos el selector y seleccionamos Process Name. Después en la parte derecha escribimos el nombre del proceso, que coincide con el nombre del fichero. Seguidamente **pulsamos el botón Add y después el botón Apply** de la parte inferior derecha.



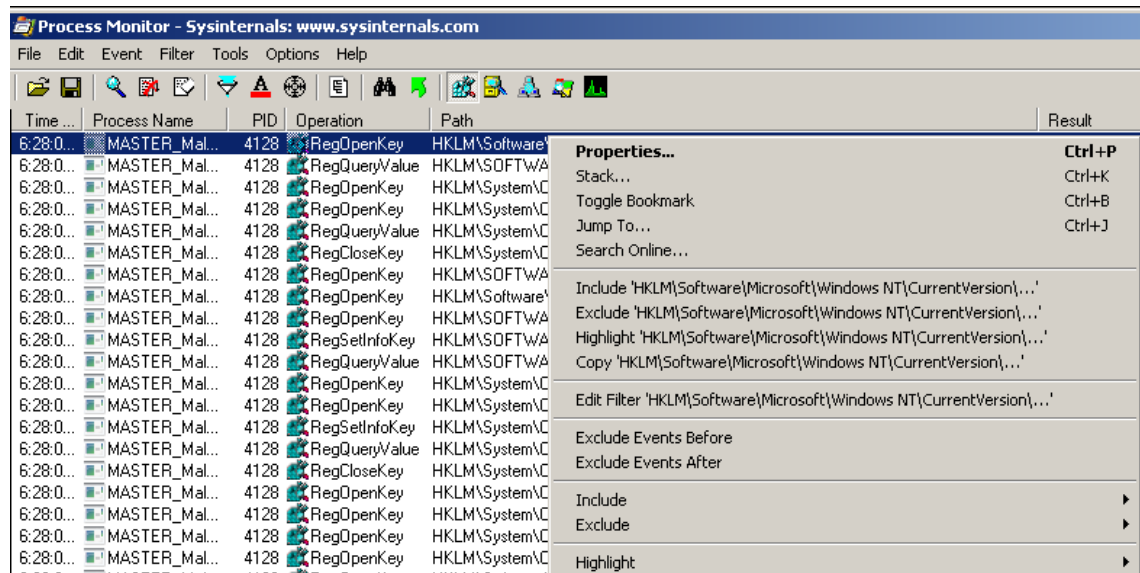
Captura 22: procmon ventana Filter

En la siguiente imagen vemos las diferentes opciones de telemetría que podemos ver. Marcaremos las que nos interesen. De izquierda a derecha, podemos ver telemetría relativa a Registro de Windows, Ficheros, Comunicaciones de red, Procesos e Hilos y finalmente el perfilado de eventos.



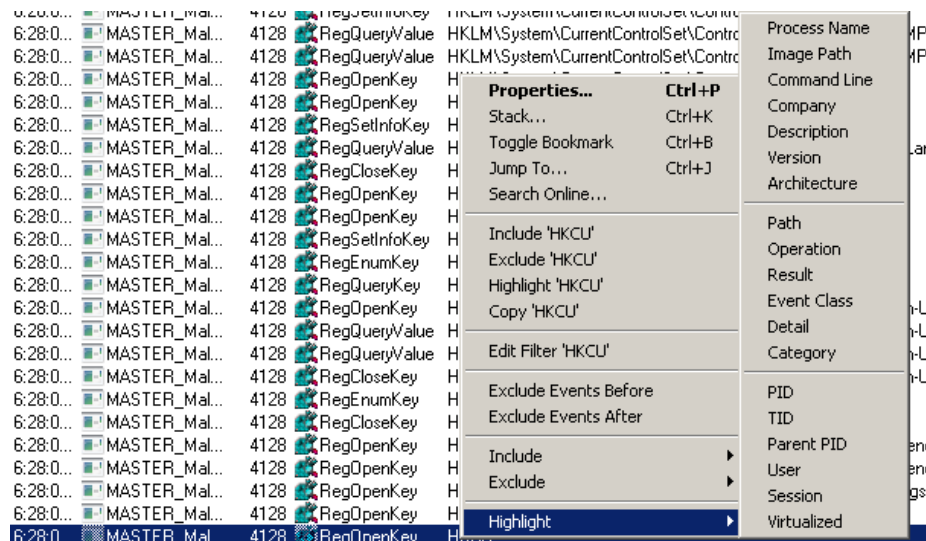
Captura 23: procmon ventana Filter

Marcaremos a continuación el Icono de registro de Windows para ver únicamente esa telemetría. Ahora haremos botón derecho sobre un evento para resaltar operaciones que nos interesen y así comenzar a buscar acciones que hayamos identificado anteriormente como las opciones de persistencia que pudimos identificar en fases anteriores.



Captura 24: procmon ventana principal

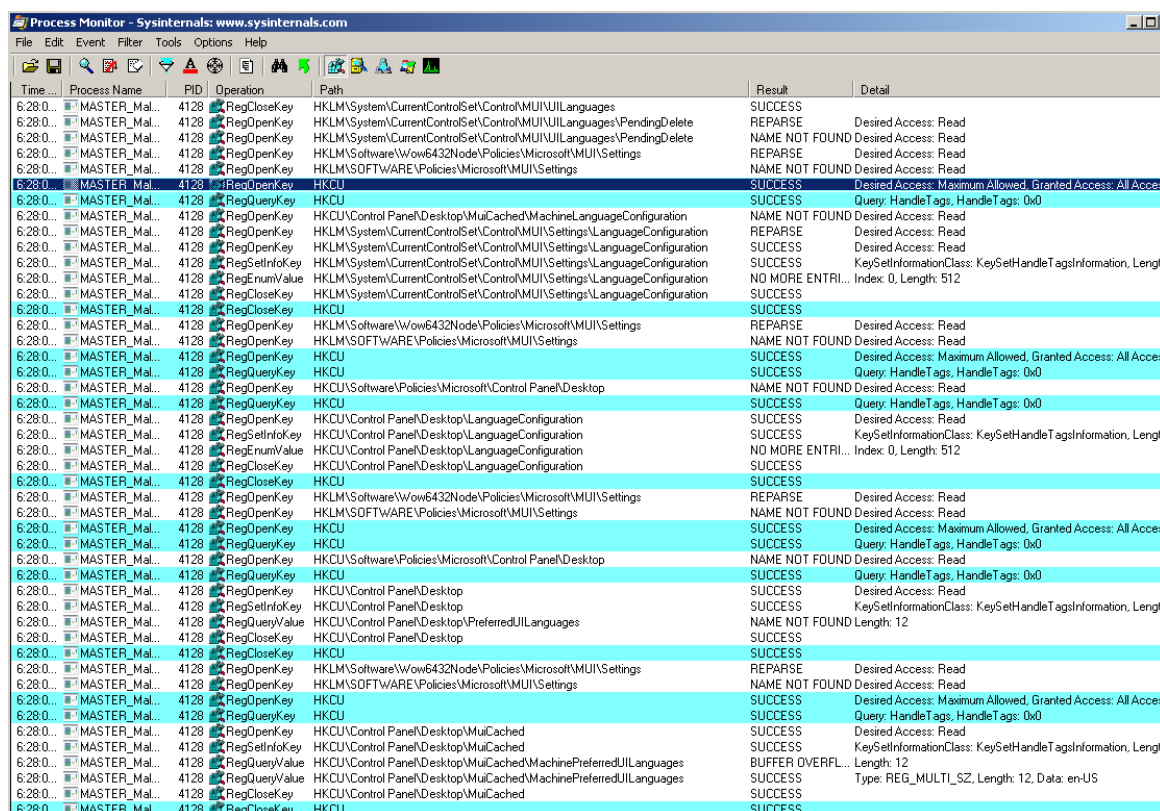
Al marcar la opción Highlight se resaltarán los eventos del tipo seleccionado.



Captura 25: procmon ventana principal, highlight

Hemos marcado las claves que comienzan por HKCU ya que anteriormente vimos que se hacía una posible persistencia sobre la clave de registro:

HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run



Time	Process Name	PID	Operation	Path	Result	Detail
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKLM\System\CurrentControlSet\Control\MUI\UILanguages	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\System\CurrentControlSet\Control\MUI\UILanguages\PendingDelete	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\System\CurrentControlSet\Control\MUI\UILanguages\PendingDelete	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Desired Access: Maximum Allowed. Granted Access: All Access
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Control Panel\Desktop\MuiCachedMachineLanguageConfiguration	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\System\CurrentControlSet\Control\MUI\Settings\LanguageConfiguration	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\System\CurrentControlSet\Control\MUI\Settings\LanguageConfiguration	SUCCESS	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegSetInfoKey	HKLM\System\CurrentControlSet\Control\MUI\Settings\LanguageConfiguration	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 12
6:28:0...	MASTER_Mal...	4128	RegEnumValue	HKLM\System\CurrentControlSet\Control\MUI\Settings\LanguageConfiguration	NO MORE ENTRI...	Index: 0, Length: 512
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKLM\System\CurrentControlSet\Control\MUI\Settings\LanguageConfiguration	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Desired Access: Maximum Allowed. Granted Access: All Access
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Software\Policies\Microsoft\Control Panel\Desktop	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Control Panel\Desktop\LanguageConfiguration	SUCCESS	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegSetInfoKey	HKCU\Control Panel\Desktop\LanguageConfiguration	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 12
6:28:0...	MASTER_Mal...	4128	RegEnumValue	HKCU\Control Panel\Desktop\LanguageConfiguration	NO MORE ENTRI...	Index: 0, Length: 512
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKCU\Control Panel\Desktop\LanguageConfiguration	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Desired Access: Maximum Allowed. Granted Access: All Access
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Software\Policies\Microsoft\Control Panel\Desktop	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Control Panel\Desktop	SUCCESS	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegSetInfoKey	HKCU\Control Panel\Desktop	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 12
6:28:0...	MASTER_Mal...	4128	RegEnumValue	HKCU\Control Panel\Desktop\PreferredUILanguages	NAME NOT FOUND	Length: 12
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKCU\Control Panel\Desktop	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	REPARSE	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKLM\Software\Wow6432Node\Policies\Microsoft\MUI\Settings	NAME NOT FOUND	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Desired Access: Maximum Allowed. Granted Access: All Access
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU	SUCCESS	Query: HandleTags, HandleTags: 0x0
6:28:0...	MASTER_Mal...	4128	RegOpenKey	HKCU\Control Panel\Desktop\MuiCached	SUCCESS	Desired Access: Read
6:28:0...	MASTER_Mal...	4128	RegSetInfoKey	HKCU\Control Panel\Desktop\MuiCached	SUCCESS	KeySetInformationClass: KeySetHandleTagsInformation, Length: 12
6:28:0...	MASTER_Mal...	4128	RegEnumValue	HKCU\Control Panel\Desktop\MuiCachedMachinePreferredUILanguages	BUFFER OVERFL...	Length: 12
6:28:0...	MASTER_Mal...	4128	RegEnumValue	HKCU\Control Panel\Desktop\MuiCachedMachinePreferredUILanguages	SUCCESS	Type: REG_MULTI_SZ, Length: 12, Data: en-US
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKCU\Control Panel\Desktop\MuiCached	SUCCESS	
6:28:0...	MASTER_Mal...	4128	RegCloseKey	HKCU	SUCCESS	

Captura 26: procmon ventana principal, highlight clave registro HKCU

Tras marcar la opción no es posible encontrar esta entrada en el registro, ¿Por qué pasa esto?

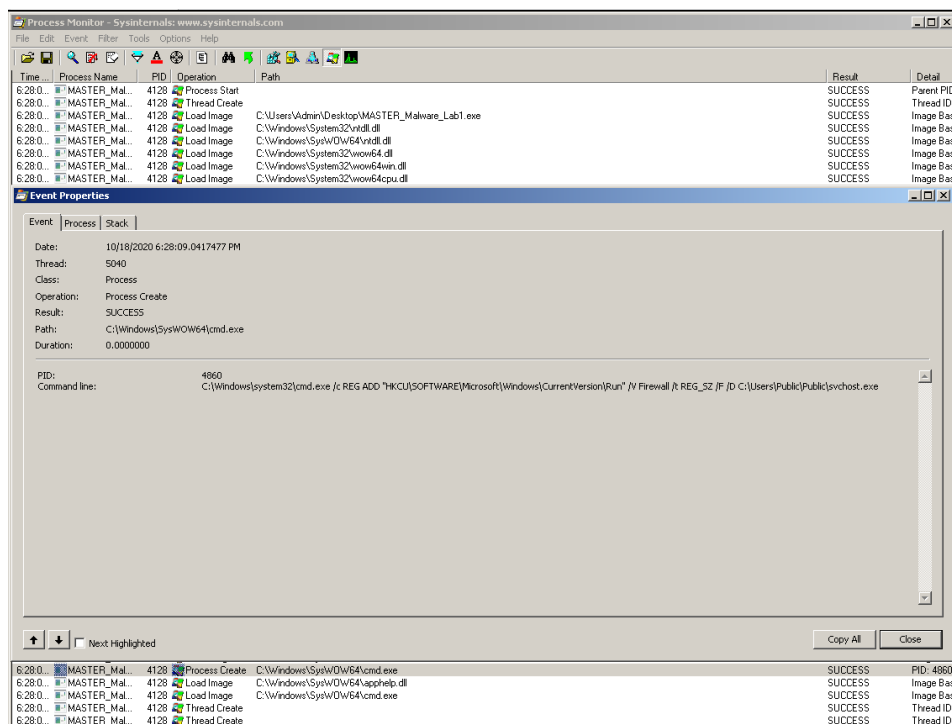
Bien, si nos fijamos en el método de persistencia que vimos anteriormente:

REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V

Lo que hace el malware realmente es ejecutar un comando de Windows y no hacer una llamada a la API de Windows utilizada para añadir una entrada en el registro. Recordemos que procmon estará viendo que llamadas de sistema se utilizan.

Lo que si pudimos ver es que importaba la función **System** que como comentamos se utilizaba para ejecutar comandos de sistema.

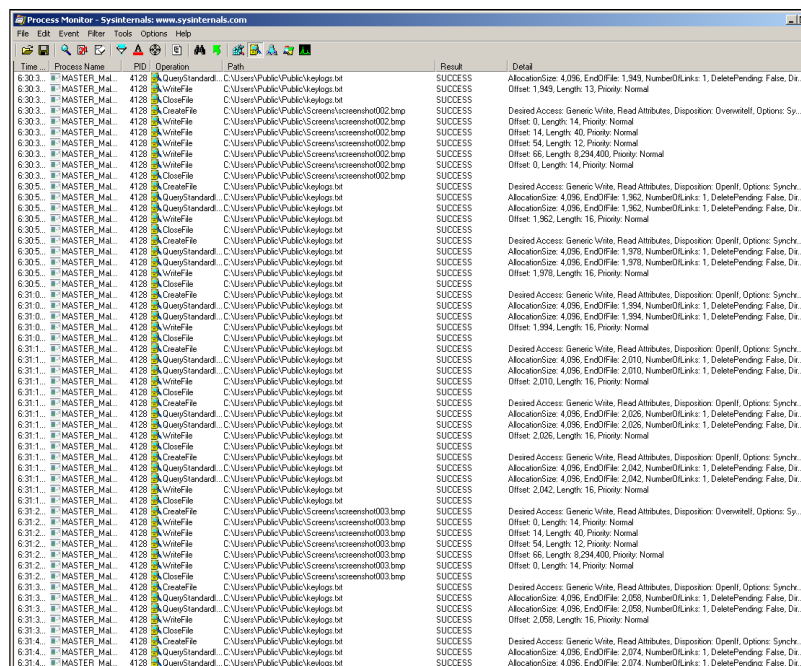
Filtremos ahora para ver la telemetría relacionada con procesos e hilos. Una vez que accedemos a estos datos, podemos ver en la siguiente captura de pantalla como el malware esta ejecutando el comando de sistema para incluir la clave de registro



Captura 27: procmon ventana principal, telemetría procesos e hilos

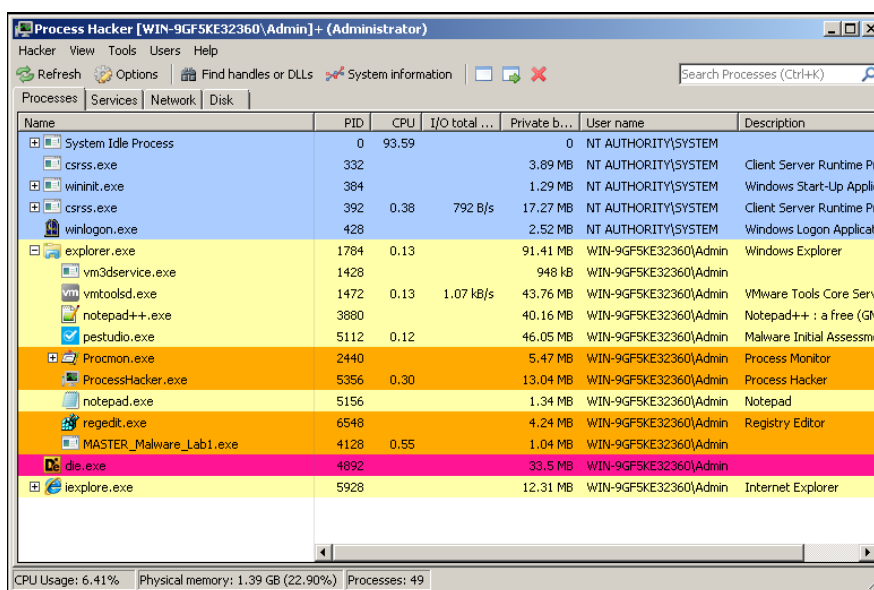
En la siguiente captura de pantalla encontramos la telemetría relativa a ficheros. Se puede ver que se escribe el fichero keylog.txt en una ruta concreta y que se crean imágenes con extensión .bmp.

Toda esta telemetría concuerda con las API de sistema que se identificaron relativas a captura de teclado, a manejo de ficheros y a creación de imágenes.



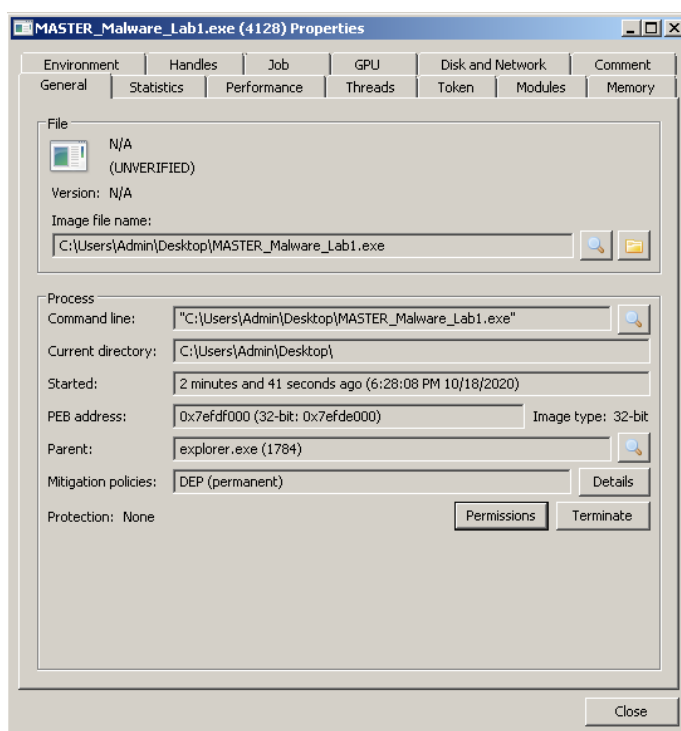
Captura 28: procmon ventana principal, telemetría de ficheros

A continuación, vemos la herramienta Procces Hacker, en la que el proceso del malware se puede observar como activo, con el PID 4128.



Captura 29: Process Hacker

Si hacemos doble click sobre el proceso, se abrirá una nueva ventana que nos ofrece diferente información relativa al proceso en ejecución.



Captura 30: Process Hacker, propiedades del proceso

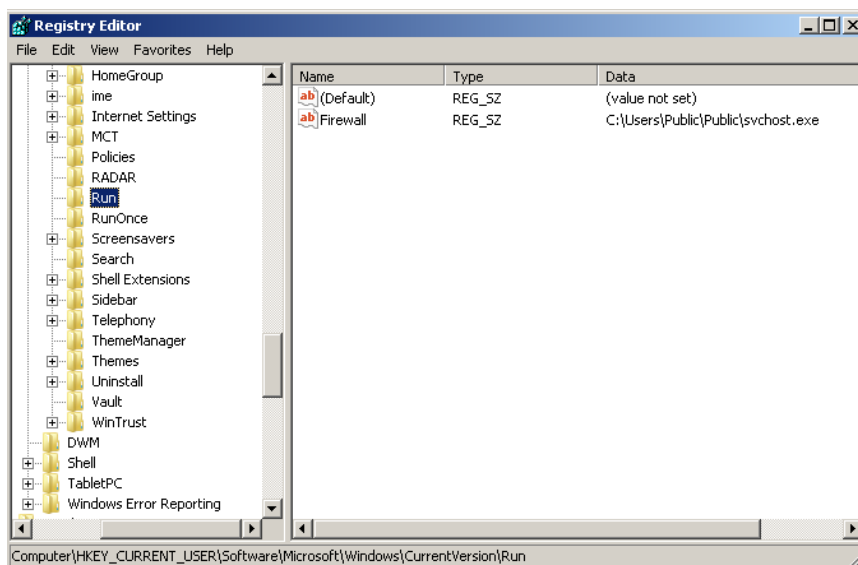
Hay diferentes secciones que utilizaremos en secciones posteriores, como los hilos o la memoria asociada al proceso.

En la siguiente captura vemos la parte de memoria, como se distribuye tanto el binario en ejecución como las diferentes DLL importadas.

Base address	Type	Size	Protect...	Use	Total WS	Private WS	Shareable WS
0x10000	Mapped	64 kB	RW	Heap (ID 2)	4 kB		4 kB
0x20000	Mapped	64 kB	RW	Heap (ID 3)	12 kB		12 kB
0x30000	Private	4 kB	RW		4 kB	4 kB	
0x40000	Image	4 kB	WCX	C:\Windows\System32\apisetschem...	4 kB		4 kB
0x50000	Mapped	16 kB	R		16 kB		16 kB
0x60000	Mapped	4 kB	R		4 kB		4 kB
0x70000	Private	4 kB	RW		4 kB	4 kB	
0x80000	Private	256 kB	RW	Stack (thread 5040)	16 kB	16 kB	
0xc0000	Mapped	412 kB	R	C:\Windows\System32\locale.nls	180 kB		180 kB
0x130000	Private	4 kB	RW		4 kB	4 kB	
0x190000	Private	256 kB	RW	Stack (thread 940)	16 kB	16 kB	
0x1e0000	Private	1,024 kB	RW	Stack 32-bit (thread 5040)	12 kB	12 kB	
0x300000	Image	56 kB	WCX	C:\Users\Admin\Desktop\MASTER_M...	48 kB	8 kB	40 kB
0x310000	Private	256 kB	RW	Stack (thread 4620)	16 kB	16 kB	
0x3e0000	Private	512 kB	RW	Heap (ID 1)	32 kB	32 kB	
0x530000	Private	1,024 kB	RW	Heap 32-bit (ID 1)	232 kB	232 kB	
0x790000	Private	64 kB	RW	Heap 32-bit (ID 2)	12 kB	12 kB	
0x7a0000	Mapped	1,568 kB	R		44 kB		44 kB
0x930000	Mapped	1,540 kB	R		124 kB		124 kB
0xac0000	Mapped	20,480 kB	R		32 kB		32 kB
0x1ed0000	Private	1,024 kB	RW	Stack 32-bit (thread 4620)	8 kB	8 kB	
0x2020000	Private	1,024 kB	RW	Stack 32-bit (thread 940)	8 kB	8 kB	
0x661d0000	Image	304 kB	WCX	C:\Windows\SysWOW64\apphelp.dll	80 kB	8 kB	72 kB
0x726c0000	Image	12 kB	WCX	C:\Windows\SysWOW64\api-ms-win...	12 kB		12 kB
0x726d0000	Image	20 kB	WCX	C:\Windows\SysWOW64\api-ms-win...	20 kB		20 kB
0x726e0000	Image	12 kB	WCX	C:\Windows\SysWOW64\api-ms-win...	12 kB		12 kB
0x726f0000	Image	12 kB	WCX	C:\Windows\SysWOW64\api-ms-win...	12 kB		12 kB

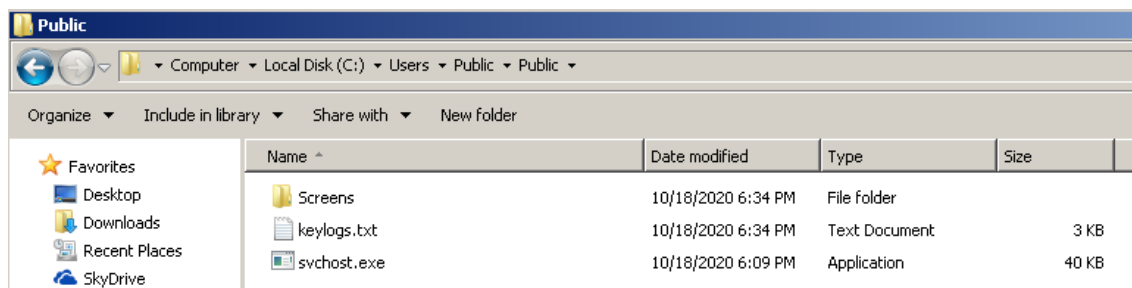
Captura 31: Process Hacker, detalles de memoria

Si abrimos el editor de registro y accedemos a la clave que se identificó anteriormente, podemos verificar que efectivamente se ha creado la clave de Registro Firewall que apunta al fichero svchost.exe, el cual se trata de una copia del fichero malicioso original del malware. De este modo es como el malware alcanza la persistencia.

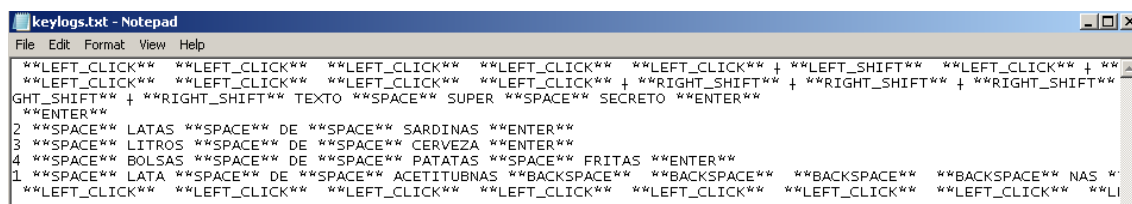


Captura 32: Regedit

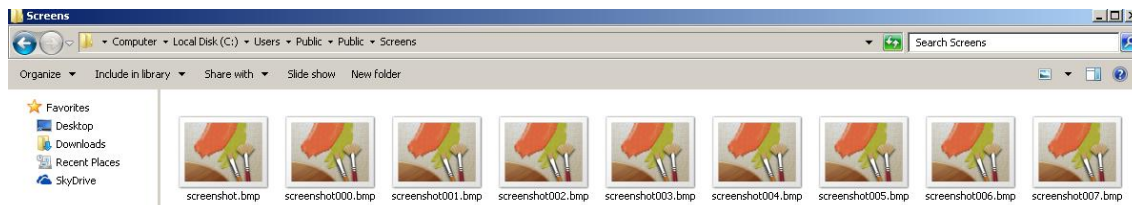
En la siguiente captura accederemos a la ruta en la que vimos que se guarda el fichero que almacena las pulsaciones de teclas, la copia del fichero malicioso a la que apunta la clave de registro y una carpeta que almacena las capturas de pantalla



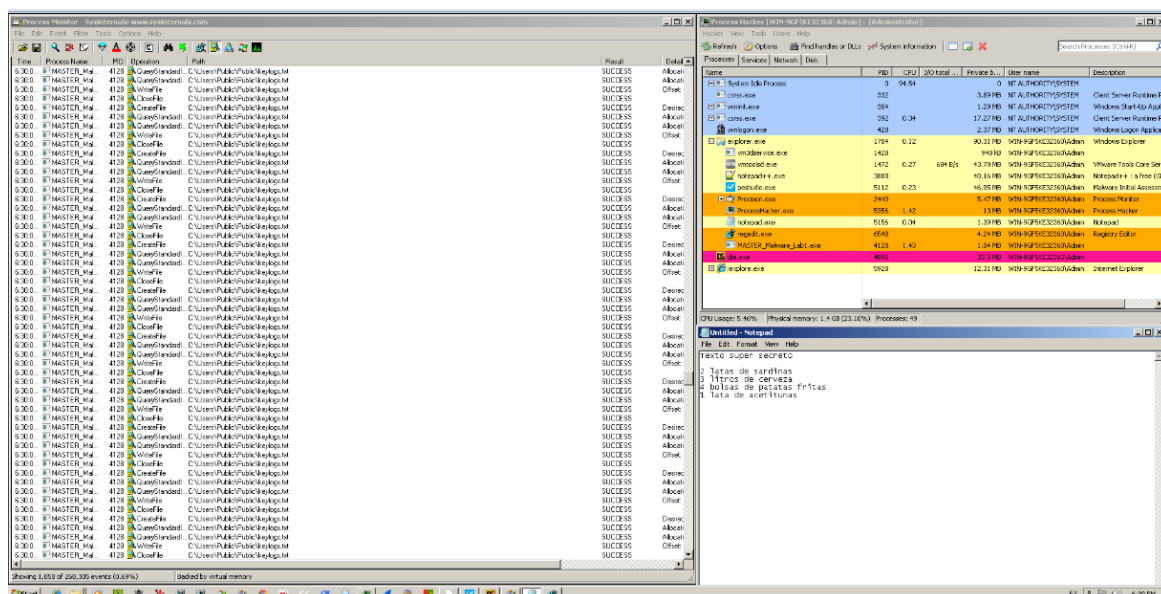
Captura 33: Directorio ocupado por el malware



Captura 34: Fichero que almacena las pulsaciones de teclas



Captura 35: Directorio con las capturas de pantalla



Captura 36: Captura de pantalla

5.1.3. ANÁLISIS ESTÁTICO AVANZADO

A continuación, comenzaremos con las tareas avanzadas de análisis. Como puede verse, se abordan de forma conjunta tanto la parte dinámica como la parte estática. Si bien se comenzará con el análisis estático, en el caso de que la muestra esta empacada, puede ser necesario realizar tareas dinámicas de desempacado/desofuscado para luego abordar nuevamente el análisis estático avanzado. El siguiente esquema muestra de forma esquemática el flujo que seguiremos para realizar los análisis.



Captura 37: Metodología de análisis avanzado

Hemos verificado anteriormente en la fase de análisis estático/dinámico básico que se trata de una muestra no empacada por lo que directamente seguiremos el flujo de la izquierda del diagrama anterior y también pudimos extraer los IoC, Strings e imports interesantes, así que abriremos en IDA el fichero a analizar.

IMPORTS de interés

- GetAsyncKeyState
- IsDebuggerPresent
- GetCurrentDirectoryW
- CreateFileA
- WriteFile
- CopyFileW
- Fclose
- FindWindowA
- ShowWindow
- System
- BitBlt
- CreateCompatibleBitmap
- CreateCompatibleDC

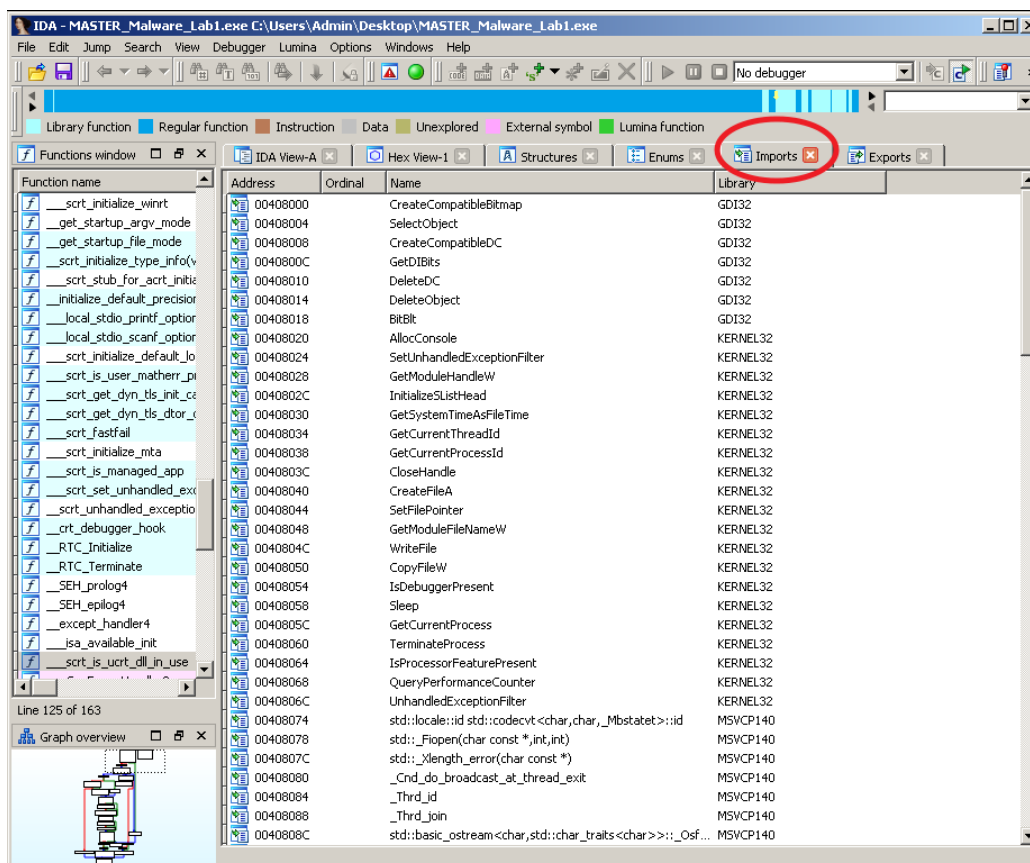
STRINGS de interés

- C:\Users\Public\Public
- C:\Users\Public\Public\keylogs.txt
- C:\Users\Public\Public\Screens\Screenshot.bmp
- C:\Users\Public\Public\svchost.exe

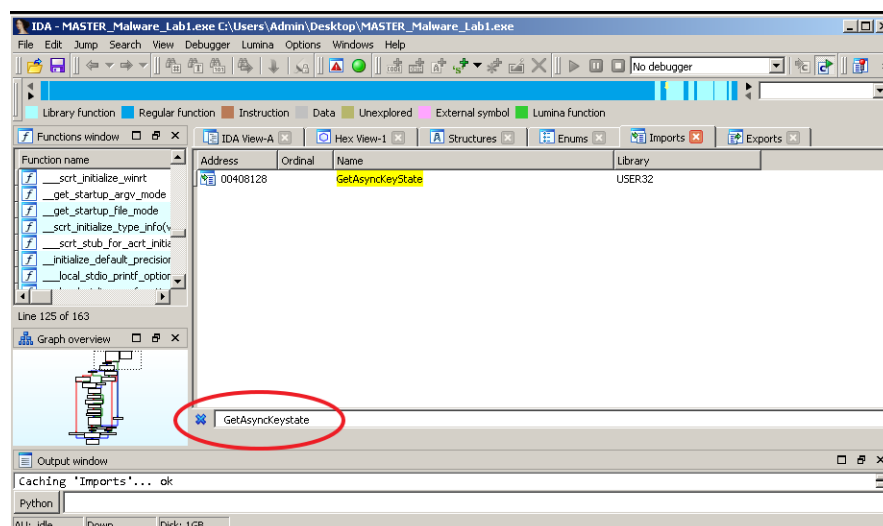
- REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V

1. Localizamos en el código los IoC, Strings e Import de interés descubiertos durante el análisis básico.

Una vez abierto en IDA, procedemos a abrir la pestaña de importaciones. **Luego buscaremos (Ctrl+f) los “imports”** que nos interesan. Comenzaremos con la función **GetAsyncKeystate**



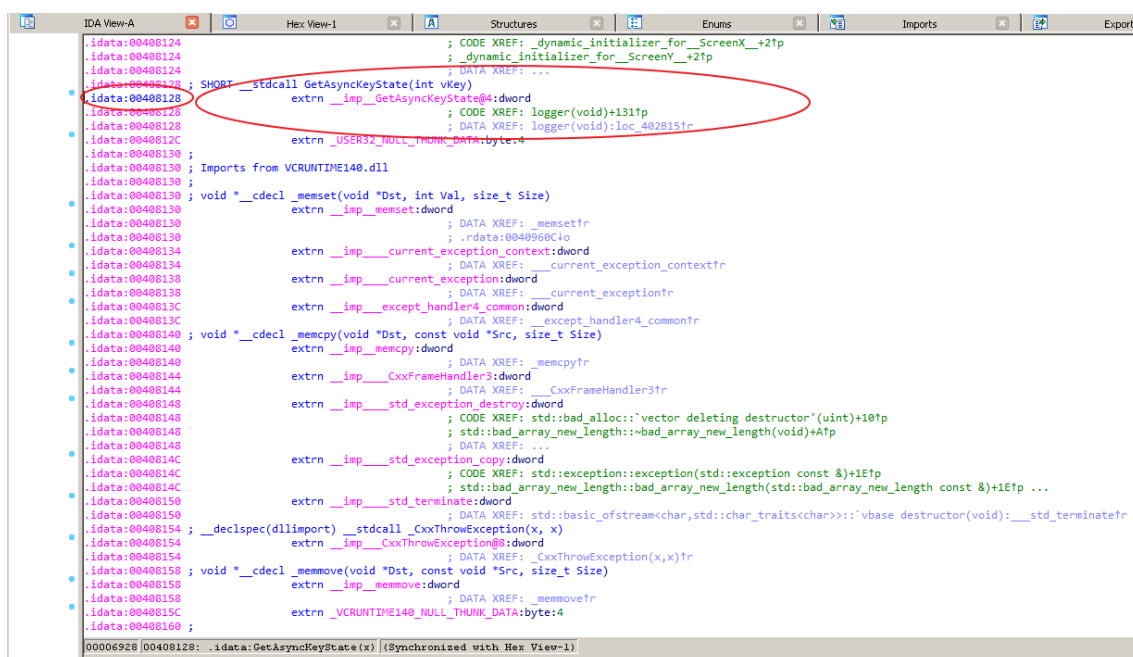
Captura 38: IDA Imports



Captura 39: IDA búsqueda de Import GetAsyncKeyState

Ahora haremos **doble Click sobre el nombre de la función**, e IDA nos mostrará en que parte del desensamblado del binario se está importando la función, **lo veremos en la sección IDA View-A.**

En nuestro malware, vemos que se realiza la importación en la **sección idata.**

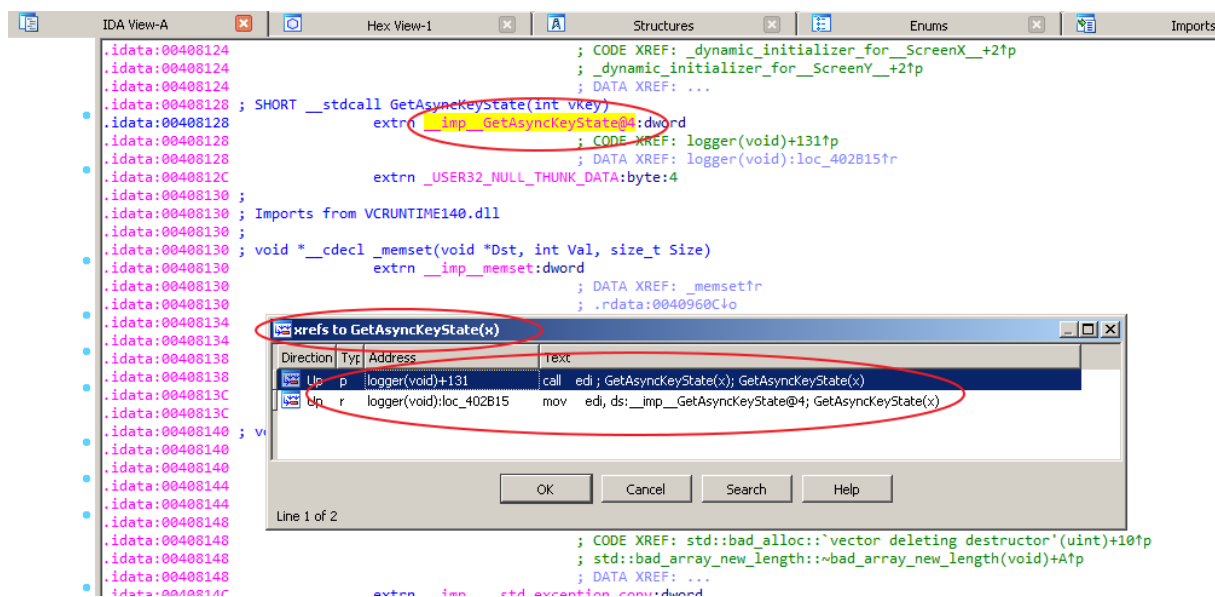


Captura 40: IDA View-A

Como vemos en la siguiente captura, hacemos Click sobre el nombre de la función y seguidamente **pulsamos la Tecla x, con el objetivo de ver las referencias cruzadas**, es decir desde que funciones de la sección ejecutable del código se invoca esa función para ser utilizada.

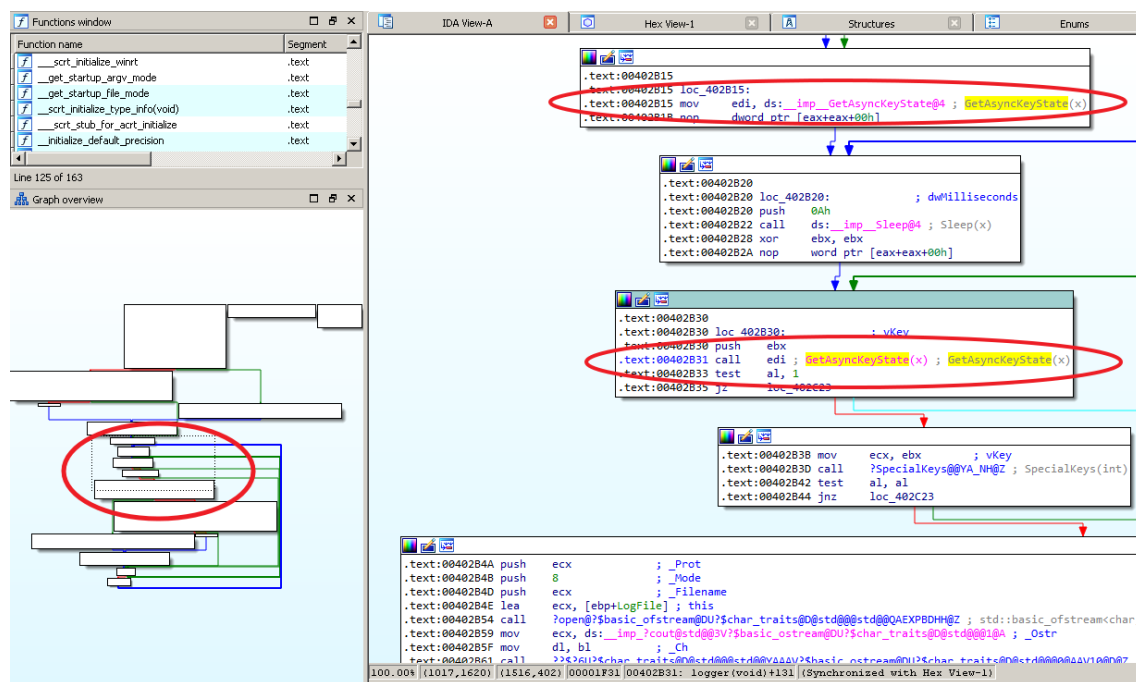
La función es llamada dos veces por la función `logger(void)`.

En la siguiente captura, de abajo hacia arriba en la **columna Address**, primero vemos como se hace un **mov** al registro **edi** de la dirección de memoria donde está la función, y seguidamente se realiza la invocación mediante un **Call** a la dirección de memoria contenida en el **registro edi** que almacenamos previamente.



Captura 41: IDA xrefs

Ahora hagamos **doble Click sobre la función logguer(void)** en la parte que realiza el **Call**, para que IDA nos lleve a la función en IDA View-A.

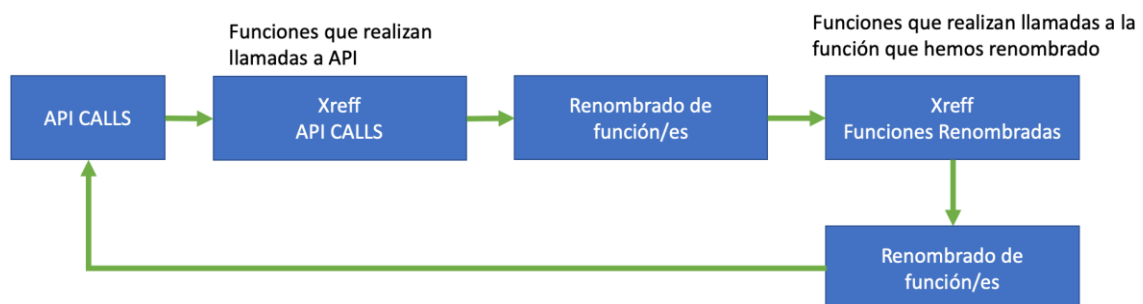


Captura 42: IDA View-A con en la parte del código que invoca la función

2. Renombrado de la función.

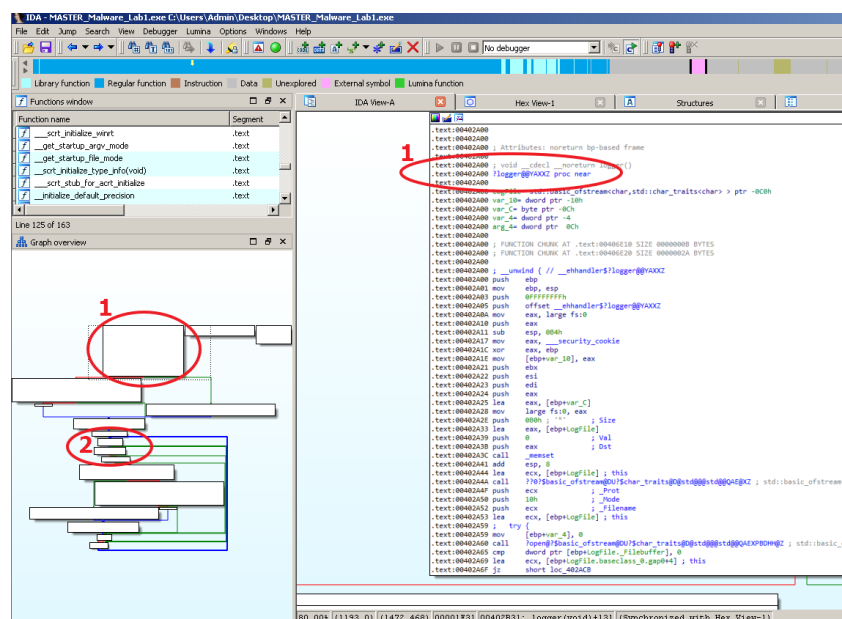
Una vez que nos hemos situado en la función que realiza la llamada, y sabiendo que está relacionada con la funcionalidad de keylogger de esta muestra, **tendremos que analizarla con más detalle para entenderla mejor y renombraremos la función en la que está contenida esta llamada con un identificador lógico**, que nos ayude a saber que en esta parte del código se realiza el keylogger. El análisis inicial para entender mejor la función requerirá **principalmente observar los Call que hay en el código, para entender algo mejor el flujo**.

Para renombrar la función, obviamente debe estar relacionado con la parte de keylogger, ya que, si esta función concreta realiza otras muchas operaciones de interés, será mejor elegir otro nombre. Ahora no es el momento de entender cada función al 100%, se trata de ir renombrando funciones para ir entendiendo de forma global las diferentes funciones.



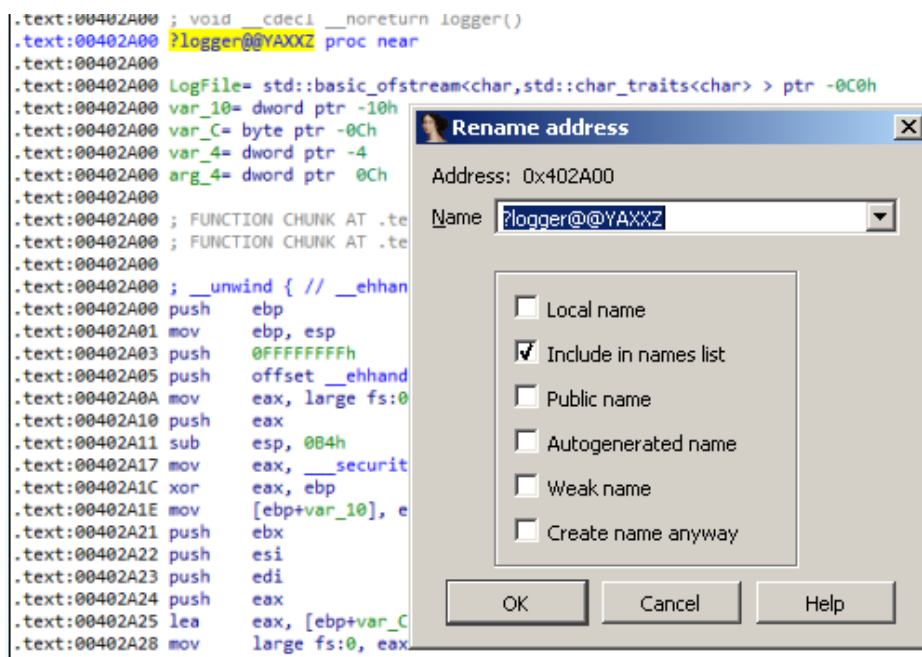
Captura 43: Esquema renombrado de funciones

En la siguiente captura, vemos en **1** el **inicio de la función logger**. En **2** esta la parte donde se hace la invocación. Al examinar los Call anteriores y el código, de momento sin fijarnos en el detalle, veremos algunas llamadas a funciones que trabajan con ficheros y algunas funciones que requieren más análisis, por lo que inicialmente vamos a renombrar esta función como **"KEYLOGGER"**.



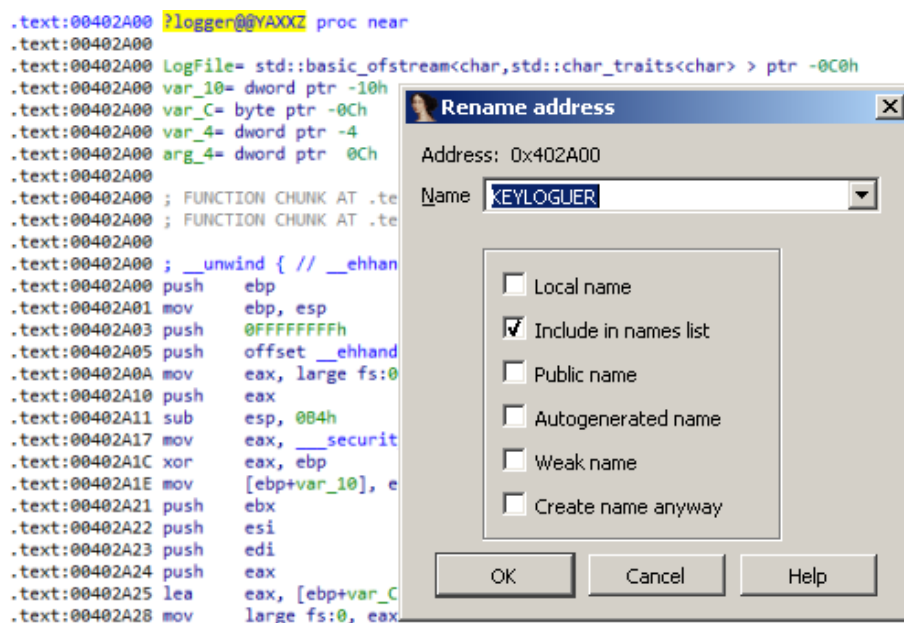
Captura 44: IDA View-A con en la parte del código que invoca GetAsyncKeystate

Para renombrar la fusión debemos ponernos sobre su nombre y pulsar la **tecla n**



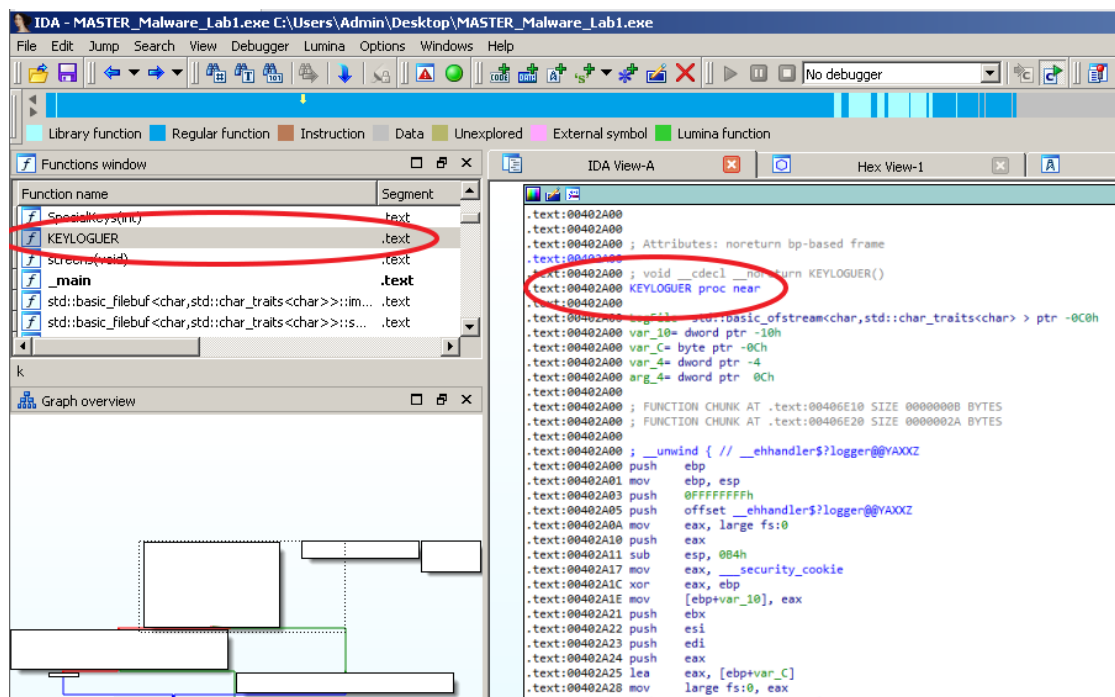
Captura 45: IDA View-A renombrado de función

Una vez que se abre la ventana, le pondremos el nombre que nos interesa.



Captura 46: IDA View-A renombrado de función

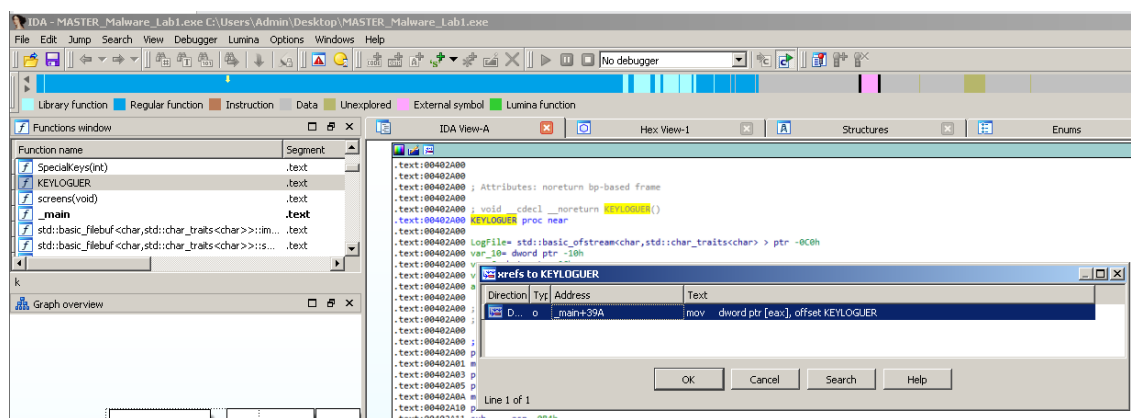
Al pulsar el botón **OK** la función será renombrada a "KEYOGUER"



Captura 47: IDA Función renombrada

3. Buscar referencias cruzadas que invocan la función renombrada.

Ahora hacemos **Click sobre el nombre de la función**, y como hicimos anteriormente pulsamos la **Tecla x** para encontrar la/s funciones que llaman a la función “KEYLOGUER”

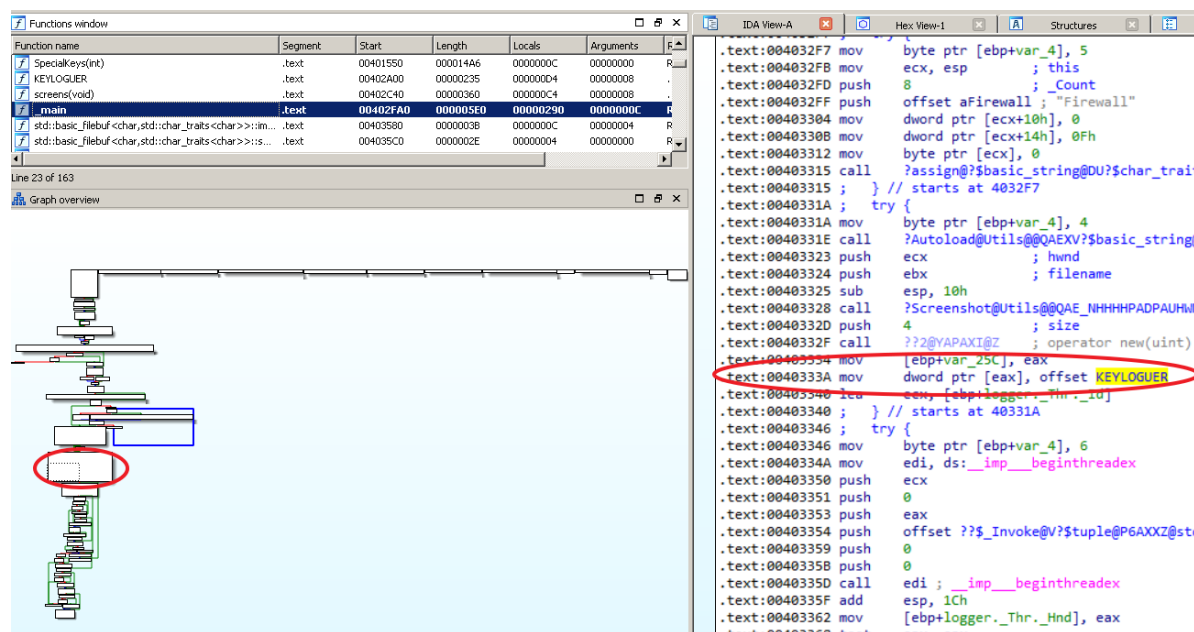


Captura 48: IDA referencia cruzada de la función KEYLOGUER

En la captura anterior vemos como la función que llama a “KEYLOGUER” es la función main. El nombre es bastante aclarador y generalmente será así, ya

que nos indica claramente que se esta invocando desde la función principal del malware. Haremos doble click sobre ella para ver la función “main”.

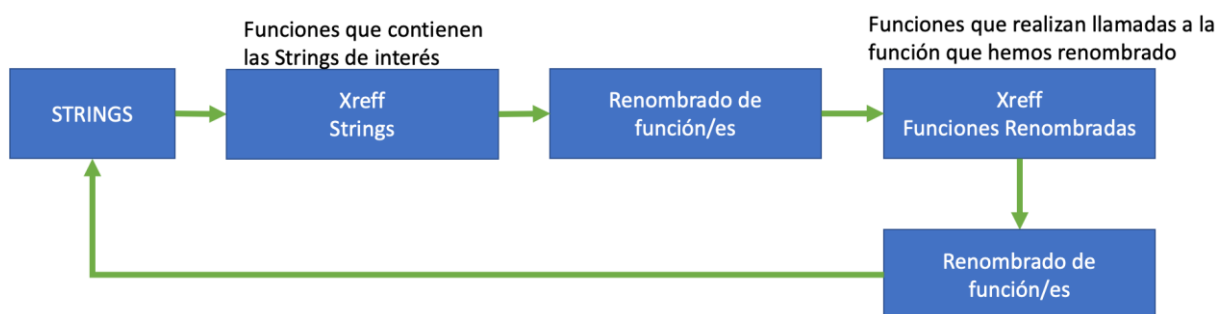
En la siguiente captura vemos la función main del malware. En la vista Gráfica de la izquierda se puede observar que es una función grande y compleja. Y sabemos en qué parte se invoca la funcionalidad de keylguer por la invocación a la función “KEYLOGUER”



Captura 49: IDA función main

Seguiremos realizando los pasos del 1 a 3 de forma iterativa con todos los imports, si alguna de las funciones incluye muchas otras llamadas o no está totalmente claro el motivo de su uso, la renombraremos con un identificador o nombre que genérico que nos permita saber que debemos prestar atención, por ejemplo se puede renombrar la función como “INTERESANTE1”, “INTERESANTE2”, etc.

Seguidamente haremos el mismo proceso con las Strings que habíamos anotado.



Captura50: Esquema renombrado de funciones a partir de strings

Veamos como continua todo el resto del proceso en el siguiente video.

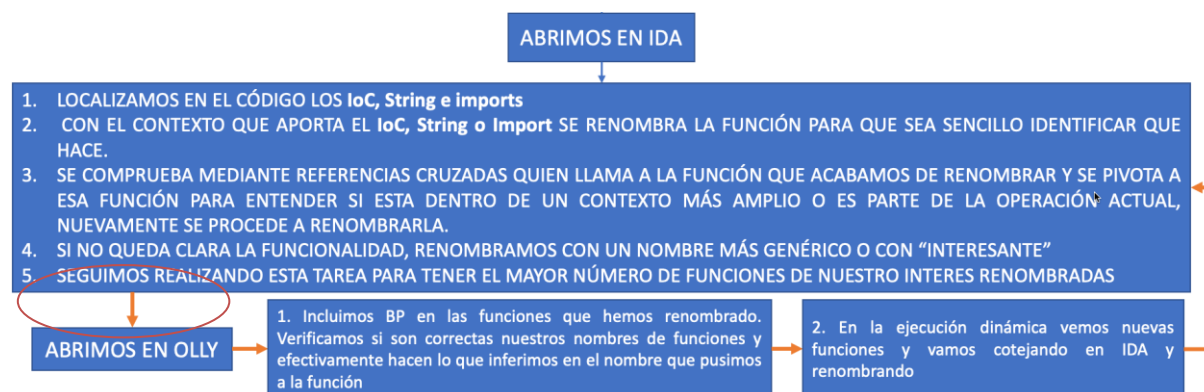
Video 1. Reversing estático avanzado muestra MASTER_Malware_Lab1.exe
Hash: 82B5DD6165DC0883F703C3D11BB30C12CF5F6E4BCEF8C8063F0260B0D862E2E8

5.1.4. ANÁLISIS DINÁMICO AVANZADO

Una vez que tenemos identificado el funcionamiento del binario, vayamos a ejecutarlo con el debugger. Ahora partimos con ventaja, **sabemos en que direcciones podemos poner breackpoints para observar partes concretas de la ejecución de nuestro malware. Sabemos también que funciones no comprendemos totalmente con el análisis estático, y podremos poner breackpoint sobre ellas para analizarlas en ejecución. Finalmente, hay funcionalidades como el keylogger que para verlas “en acción” requieren pulsación de teclas, por lo que el análisis dinámico nos será muy útil en este tipo de situaciones tambien.**

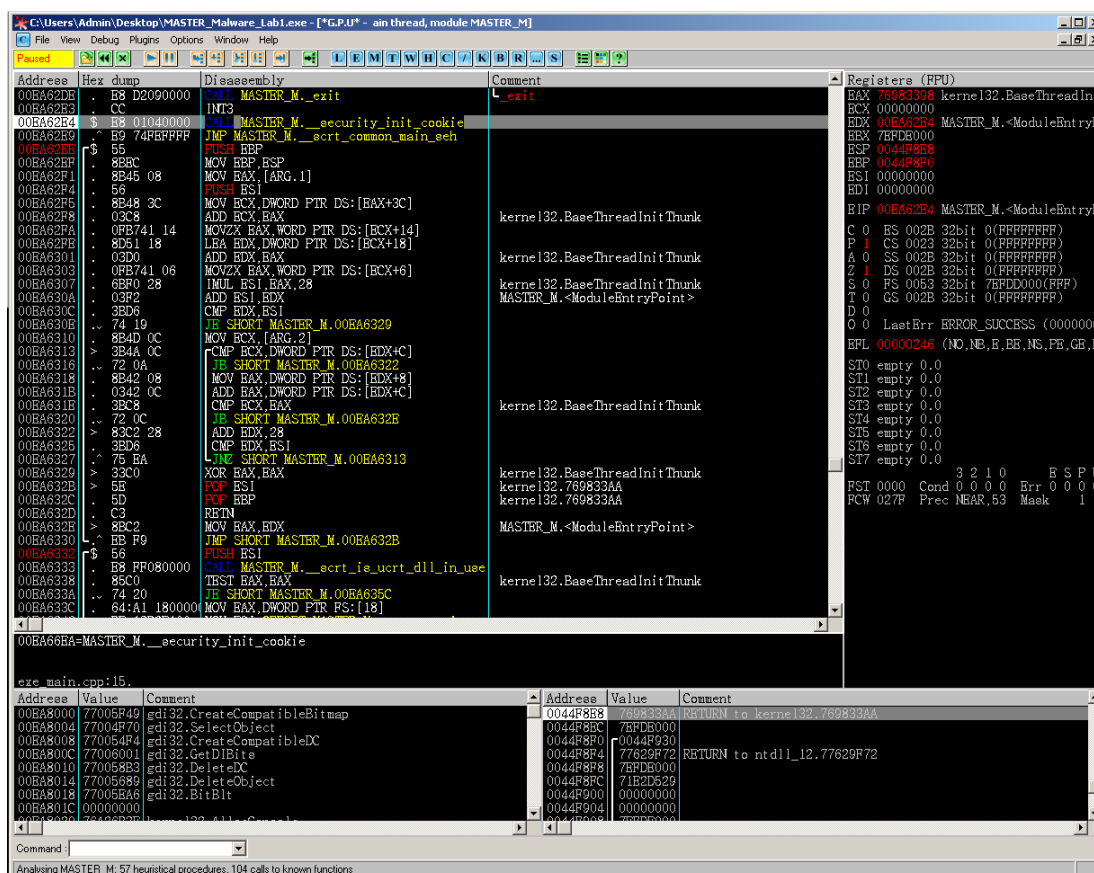
Para la realización del análisis Dinámico avanzado recomiendo mantener abierto tanto el IDA como el debugger, en este caso será Olly. En malware complejos será de mucha utilidad ir pudiendo situar el punto de ejecución, dado que, en el análisis dinámico, en ocasiones, los saltos en memoria entre unas funciones u otras o entre el código de usuario y el código de las librerías, puede resultar complejo para realizar un correcto seguimiento.

Siguiendo el diagrama de nuestra metodología:



Captura 51: Esquema metodología análisis dinámico avanzado

1. Abriremos la aplicación en Olly Debugger



Captura 52: Olly Debugger

2. Crearemos BP (Break Points) en las funciones que identificamos previamente

Antes de comenzar a colocar los BP, debemos tener en cuenta que estos se ponen en direcciones de memoria. Y en ocasiones cuando abramos nuestro programa en Olly, puede darse la situación que su base address sea diferente, así que habrá que en ese caso habrá que realizar un "Rebasing" en IDA para alinear ambos espacios de memoria.

En primer lugar, comprobamos el base address del malware en Olly, abrimos la ventana de "Executable modules" con las **teclas Alt+E**. En nuestro caso vemos como la dirección Base de nuestro ejecutable es "00EA0000"

Memory map								
Address	Size	Owner	Section	Contains	Type	Access	Initial	Map
00010000	00010000	apiasetec			Map	RW	RW	
00020000	00010000				Map	RW	RW	
00030000	00009000				Priv	RWE	RWE	
00040000	00001000				Imag	R	RWE	
00050000	00004000				Map	R	R	
00060000	00001000				Map	R	R	
00070000	00001000				Priv	RW	RW	
00080000	00001000				Priv	RW	RW	
00090000	00001000				Priv	RWE	RWE	
000A0000	00001000				Priv	RWE	RWE	
000B0000	00003000	api-ms-w			Priv	RW	RW	
000C0000	00001000				Priv	RW	RW	
000D0000	00001000				Priv	RW	RW	
00100000	00007000				Priv	RW	RW	
001D0000	00001000	MASTER_M		PE header	Imag	R	RWE	
001D1000	00007000	MASTER_M	.text	SFX,code	Imag	R	RWE	
001D8000	00003000	MASTER_M	.rdata	data,import	Imag	R	RWE	
001DB000	00001000	MASTER_M	.data		Imag	R	RWE	
001DC000	00001000	MASTER_M	.rsrc	resources	Imag	R	RWE	
001DD000	00001000	MASTER_M	.reloc		Imag	R	RWE	
001E0000	00067000				Map	R	R	WDe
00299000	00007000				Priv	RW	Guar	RW
004CC000	00001000				Priv	RW	Guar	RW
004CD000	00003000			stack of a	Priv	RW	Guar	RW
00530000	0001F000				Priv	RW		RW
00630000	0000E000				Map	R	R	
007B0000	00003000				Map	R	R	
007C0000	00181000				Map	R	R	
00950000	0012D000				Map	R	R	
70AE0000	00001000	api-ms-w		PE header	Imag	R	RWE	
70AE1000	00001000	api-ms-w	.text	code,export	Imag	R	RWE	
70AE2000	00001000	api-ms-w	.rsrc	data,resource	Imag	R	RWE	
70AF0000	00001000	api-ms_1		PE header	Imag	R	RWE	
70AF1000	00003000	api-ms_1	.text	code,export	Imag	R	RWE	
70AF4000	00001000	api-ms_1	.rsrc	data,resource	Imag	R	RWE	
70B00000	00001000	api-ms_2		PE header	Imag	R	RWE	
70B01000	00001000	api-ms_2	.text	code,export	Imag	R	RWE	
70B02000	00001000	api-ms_2	.rsrc	data,resource	Imag	R	RWE	
70B10000	00001000	api-ms_3		PE header	Imag	R	RWE	
70B11000	00001000	api-ms_3	.text	code,export	Imag	R	RWE	
70B12000	00001000	api-ms_3	.rsrc	data,resource	Imag	R	RWE	
70B20000	00001000	api-ms_4		PE header	Imag	R	RWE	
70B21000	00001000	api-ms_4	.text	code,export	Imag	R	RWE	
70B22000	00001000	api-ms_4	.rsrc	data,resource	Imag	R	RWE	
70B30000	00001000	api-ms_5		PE header	Imag	R	RWE	
70B31000	00001000	api-ms_5	.text	code,export	Imag	R	RWE	
70B32000	00001000	api-ms_5	.rsrc	data,resource	Imag	R	RWE	

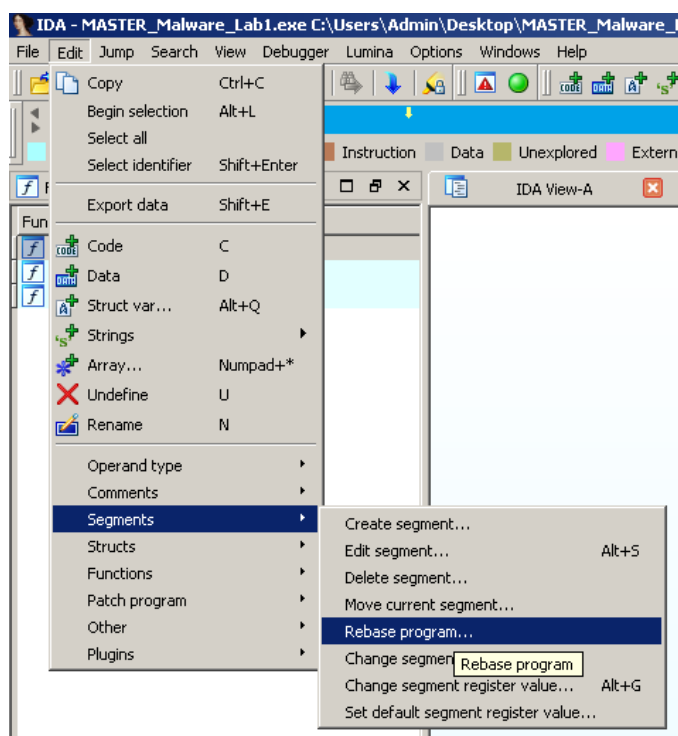
Captura 54: Olly Ventana Executable modules

Ahora verificamos en IDA y vemos que está en el espacio de memoria de 402FA0, por lo tanto al no coincidir los espacios de memoria, no podremos poner BP en los lugares correctos, ya que los diferentes segmentos de código en IDA indican una dirección de memoria que no es correcto en IDA. **Para solventar este problema utilizamos la opción Rebasing en IDA.**

```
.text:00402FA0
.text:00402FA0
.text:00402FA0 ; Attributes: bp-based frame
.text:00402FA0
.text:00402FA0 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00402FA0 _main proc near
.text:00402FA0
.text:00402FA0 var_27C= dword ptr -27Ch
.text:00402FA0 block= dword ptr -278h
.text:00402FA0 s= std::basic_string<char,std::char_traits<char>,std::allocator<char> > ptr -274h
.text:00402FA0 var_25C= dword ptr -25Ch
.text:00402FA0 stemp= std::basic_string<wchar_t,std::char_traits<wchar_t>,std::allocator<wchar_t> > ptr -25i
.text:00402FA0 screen= std::basic_string<char,std::char_traits<char>,std::allocator<char> > ptr -240h
.text:00402FA0 logger= std::thread ptr -228h
.text:00402FA0 screens= std::thread ptr -220h
.text:00402FA0 Filename= word ptr -218h
.text:00402FA0 var_10= dword ptr -10h
.text:00402FA0 var_C= dword ptr -0Ch
.text:00402FA0 var_4= dword ptr -4
.text:00402FA0 argc= dword ptr 8
.text:00402FA0 argv= dword ptr 0Ch
.text:00402FA0 envp= dword ptr 10h
.text:00402FA0
.text:00402FA0 ; FUNCTION CHUNK AT .text:00406EB0 SIZE 0000006E BYTES
.text:00402FA0 ; FUNCTION CHUNK AT .text:00406F23 SIZE 0000002A BYTES
.text:00402FA0
.text:00402FA0 ; __unwind { // __ehandler$_main
.text:00402FA0 push ebp
```

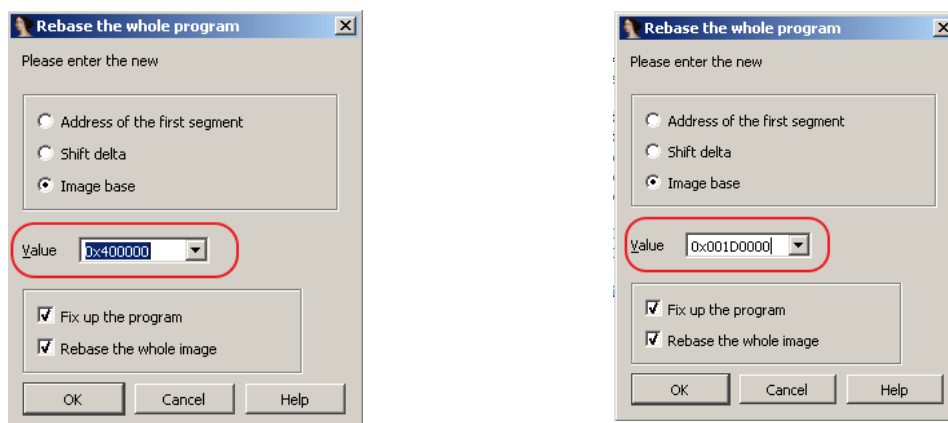
Captura 55: IDA, función main

En Ida abrimos Rebasing



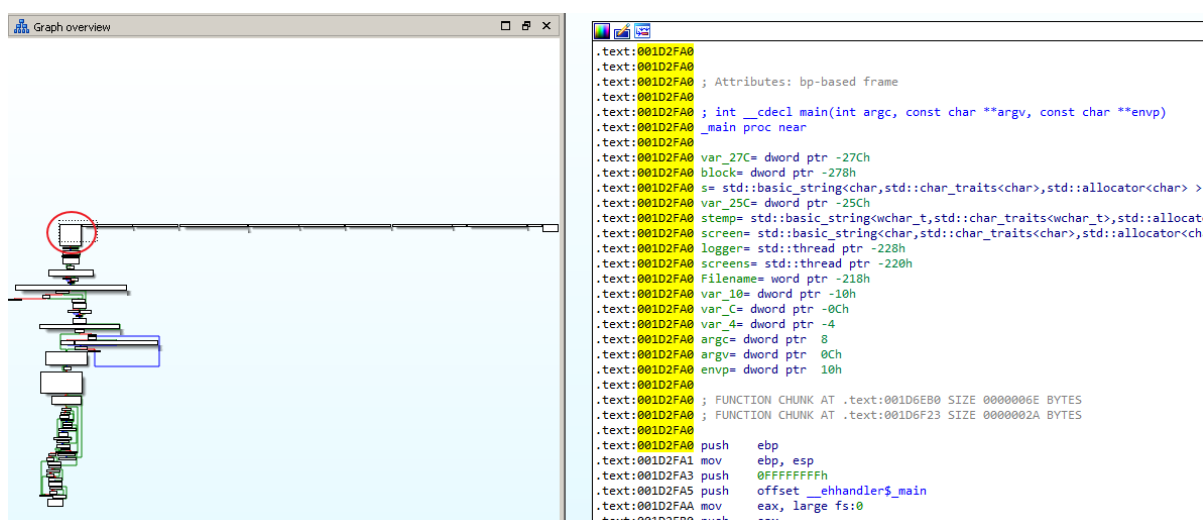
Captura 56: IDA, Reabse program

Se abre una ventana nueva con la dirección base actual, y cambiaremos ese valor por el obtenido en Olly "001D0000"



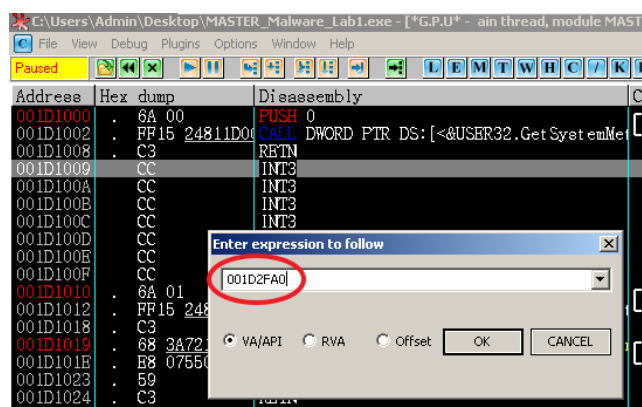
Captura 57: IDA, Rebase cambio de valor

Ahora que ambos programas están alineados, comencemos a poner BP:
BP al inicio en la función **main**, en mi caso la dirección de memoria es **001D2FA0**



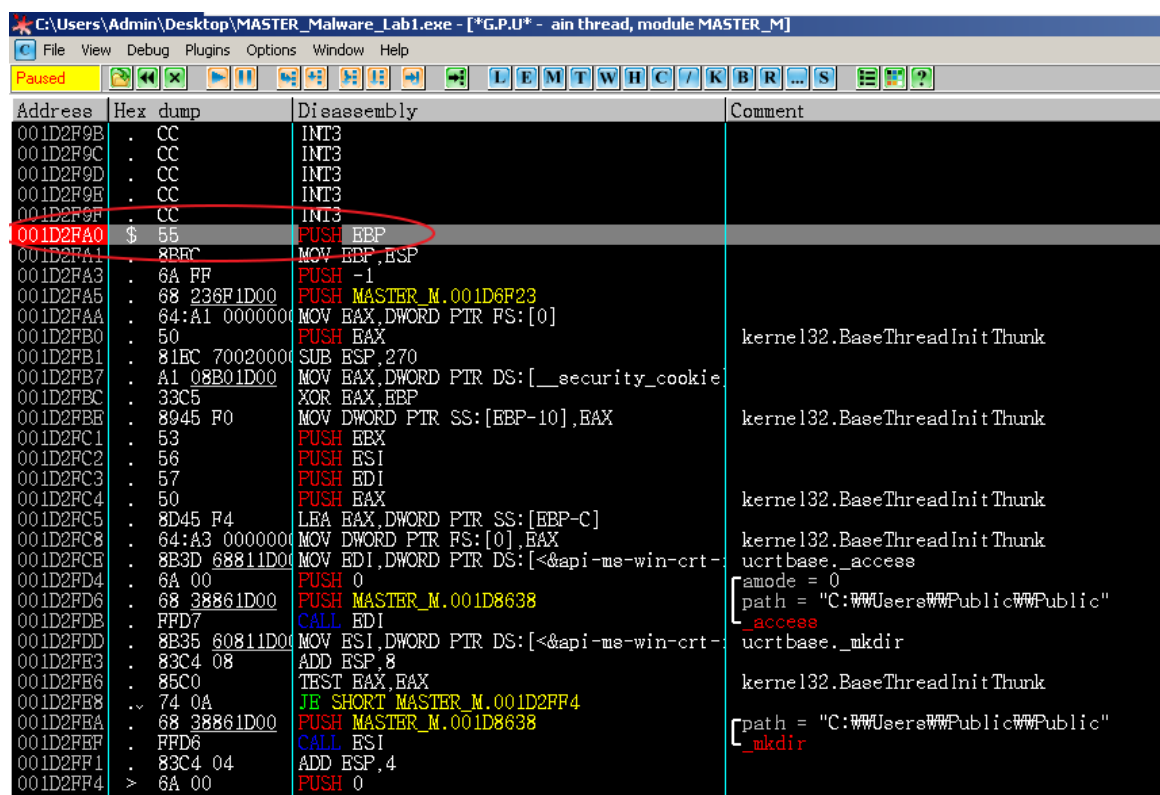
Captura 58: IDA, función main

Copiamos la dirección y vamos a OllyDebugger, pulsamos la **tecla CTRL+G** para poder introducir la dirección de memoria a la que queremos ir. También sería posible escribir el nombre de la función, en ese caso habríamos escrito main.



Captura 59: Olly, Follow Expression

Una vez situados en la función main, pulsamos la **tecla F2** para colocar un breakpoint.



Captura 60: Olly, BP en función main

Repetiremos esta tarea con las funciones que se identificaron y las partes de código de nuestro interés.

Video 2 y 3. Reversing dinámico avanzado muestra MASTER_Malware_Lab1.exe hash: 82B5DD6165DC0883F703C3D11BB30C12CF5F6E4BCEF8C8063F0260B0D862E2E8

Recorreremos el flujo de ejecución de toda la muestra para verificar nuestros análisis iniciales y anotar cualquier otro detalle que nos parezca de interés. Finalmente podemos crear una ficha con el resumen y hallazgos del análisis:

C:\Users\Public\Public
C:\Users\Public\Public\keylogs.txt
C:\Users\Public\Public\Screens\Screenshot.bmp
C:\Users\Public\Public\svchost.exe

Análisis muestra: MASTER_Malware_Lab1.exe

1. Hash:
82B5DD6165DC0883F703C3D11BB30C12CF5F6E4BCEF8C8063F0260B0D862E2E8
2. Nombre muestra: MASTER_Malware_Lab1.exe
3. Funcionalidad de la muestra:

- 3.1. La muestra no se encuentra empacada.
- 3.2. Crea carpetas, ficheros y se copia a si misma:
 - 3.2.1. C:\Users\Public\Public\keylogs.txt
 - 3.2.2. C:\Users\Public\Public\Screens\Screenshot.bmp
 - 3.2.3. C:\Users\Public\Public\svchost.exe
- 3.3. Ejecuta comando de sistema en una ventana oculta.
- 3.4. Persistencia mediante inserción de clave en el registro
 - 3.4.1. REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V
- 3.5. Info stealer
 - 3.5.1. Keylogger
 - 3.5.2. Screenshots
4. No dispone de funcionalidades de comunicación
5. La muestra no se encuentra en VT

Como podemos ver, ya disponemos de toda la información sobre el funcionamiento de la muestra.

EJERCICIO PRÁCTICO

Tarea 1:

Realice el reversing básico estático y dinámico de la muestra MASTER_Malware_Lab1.exe

Tarea 2:

Realice el reversing Avanzado estático y dinámico de la muestra MASTER_Malware_Lab1.exe

5.2. DROPPERS Y DOWNLOADERS

Recordemos que los Droppers son: Generalmente el malware no suele mostrarse como un elemento único, es decir, se trata de varias piezas contenidas unas dentro de otras que se van generando de forma dinámica. Las nuevas partes que lo componen pueden volcarse a memoria o a disco. Al elemento inicial que genera la siguiente parte se la conoce como Dropper y al acto de generación de ese siguiente elemento "Dropeo".

Algunas características a tener en cuenta sobre los Droppers son:

- El segundo payload va embebido en la sección resource de la cabecera PE o en una parte del binario no declarada como sección y a la que se le conoce como Overlay.
- Los nombres de las entradas en Resource se pueden cambiar fácilmente.
- Hay varias herramientas para analizar como PE Studio o pescanner.

- Las llamadas a API que podemos encontrar:
 - a. FindResource
 - b. LoadResource
 - c. SizeOfResource
 - d. LockResource
 - e. FreeResource

Los Downloaders siguen la misma filosofía que los Droppers, la única diferencia es que el siguiente elemento malicioso será descargado de una ubicación remota utilizando una conexión a internet. Es decir, el código malicioso básicamente descargará el malware real.

- Algunas de las llamadas API que podemos encontrar:
 - URLDownloadToFile
 - ShellExecute
 - WinExec

5.3. EMPAQUETADO Y OTRAS TÉCNICAS DE OFUSCACIÓN

En esta sección, comprenderemos que son los packers y otras técnicas de ofuscación, los distintos tipos que existen y cómo podemos analizar muestras que implementan estas medidas dedicadas a que sea más difícil tanto su detección como su análisis.

- Los autores de malware protegen las muestras a través del empaquetado.
- Los packers comprimen y/o encriptan la muestra.
- Dificulta el análisis (estático, desensamblado y código).
- Los packers también se usan para propósitos legítimos.
- Algunos packers conocidos: UPX, Aegis, Hyperion, Themida ...
- A veces los atacantes construyen sus propios packers (Modding).

Para aplicar un packer, básicamente se trata de añadir código que no es estrictamente malicioso, sino que sólo tiene por objeto ocultar el código malicioso. Así que el objetivo es esconder la carga útil del malware, tanto de los sistemas antivirus de la víctima como de los investigadores que analizan el archivo.

Veamos ahora algunos tipos:

Packers

Primero pensemos en el empaquetado como en la compresión del archivo, es decir se aplican diferentes algoritmos que comprimen su tamaño y lo hacen ilegible hasta que no es descomprimido.

El malware que usa esta técnica se desempaqueta a sí mismo en la memoria cuando se ejecuta el "archivo empaquetado". A veces esta técnica también se llama "compresión ejecutable". Este tipo de compresión se inventó para hacer los archivos más pequeños. Así los usuarios no tendrían que desempaquetarlos manualmente antes de que pudieran ser ejecutados. En esencia, se consigue hacer más difícil la ingeniería inversa, con el beneficio añadido de una huella más pequeña en la máquina infectada.

Existen Packers comerciales y otros creados por los desarrolladores de malware. Para los primeros en ocasiones existen programas que son capaces de detectarlos y en algunos casos de desempacarlos de forma automática, y para los segundos, no quedará más remedio que analizarlo en profundidad para poder extraer la carga maliciosa de memoria una vez que se ha desempaquetado y antes de que se ejecute.

Crypters

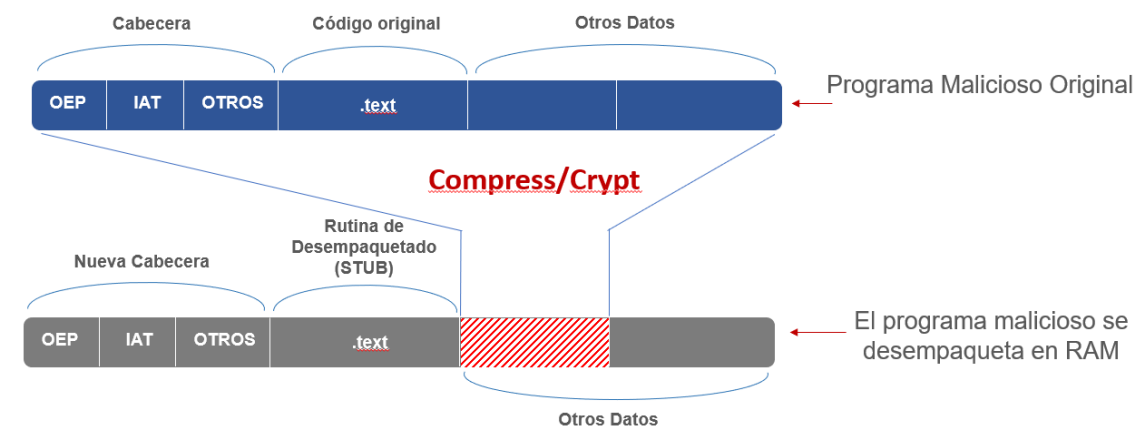
La técnica más básica para los crypters se llama ofuscación. La ofuscación también se usa a menudo en scripts, como javascripts y vbscripts. Pero la mayoría de las veces no es muy difícil evitarlos o eliminarlos. Los métodos más complejos utilizan una encriptación real. La mayoría de los crypters no sólo encriptan el archivo, sino que el software del crypters ofrece al usuario muchas otras opciones para hacer que el ejecutable oculto sea tan difícil de detectar por los proveedores de seguridad como sea posible.

Protectores

Un protector tiene por objeto evitar la manipulación y la ingeniería inversa de los programas. Los métodos utilizados pueden, y normalmente incluirán tanto el empaquetado como la encriptación. Esa combinación, más algunas características añadidas, hace lo que normalmente se denomina un protector. Así que un investigador se enfrentará a capas protectoras alrededor de la carga útil, haciendo difícil la ingeniería inversa.

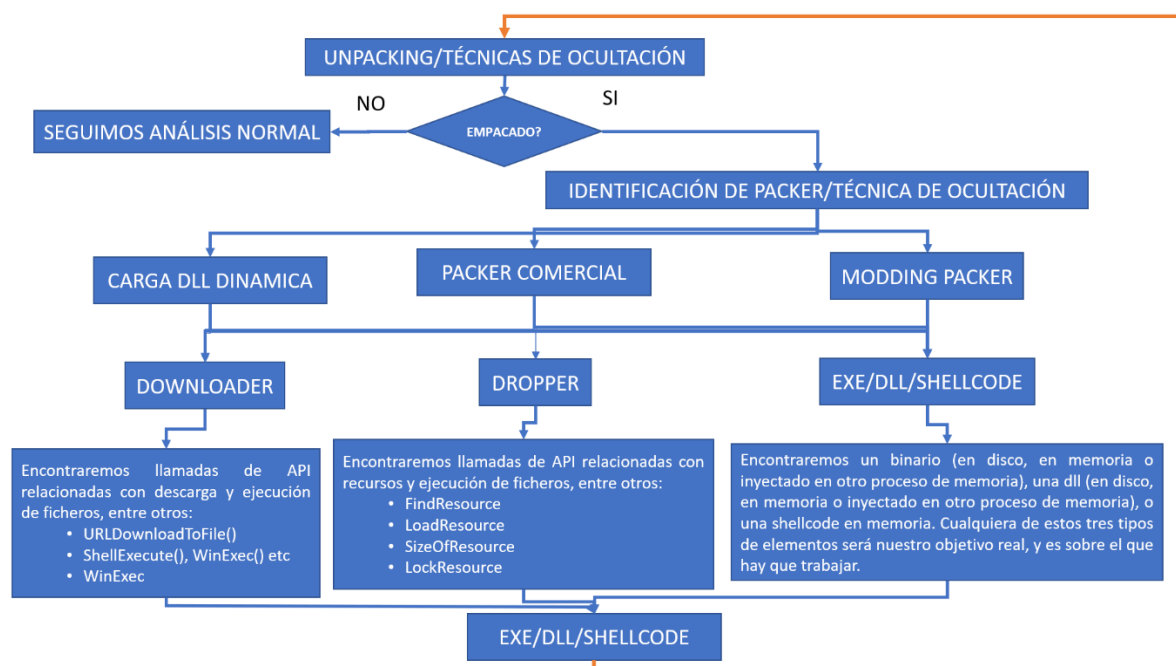
Un enfoque completamente diferente, que también cae bajo el paraguas de los protectores, es la virtualización de código, que utiliza un conjunto de instrucciones virtuales personalizadas y diferentes cada vez que lo utiliza para proteger su aplicación. De estos protectores existen versiones profesionales que se utilizan en la industria de los juegos contra la piratería. Pero la propia técnica también se ha abierto camino en el malware, más concretamente en el Ransomware. Lo que permite a los ransomware que no necesiten un servidor C&C para comunicar la clave de cifrado. La protección es tan eficiente que la clave de encriptación puede ser codificada en el propio Ransomware. Un ejemplo es Locky Bart que usa WProtect, un proyecto de virtualización de código abierto.

El desempacado es casi un arte por la complejidad que puede suponer y por las diferentes técnicas que pueden utilizarse. Trataremos de aplicar una metodología que nos sirva para la mayoría de los casos.



Captura 61: Esquema general de malware empaquetados

Seguiremos una metodología de análisis a la hora de estructurar el flujo de trabajo, dado que el malware implementa diferentes tipos y variantes de técnicas de evasión y ocultación. El siguiente es un esquema que simplifica las principales variantes que podemos encontrar. En ocasiones utilizarán más de una técnica combinada. Una vez entendamos como analizar cada una de estas técnicas por separado, podremos abordar análisis sobre muestras más complejas.



Captura 62: Metodología de trabajo con malware Empaquetados

IDENTIFICACIÓN DE PACKER/TÉCNICA DE OCULTACIÓN

Ya hablamos brevemente sobre la detección de Packers en capítulos anteriores, ahora ampliaremos la información relativa a las técnicas de evasión/ocultación de los payload maliciosos, ya que están directamente relacionados con los Packers.

• CARGA DLL DINÁMICA

Antes de continuar debemos hacer un inciso importante para hablar de Linking, dada la relación que tiene con este capítulo tal y como puede verse en la caja que hemos llamado en nuestro esquema "CARGA DLL DINÁMICA".

Sabemos que es muy útil a la hora de realizar un análisis de malware la lista de funciones API que importa el ejecutable. Estas funciones importadas se conectan con el ejecutable a través de linking. Hay tres formas de enlazar las librerías en el ejecutable.

- **Static Linking:** La librería se une de forma estática, todo el código está copiado en el ejecutable. Complica el diferenciar cuáles son librerías propias y cuáles copiadas ya que las PE Header no te ofrecen información al respecto.
- **Runtime Linking:** Es usada especialmente en malware cuando este es ofuscado o empaquetado. Este tipo sólo conecta con la librería cuando una función del ejecutable lo necesita. La carga de librerías no aparece en la IAT. Se cargan en tiempo de ejecución empleando `LoadLibrary()` / `GetProcAddress()`.
- **Dynamic Linking:** Es usada especialmente en malware cuando este es ofuscado o empaquetado. Este tipo se conecta, es decir carga la librería nada más comenzar la ejecución del malware. No podíamos ver inicialmente que librerías usaba.

Algunas de las API de llamada de sistema que veremos son:

- `LoadLibrary`
- `GetProcAddress`
- `FreeLibrary`

Veremos ejemplos prácticos que nos ayudarán a entender este proceso, ya que es frecuentemente usado por los desarrolladores de malware.

EJEMPLO

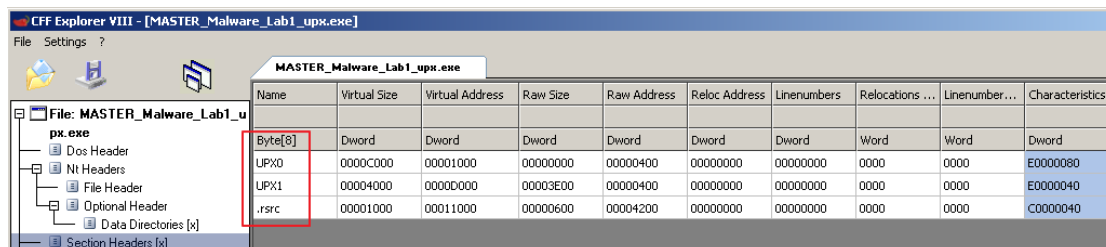
Veamos ahora en detalle los dos grandes grupos de Packers que nos encontraremos:

- **PACKER CONOCIDO**
 - Normalmente muy pocos strings y las que hay son ilegibles.

MSVCP140.dll	ax\$pa}\	+A I
VCRUNTIME140.dll	aRichq\	{#ld
LoadLibraryA	UPX0	Gaww
GetProcAddress	UPX1	y.&6
VirtualProtect	3.08	8,8o
VirtualAlloc	UPX!	jnWjn
VirtualFree	?<;1H	"ttJ
memset	VsP_B@	H@@\$
ExitProcess	VPod	StXJX)
	1\Qj	`\$5\$

Captura 63: Strings empacados

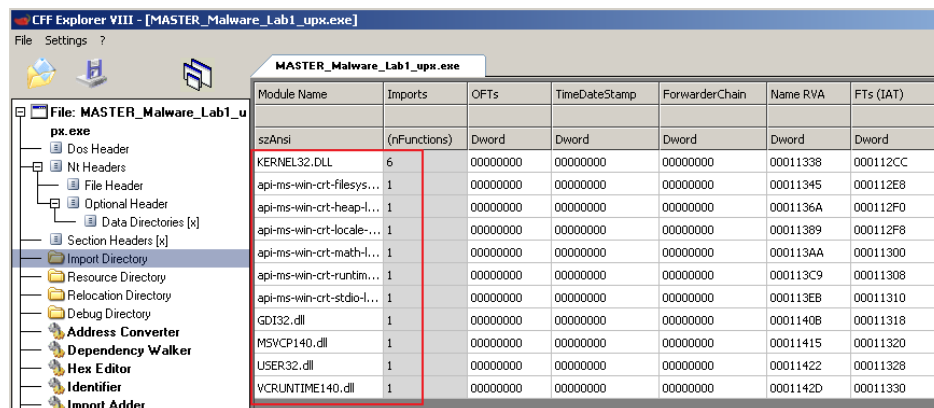
- Modificación de recursos o creación de nuevas secciones en los ficheros. El objetivo es incluir código ejecutable o para ser desempacado después.



Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations ...	Linenumber...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
UPX0	0000C000	00001000	00000000	00000400	00000000	00000000	0000	0000	E0000080
UPX1	00004000	0000D000	00003E00	00000400	00000000	00000000	0000	0000	E0000040
.rsrc	00001000	00011000	00000600	00004200	00000000	00000000	0000	0000	C0000040

Captura 64: Secciones de packer UPX

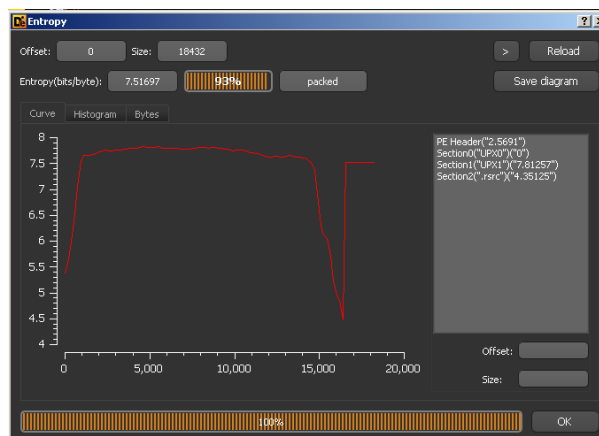
- Pocas funciones conocidas que se importan. Las que se importan serán básicamente las que necesita para realizar el proceso de desempacado, dropeo, carga de dll dinámica o descarga del siguiente fichero.



Module Name	Imports	OFs	TimeDateStamp	ForwarderChain	Name RVA	FTs (IAT)
szAnsi	(nFunctions)	Dword	Dword	Dword	Dword	Dword
KERNEL32.DLL	6	00000000	00000000	00000000	00011338	000112CC
api-ms-win-crt-filesys...	1	00000000	00000000	00000000	00011345	000112E8
api-ms-win-crt-heap-l...	1	00000000	00000000	00000000	0001136A	000112F0
api-ms-win-crt-locale...	1	00000000	00000000	00000000	00011389	000112F8
api-ms-win-crt-math-l...	1	00000000	00000000	00000000	000113AA	00011300
api-ms-win-crt-runtim...	1	00000000	00000000	00000000	000113C9	00011308
api-ms-win-crt-stdio-l...	1	00000000	00000000	00000000	000113EB	00011310
GD132.dll	1	00000000	00000000	00000000	0001140B	00011318
MSVCP140.dll	1	00000000	00000000	00000000	00011415	00011320
USER32.dll	1	00000000	00000000	00000000	00011422	00011328
VCRUNTIME140.dll	1	00000000	00000000	00000000	0001142D	00011330

Captura 65: Imports de malware empacado con UPX

- La entropía es alta: los valores de los bits son muy aleatorios. Como vimos anteriormente hay diferentes herramientas que nos ayudan a calcular la Entropía.



Captura 66: Entropía packer UPX

- La estructura de la IAT nos da pistas entre tamaños de disco y memoria. Como los Packers y otro tipo de técnicas comprimen la información del payload final, los tamaños en memoria y en disco no coinciden.

CFF Explorer VIII - [MASTER_Malware_Lab1_upx.exe]

File Settings ?

MASTER_Malware_Lab1_upx.exe

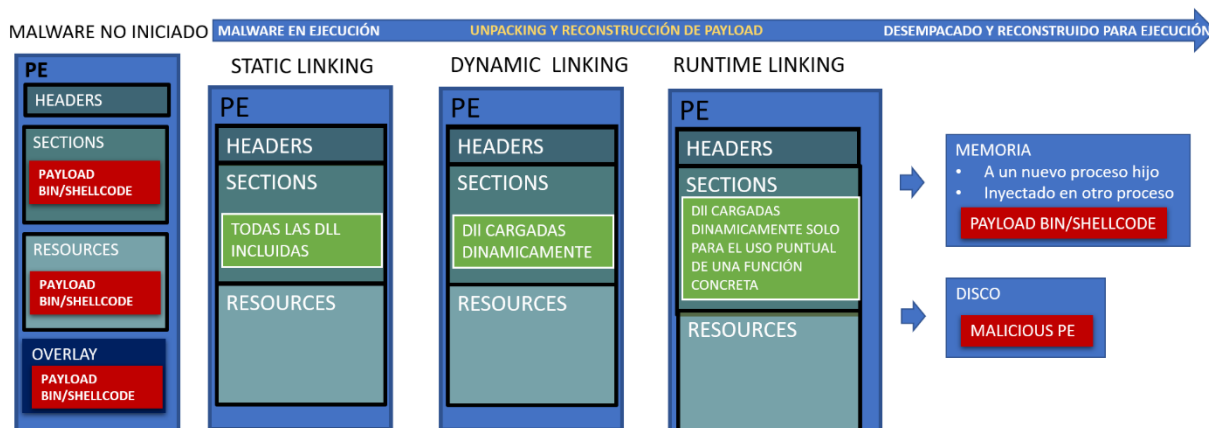
Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations ...	Linenumber...	Characteristics
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
UPX0	0000C000	00001000	00000000	00000400	00000000	00000000	0000	0000	E0000080
UPX1	00004000	0000D000	00003E00	00000400	00000000	00000000	0000	0000	E0000040
.rsrc	00001000	00011000	00000600	00004200	00000000	00000000	0000	0000	C0000040

Captura 67: Entropía packer UPX

Para desempacarlo podremos utilizar clientes UPX como vemos en el caso anterior siempre y cuando no este protegido por password, también podremos usar programas específicos para desempacar versiones concretas de packers conocidos. Generalmente lo más rápido y sencillo será realizar la ejecución de la muestra en el debugger y realizar la extracción del payload desempacado en disco o memoria.

- MODDING PACKER:** Si no hay un packer conocido (nos lo revelará alguna de las herramientas) requerirá un proceso manual. Esta puede ser una tarea tediosa y compleja, pero nos apoyaremos en llamadas a API que el malware ha de realizar para colocar el payload final en memoria y luego ejecutarlo, este es un punto común en todas las técnicas. Se prestará atención y se pondrán breakpoints en VirtualAlloc, VirtualProtect, WriteProcessMemory o CreateRemoteThread entre otras.

El siguiente gráfico presenta de forma esquemática el proceso de desempacado:



Captura 68: Esquema general de desempacado

1. Los payload maliciosos generalmente empacados, pueden esconderse en:

- SECTIONS:** Secciones del fichero, nuevas o preexistentes.
- RESOURCES:** En los recursos del fichero.
- OVERLAY:** En alguna parte del archivo habilitada para tal efecto y que no está declarada en las secciones.

2. Las DLL del fichero malicioso pueden cargarse de tres formas diferentes:

- STATIC LINKING:** Todas las dll están presentes inicialmente.
- DINAMYC LINKING:** Solo veremos algunas dll, generalmente las necesarias para el desempacado. El resto se cargarán durante el proceso de desempacado.
- RUNTIME LINKING:** Solo veremos algunas dll, generalmente las necesarias para el desempacado. El resto no se cargarán todas, sólo las necesarias para el flujo de ejecución actual, si a partir de una acción del malware se requiere esa carga, se hará en tiempo de ejecución. Un ejemplo sería el malware que captura credenciales bancos. Dichas funcionalidades no se activarán hasta que no se introduzca una url concreta en el navegador. Es en este punto donde se cargarán las dll necesarias para las tareas de captura de las credenciales.

A lo largo de todo el proceso de desempacado existen varios puntos donde podremos obtener el payload desempacado, podremos encontrarlo en disco o en memoria. Generalmente nos interesará obtener el payload antes de que sea ejecutado, ya que como veremos en secciones posteriores, el malware más complejo utilizará varias técnicas de inyección en procesos que dificultarán el análisis.

Payload en disco: Estaremos atentos y pondremos breakpoints durante la ejecución dinámica a funciones de escritura en disco como WriteFile.

Payload en memoria: En este caso, prestaremos atención a funciones que reservan espacio en memoria del propio proceso o de uno remoto, con el objetivo de escribir el código desempaado. Algunas de estas funciones son VirtualAlloc, VirtualAllocEx. También observaremos funciones que cambian el modo de protección de secciones de memoria para pasarlo de lectura a escritura o también a ejecución (RWX), principalmente ponemos foco en VirtualProtect y VirtualProtectEx. También debemos estar atentos a funciones como WriteProcessMemory, cuando el malware trate de escribir en memoria el payload decodificado.

Veamos ejemplos que nos ayuden a implementar la metodología expuesta.

En el siguiente ejemplo analizaremos una muestra para proceder a su desempaado. Lo haremos tanto de forma manual como de forma automática.

Análisis muestra: malware.exe

1. Hash:
49A48D4FF1B7973E55D5838F20107620ED808851231256BB94C85F6C80B8EBFC
2. Tipo Fichero: PE 32 bits
3. Nombre muestra: malware.exe
4. Funcionalidad de la muestra:
 - a. Ransomware Loki
 - b. Persistencia mediante clave de registro
 - c. Empacado Modding dropea shellcode, dll y MZ en memoria
 - d. Borra copias de seguridad de Windows ShadowCopies
 - i. vssadmin.exe Delete Shadows /All /Quiet
 - e. Borra ficheros ejecutando comandos de sistema
 - i. cmd.exe /C del /Q /F \"
 - f. La extensión de ficheros renombrados es .loki
 - a. Cifra un total de 178 extensiones de fichero diferentes.
 - g. Crea el fichero Wallet.dat
 - h. Los ficheros con la información de rescate son:
 - i. _Locky_recover_instructions.bmp
 - ii. _Locky_recover_instructions.txt
 - iii. _HELP_instructions.txt
5. Dispone de funcionalidades de comunicación
 - a. <http://31.41.44.130/submit.php>
 - b. <http://5.135.76.18/submit.php>
 - c. <http://93.170.131.108/submit.php>

6. Detecta los siguientes operativos para su ejecución:
 - a. Windows 10
 - b. Windows 2000
 - c. Windows 2003
 - d. Windows 2003 R2
 - e. Windows 7
 - f. Windows 8
 - g. Windows 8.1
 - h. Windows Server 2008
 - i. Windows Server 2008 R2
 - j. Windows Server 2012
 - k. Windows Server 2012 R2
 - l. Windows Server 2016 Technical Preview
 - m. Windows Vista
 - n. Windows XP
7. 62/67 motores de Virus Total detectan la muestra como maliciosa.

Videos Master_Unpack1,2,3,4

En el siguiente enlace puede encontrarse otro ejemplo de desempacado de la serie de vídeos de [OALabs](#), los cuales son muy recomendables: [Enlace desempacado Hermes Ransomware](#)

PROBLEMAS DEL DESEMAPCADO CON LA TABLA IAT

Recordemos que la IAT es es la tabla de importación de direcciones (en inglés import address table o IAT), que se usa como una tabla de búsqueda cuando la aplicación llama una función en un módulo diferente. Esto puede darse en forma de importación por ordinal o importación por nombre. Debido a que un programa compilado no puede conocer la ubicación en memoria de las bibliotecas de las que depende directamente, cada vez que se hace una llamada a la API es necesario un salto indirecto. Como el enlazador dinámico carga los módulos y los une, este escribe la dirección actuales en las ranuras IAT, de manera que apuntan a las ubicaciones de memoria correspondientes de la biblioteca de funciones.

Un ejecutable en Windows requiere algo más que el código del programador para poder funcionar. Para hacer casi cualquier cosa útil, necesita las API del sistema operativo. Ejemplos de actividades útiles podrían ser escribir en el disco, leer del disco, mostrar cualquier cosa en la pantalla, escuchar la entrada del teclado entre otras posibles funcionalidades que se obtienen de las API de sistema. Para hacer cualquiera de estas cosas, las APIs del sistema operativo Windows deben ser usadas y por lo tanto, el código de la API debe estar disponible para el programa. De hecho, incluso si un programador ejecuta un programa utilizando la biblioteca estándar de C en Windows, en última instancia, estas llamadas se convierten en llamadas a la API del SO de todos modos por el compilador. La forma en que el programa hace que

estas APIs estén disponibles es utilizando la Tabla de Direcciones de Importación para decirle al Administrador de Memoria del SO y al cargador de imágenes dónde están y qué APIs se necesitan en la IAT.

Para que el archivo se ejecute, debe tener una IAT válida, es posible que en el momento en que dumpeamos de memoria el código del malware usando por ejemplo OllyDbg, no volcamos una IAT válida. Por lo tanto, es necesario crear una IAT válida basada en el código del archivo y también en los nuevos desplazamientos de memoria. Para hacer esto, tenemos que obtener el nuevo OEP, y utilizarlo en uno de los diferentes programas de reconstrucción de importaciones como Scylla, PEUnmapper o Import REConstructor, y luego escanear la imagen en memoria para las importaciones. El programa calculará entonces una nueva IAT y permitirá "arreglar" el archivo volcado. Una vez hecho esto, el archivo debería poder ejecutarse en Windows porque la herramienta habrá establecido una IAT adecuada y el sistema operativo sabrá dónde encontrar las API adecuadas para que el programa funcione.

Para realizar todo el proceso anterior, debemos abordar una tarea en ocasiones realmente compleja y que requiere mucho tiempo, se trata de obtener el nuevo OEP. Este proceso es complejo, porque en función de cómo este desarrollado el packer tendremos que abordar la tarea de forma diferente.

Algunos tips que nos pueden ayudar son:

- Generalmente busca la limpieza de registros (pushad/popad) seguida de un salto (jmp).
- Colocar un punto de interrupción en las funciones de la API LoadLibrary y GetProcAddress. La mayoría de los empaquetadores utilizan estas funciones para cargar las bibliotecas necesarias y para arreglar la tabla de importación. La fijación de las importaciones se realiza principalmente al final del proceso de desempaquetado. Después de estar seguro de que las importaciones están fijadas, es posible que haya una pequeña rutina de descifrado que descifre el OEP y haga un salto a él. Pero, como se mencionado anteriormente, esto es específico del empaquetador y podría no funcionar para todos ellos.

En el siguiente enlace hay disponible otro vídeo muy útil para aprender diferentes formas de desempacar y reconstruir la Tabla de importaciones IAT: [Enlace](#)

En el siguiente enlace hay un artículo interesante sobre la reconstrucción de tabla IAT: [Enlace](#)

5.4. ANÁLISIS DE DOCUMENTOS MALICIOSOS

Consiste en realizar ingeniería inversa de un documento ofimático, extrayendo el código de script malicioso y/o el binario contenido en el caso de utilizarse esa técnica. Se analizarán sus instrucciones con el objetivo descubrir cuales son todas sus funcionalidades, las cuales general mente se centran en descargar y ejecutar el binario malicioso final o extraerlo del propio contenido del documento.

Los objetivos que se persiguen son:

- Extraer el payload malicioso final
- Obtener los IoC asociados, como podrían ser url de descarga del binario.
- Comprender las técnicas de auto-defensa e intentar hacer bypass de ellas.
- Identificar singularidades en el Código de script para posibles tareas de identificación y atribución.

Algunas herramientas útiles en este proceso:

- Oledump.py
- Olevba.py
- Msoffice-crypt
- Rtfobj.py
- pyxswf.py
- pdfid
- peepdf.py
- pdf-parser.py
- PDF Stream Dumper
- ViperMonkey
- Cyberchief
- scdbg
- shellcode2exe
- DIARIO

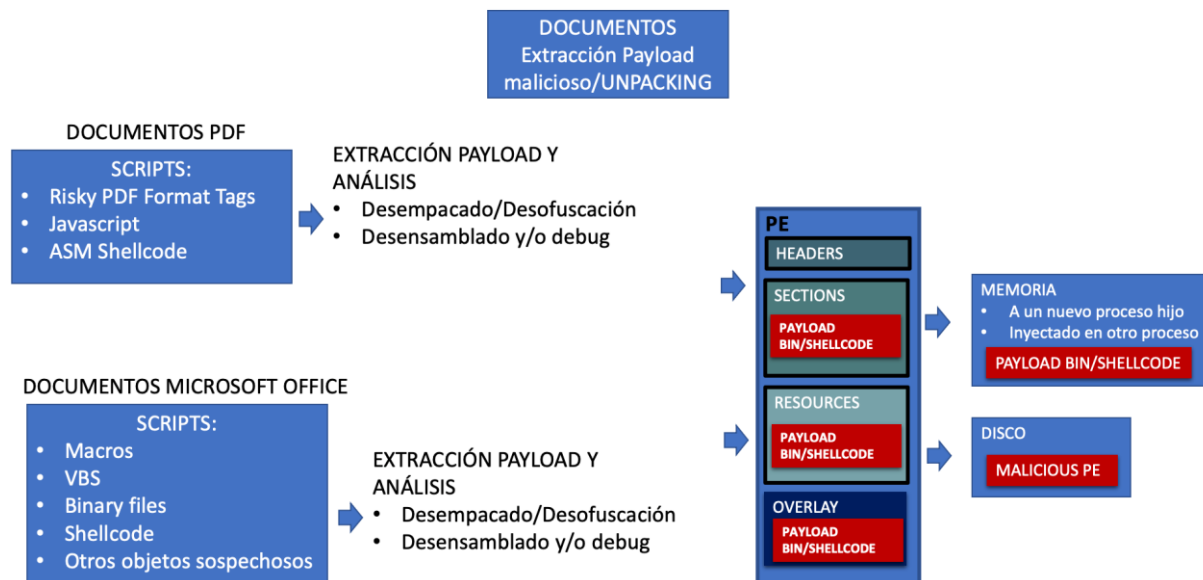
El proceso que seguiremos para analizar documentos maliciosos consta de las siguientes fases:

1. Localizar el código incrustado, como el shellcode, macros VBA, JavaScript u otros objetos sospechosos.
2. Extraer el código u objeto sospechoso del archivo.
3. Si es relevante, obtener y examinar el código malicioso.
4. Si es relevante, desofuscar, desensamblar y/o depurar el código de shell.
5. Obtener el payload malicioso y/o los indicadores asociados como por ejemplo una

url de descarga.

6. Entender los siguientes pasos en la cadena de infección.

7. Proceder con el análisis del payload y/o indicadores extraídos.



Captura 69: Esquema general de análisis de documentos maliciosos

Veamos a continuación las particularidades que podemos encontrar en cada formato de documentos maliciosos:

Documentos Microsoft office:

- Los archivos de documentos binarios soportados por Microsoft Office utilizan el formato OLE2 (también conocido como Almacenamiento Estructurado).
- Las secuencias SRP en documentos OLE2 a veces almacenan una versión en caché de código de macro anterior.
- Los documentos OOXML (.docx, .xism, etc.) soportados por MS Office usan compresión zip para almacenar contenidos. Las macros incrustadas en los archivos OOXML se almacenan dentro del archivo binario OLE2, que está dentro del archivo zip.
- Los documentos RTF no soportan macros, pero pueden contener otros archivos incrustados como objetos OLE1.

Documentos PDF:

Las Tags que debemos buscar por su posible implicación en tareas maliciosas son:

- /OpenAction y /AAA especifican la acción a ejecutar automáticamente.
- /JavaScript y /JS especifican el JavaScript a ejecutar.
- /GoTo cambia la vista a un destino especificado dentro del PDF o en otro archivo PDF.

- /Launch puede lanzar un programa o abrir un documento.
- /URI accede a un recurso por su URL.
- /SubmitForm y /GoToR pueden enviar datos a una URL.
- /RichMedia puede ser usado para incrustar Flash en un PDF.
- /ObjStm puede ocultar objetos dentro de un Object Stream.
- Debe tenerse en cuenta la ofuscación con códigos hexadecimales en los códigos de script embebidos.
- En ocasiones el código de script utilizará funciones de ofuscación que deberán ser analizadas de forma estática o dinámica.

Buscaremos url sospechosas, código javascript y otros tipos de objetos que pudieran estar contenidos.

Herramientas como **pdfstreamdumper** nos ayudarán a detectar el código posiblemente sospechoso y también a analizarlo.

Documentos RTF:

- Los documentos RTF no soportan macros, pero pueden contener otros archivos incrustados como objetos OLE1.

Buscaremos objetos OLE1 embebidos que generalmente contienen exploits y código de script.

Herramientas como rtfobj.exe nos ayudarán a detectar el código posiblemente sospechoso y también a extraerlo para su posterior análisis.

Generalmente encontraremos muchas técnicas de ofuscación diferentes para complicar la detección del documento y dificultar su análisis.

Veamos a continuación diferentes ejemplos de varios tipos de formato de documento.

Microsoft Office Excell

Análisis muestra:
9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm

8. Hash:

9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66

9. Tipo Fichero: Microsoft office XLSM

10. Nombre

muestra:

9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66
.xlsm

11. Funcionalidad de la muestra:

- a. Scripts Embebidos
- b. Auto open Scripts embebidos
- c. Downloader

- d. Incluye un payload VBS. El payload se encuentra ofuscado.
 - e. Ejecuta comandos PowerShell para descargar un ejecutable.
 - f. Escribe en disco C:\Admin\AppData\Local\Temp\oJGAx.exe
12. Dispone de funcionalidades de comunicación
- a. URL: <http://185.239.242.194/2391911/shawn.exe>
 - b. IP: 185.239.242.194
13. 27 motores de Virus Total detectan la muestra como maliciosa.
14. Durante el proceso de análisis la el siguiente payload no estaba disponible.

Puede ver el Vídeo Master_Excel

Microsoft Office PowerPoint

Análisis muestra:

5e3cb24d52ef617a4dca51ece6721a0745a17909f1b7fd4a54a59ceba7ea32b6.ppt

1. Hash:
5e3cb24d52ef617a4dca51ece6721a0745a17909f1b7fd4a54a59ceba7ea32b6
2. Tipo Fichero: Microsoft office PPT
3. Nombre muestra:
5e3cb24d52ef617a4dca51ece6721a0745a17909f1b7fd4a54a59ceba7ea32b6.ppt
4. Funcionalidad de la muestra:
 - a. Scripts Embebidos
 - b. Auto Close Scripts embebidos
 - c. Downloader
 - d. Incluye un payload VBS. El payload no se encuentra ofuscado.
 - e. Ejecuta script mediante mshta.exe para descargar un ejecutable.
 - f. Redirección a pastebin
5. Dispone de funcionalidades de comunicación
 - a. URL: <http://j.mp/asdaxasdaxasdadsddodkasodkaos>
 - b. URL: <https://pastebin.com/raw/v5yqXZYb>
 - c. DOMINIO: j.mp
6. 34 motores de Virus Total detectan la muestra como maliciosa.
7. Durante el proceso de análisis la el siguiente payload no estaba disponible en pastebin.

Puede ver el Vídeo Master_Powerpoint

Formato PDF

Análisis muestra:

0b3b3b22c8a6e3474150ea1cb8ab494413d3a641d475916114b8c4a94393f753.pdf

8. Hash:

0b3b3b22c8a6e3474150ea1cb8ab494413d3a641d475916114b8c4a94393f753

9. Tipo Fichero: PDF

10. Nombre muestra:

0b3b3b22c8a6e3474150ea1cb8ab494413d3a641d475916114b8c4a94393f753.pdf

11. Funcionalidad de la muestra:

a. Scripts Embebidos

b. Downloader

c. Contiene exploits

i. cve-2007-5659

ii. cve-2008-2992

iii. cve-2009-0927

d. Incluye un payload javascript. El payload se encuentra ofuscado.

e. Ejecuta un shellcode que mediante la invocación de la función URLDownloadToFile perteneciente a la librería urlmon.dll, realiza la descarga de un fichero.

f. Escribe en disco C:\Admin\AppData\Local\Temp*.exe

12. Dispone de funcionalidades de comunicación

a. URL: <http://juvitec.net/css/load.php?e=2>

b. DOMINIO: juvitec.net

13. 51 motores de Virus Total detectan la muestra como maliciosa.

14. Durante el proceso de análisis la el siguiente payload no estaba disponible.

Puede ver el Vídeo Master_pdf

Documentos RTF

Análisis muestra:

9da7d051bdf010d86a461d407c30d40f57d495fb6a5d22735ff792724bc3831e.rtf

15. Hash:

9da7d051bdf010d86a461d407c30d40f57d495fb6a5d22735ff792724bc3831e

16. Tipo Fichero: RTF

17. Nombre muestra:

9da7d051bdf010d86a461d407c30d40f57d495fb6a5d22735ff792724bc3831e.rtf

18. Funcionalidad de la muestra:

a. Objetos ole Embebidos

b. Contiene exploit cve-2017-8570

c. Incluye un payload de script SCT. El payload de código VBS contenido en el script del fichero SCT se encuentra ofuscado.

d. Ejecuta comandos PowerShell para descargar un ejecutable.

- e. Downloader
- 19. Dispone de funcionalidades de comunicación
 - a. URL: <http://eddyholdingshuttle.co.za/i/xpertee.exe>
 - b. DOMINIO: eddyholdingshuttle.co.za
- 20. 30 motores de Virus Total detectan la muestra como maliciosa.
- 21. Durante el proceso de análisis la el siguiente payload no estaba disponible.

Puede ver el Vídeo Master_rtf1

Análisis muestra:

2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf

1. Hash:
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
2. Tipo Fichero: RTF
3. Nombre muestra:
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
.rtf
4. Funcionalidad de la muestra:
 - a. Objetos ole Embebidos
 - b. Contiene shellcode
 - c. Ejecuta shellcode para descargar un ejecutable.
 - d. Downloader
 - e. Escribe en disco C:\Users\Public\908.exe
 - f. Utiliza una url oculta mediante el servicio bit.ly
 - g. CVE-2017-11882
5. Dispone de funcionalidades de comunicación
 - a. URL: <http://bit.ly/34vzFIU>
 - b. IP: <http://185.172.110.244/rwab/pacbebin.txt>
6. 30 motores de Virus Total detectan la muestra como maliciosa.
7. Durante el proceso de análisis la el siguiente payload no estaba disponible.

Puede ver el Vídeo Master_rtf2

5.5. ANÁLISIS DLL

Una biblioteca de enlace dinámico o más comúnmente DLL (sigla en inglés de dynamic-link library) es el término con el que se refiere a los archivos con código ejecutable que se cargan bajo demanda de un programa por parte del sistema operativo. Esta denominación es exclusiva a los sistemas operativos Windows siendo ".dll" la extensión con la que se identifican estos ficheros, aunque el concepto existe en prácticamente todos los sistemas operativos modernos.

La principal diferencia entre las DLL y los archivos EXE está en el proceso de carga.

Fichero EXE

Cuando hacemos doble clic en el archivo:

1. El Cargador de Imágenes PE de Windows analiza los encabezados PE
2. Realiza comprobaciones de integridad
3. Configura todas las secciones de memoria del archivo. Estas secciones de memoria incluyen la sección .text o .code, la sección .data, la sección .rdata, y otras relacionadas con recursos. En ocasiones añaden otras secciones no estándar que pueden esconder contenido malicioso. Estas secciones, cuando se mapean (cargan) en la RAM, pueden no estar mapeadas en la misma posición en la que estaban en el archivo PE del disco.

Fichero DLL

Los archivos DLL son una colección de funcionalidades que otros programas pueden utilizar.

Los programas cargan directamente las DLLs Utilizando las llamadas de la API LoadLibrary o LoadLibraryEx, seguidas de GetProcAddress para localizar la dirección de una función específica en la DLL.

Los archivos DLL en esencia tienen las mismas características (múltiples secciones, encabezado PE, etc.) excepto que el cargador PE de Windows no los cargará directamente para un usuario, sino que deberá ser invocado a través de sus funciones exportadas utilizando otro programa. Es por esto que vemos LoadLibrary y GetProcAddress utilizados en malware que implementa su propio cargador personalizado de DLLs, las cuales no fueron especificadas en el encabezado PE.

Si utilizamos rundll32.exe como cargador de la dll, básicamente lo que haremos es pasarle el nombre de la dll y la función exportada que estamos invocando.

El análisis estático de la DLL se realizará de forma habitual, y en esta ocasión prestaremos especial atención a las funciones "Exports". Sin embargo, para el análisis dinámico deberemos tener en cuenta algunas cuestiones dado que no es la propia dll la que analizamos, sino el binario que usaremos para lanzarla como puede ser rundll32.exe.

Existen diferentes formas de abordar el análisis, veamos a continuación una de ellas:

EN LA PRACTICA:

- Localizaremos las funciones exportadas mediante análisis estático con CFF explorer, IDA u otra herramienta en la que pueda ver los imports del fichero.
- Utilizaremos rundll32.exe como el binario que cargará la DLL, como argumentos se facilitan el nombre de la dll y el nombre o el ordinal numérico de la función exportada que hemos identificado en el punto interior. Lo que haremos será cargar en el debugger rundll32.exe en la configuración del debugger configuramos como argumento: rundll32.exe nombre_dll, dll, nombre_función_exportada /ordinal numérico de la función. Otra de las formas de analizar una dll es utilizar los cargadores de dll que incorporan los propios debuggers.
- En Olly o cualquier otro debugger configuraremos el Debugger para que pare en cada carga de DLL.
- Ejecutaremos hasta que llegue la carga de nuestra DLL maliciosa y obtendremos su dirección base por si es necesario realizar "rebased" en IDA.
- Procederemos con el análisis estático y dinámico en profundidad con la metodología utilizada habitualmente.

En el siguiente enlace se encuentra un artículo muy interesante que muestra tres aproximaciones diferentes para analizar una DLL de forma práctica: [Enlace](#)

5.6. ANÁLISIS .NET

Consiste en realizar ingeniería inversa de un binario utilizando tecnología .NET. Al realizarse la interpretación del binario por parte de la MV de .NET, no podremos utilizar las herramientas habituales como IDA u Olly. En este caso utilizaremos descompiladores como DnSpy que nos ayudará tanto a analizar el código de forma estática como dinámica.

Es común que se utilicen una amplia gama de ofuscadores y packers de código en una o más capas para ocultar el código fuente, tanto comerciales como modding.

Procederemos desofuscando/desempacando el binario utilizando herramientas como De4dot. Se analizarán sus instrucciones con el objetivo descubrir cuales son todas sus funcionalidades.

Los objetivos que se persiguen son:

- Extraer el payload malicioso final desofuscado/desempacado

- Obtener los IoC asociados, como podrían ser url de descarga del binario.
- Comprender las técnicas de auto-defensa e intentar hacer bypass de ellas.
- Identificar singularidades en el Código de script para posibles tareas de identificación y atribución.

Algunas herramientas útiles en este proceso:

- DnSpy
- De4Dot
- Megadumper

El proceso que seguiremos para analizar documentos maliciosos consta de las siguientes fases:

1. Identificar que se trata de un fichero desarrollado en tecnología .NET.

Para identificar ficheros que utilizan tecnología .NET utilizaremos el programa **ExeInfo** que nos indicará directamente si es .NET y detectará una amplia gama de packers. También utilizaremos **CFF Explorer** para examinar estáticamente el fichero y ver en detalle sus cabeceras, secciones, recursos, etc.

2. Desempacar el código u objeto sospechoso del archivo en el caso de ser necesario.

Es habitual encontrar el malware en .NET empacado utilizando herramientas como Intelillock, y será necesario utilizar el programa **De4Dot**, el cual nos va a permitir detectar packer y desempacar el fichero en muchas ocasiones.

En otras deberemos buscar un unpacker específico, y en los casos en los que esto no sea posible, deberemos hacer análisis dinámico con el debugger DnSpy.

Finalmente, incluso habrá ocasiones en las que nada de lo anterior sea posible y por lo tanto lo más rápido y sencillo será ejecutar la muestra y realizar un dumpeado del proceso utilizando la herramienta **Megadumper**.

3. Examinar el código malicioso de forma estática y dinámica.

Mediante el debugger DnSpy podremos ver y depurar el código escrito utilizando tecnologías .NET.

Prestaremos atención a los recursos que se incluyen, ya que suele ser lugar habitual para esconder Payloads cifrados. Trataremos de obtenerlo directamente del recurso y desempacarlo/desofuscarlo, pero también puede ser necesario realizar el debugging dinámico para desempacarla.

Buscaremos el módulo main e iremos añadiendo a "Análisis" las variables o funciones que nos parezcan de interés para comprender su uso y funcionamiento.

4. Obtener el payload malicioso final y/o los indicadores asociados como por ejemplo una url de descarga y entender los siguientes pasos en la cadena de infección.

Una vez extraídos todos los payloads y realizado su análisis, se anotarán datos relevantes como el tipo de empaçado, posibles IoC como URL o dominios y finalmente se anotarán todas las funcionalidades observadas.

EJEMPLO

Análisis muestra: NET_MALWARE_master_1.exe

1. Hash:
6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7
2. Tipo Fichero: .NET
3. Empacado IntelliLock v.1.5.x.0
4. Nombre muestra: NET_MALWARE_master_1.exe
5. Ficheros obtenidos del análisis:
 - a. Desempacada:
4EF28D2DBD5BC4870830B2BDC64FC35D651DC21578D873EFB8156B25018C98E6
 - b. Payload extraído:
02DFB4A65D7D9A44496C9C905C0C62CD64937BA2BBF0B985DEDA805D313A9F5B
6. Funcionalidad de la muestra:
 - a. Payload embebido en recursos del fichero (Dropper)
 - b. Contiene fichero
 - c. Descifra payload y ejecuta fichero obtenido.
 - d. Incorpora funcionalidades de Keylogger y exfiltración de datos
 - e. Incorpora capacidades de persistencia mediante escritura en clave de registro Software\Microsoft\Windows\CurrentVersion\Run
 - f. Nombre del proceso "svchost.exe"
7. Dispone de funcionalidades de comunicación
 - a. DOMINIO: dr187187.ddns.net
 - b. PUERTO: 22
8. 48 motores de Virus Total detectan la muestra como maliciosa.
9. Atribución: Doctor Western
10. Origen Actor (Árabe)
11. String interesantes para clasificación:
 - a. "# تم الاختراق من قبل دكتور الغربية" à Hackeado por Doctor Western #

Puede ver el Vídeo Master_net

5.7. TÉCNICAS DE PROCESS INJECTION

La inyección de procesos es una técnica de evasión muy común que el malware emplea con frecuencia, consiste en ejecutar un código malicioso en el espacio de direcciones de otro proceso. Se puede hacer inyectando directamente el código en otro proceso, o forzando a un DLL a ser cargado en otro proceso. Mediante esta técnica, será más difícil de detectar el malware, ya que examinando un proceso de forma básica con el explorador de tareas no veremos nada. En otras ocasiones estas técnicas se usan también para obtener persistencia.

Hay un gran número de técnicas de inyección de proceso, veremos las más representativas.

5.7.1. PE INJECTION

El malware puede copiar su código malicioso en un proceso abierto y hacer que se ejecute, ya sea a través de una shellcode, o llamando a la función `CreateRemoteThread`.

Se trata de una técnica sigilosa al no dropear ningún fichero a disco. Los pasos a seguir para realizar la inyección son:

1. Obtiene el HANDLER del proceso remoto mediante la función `OpenProces`.
2. Asigna memoria en el espacio de direcciones del proceso remoto para los datos inyectados mediante el uso de la función `VirtualAllocEx` (en este caso utiliza `VirtualAllocEx` en vez de `VirtualAlloc`, porque la reserva de memoria no se hace en el propio proceso, sino que se hace sobre un proceso remoto).
3. Escribe el código malicioso llamando a `WriteProcessMemory`, la cual escribe datos en un área de memoria en un proceso específico.
4. Iniciar la copia remota de `ThreadFunc` a través de la llamada a `CreateRemoteThread`.
5. Espera hasta que el hilo remoto termine con la función `WaitForSingleObject`.
6. Recuperar el resultado del proceso remoto con las llamadas a `ReadProcessMemory` o `GetExitCodeThread`.
7. Libera la memoria asignada con la función `VirtualFreeEx`.
8. Cierra los "handles" con la función `CloseHandle`.

Debe tenerse en cuenta que, en ocasiones tras la inyección de un PE en otro proceso, tendrá una nueva dirección base que es impredecible, requiriendo que vuelva a calcular dinámicamente las direcciones fijas de su PE. El malware debe encontrar su dirección de tabla de reubicación en el proceso inyectado, y resolver las direcciones absolutas de la imagen copiada mediante un bucle a través de sus descriptores de reubicación.

Al analizar este tipo de malware veremos dos bucles anidados antes de una llamada a `CreateRemoteThread`. Suelen utilizarse para ajustar su tabla de reubicación antes de las llamadas a `WriteProcessMemory` y `CreateRemoteThread` para hacer que se ejecute.

La aproximación que utilizaremos para extraer el PE malicioso que se inyectará en memoria, será:

1. Poner un breakpoint en `WriteProcessMemory`
2. Vemos en la pila el argumento `Buffer` que contiene la dirección de memoria donde se aloja el PE malicioso que será inyectado.
3. Dumpearemos la sección de memoria del argumento `Buffer`.
4. Pasaremos al análisis del Payload obtenido.

EJEMPLO

Análisis muestra: `malware_process_injection_1.exe`

1. Hash:
7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be
2. Tipo Fichero: PE 32 bits
3. Empacado Modding
4. Nombre muestra: `malware_process_injection_1.exe`
5. Ficheros obtenidos del análisis:
 - a. Payload extraído:
5F0AD6F81B73B78BB9AD29F357B53A60165101C11217FB495992A7BEC4C
F1F60
6. Funcionalidad de la muestra:
 - a. Inyección en proceso
 - b. Cambia la configuración de las zonas de Internet
 - c. Ejecuta el código inyectado en otro proceso
 - d. Cambia el valor de auto ejecución en el registro
 - e. La aplicación fue inyectada por otro proceso
 - f. Crea archivos en el directorio del programa
 - g. Inicia Internet Explorer
 - h. Lee la configuración de Internet Explorer
7. Dispone de funcionalidades de comunicación

Puede ver el Vídeo `Master_Process_Injection`

5.7.2. DLL INJECTION

Esta técnica consiste en inyectar malware en el espacio de direcciones virtuales de otro proceso mediante una DLL. Y hace que el proceso remoto la cargue creando un hilo remoto en el proceso de destino. Generalmente utilizamos la función LoadLibrary para cargar dll, pero si queremos hacer esto en un proceso remoto, se recurre a la función CreateRemoteThread, la cual crea un hilo que se ejecuta en el espacio de direcciones virtuales de otro proceso.

Los pasos a seguir para realizar la inyección son:

1. Buscar el proceso de destino sobre el que se inyectará, para ello utilizaremos las funciones de API CreateToolhelp32Snapshot, Process32First y Process32Next para iterar por los procesos abiertos mediante la implementación de un bucle.
2. Obtiene el HANDLER del proceso remoto mediante la función OpenProces.
3. Asigna memoria en el espacio de direcciones del proceso remoto para los datos inyectados mediante el uso de la función VirtualAllocEx.
4. Escribe el código malicioso llamando a WriteProcessMemory, la cual escribe datos en un área de memoria en un proceso específico.
5. Asigna la DLL maliciosa al proceso remoto a través de CreateRemoteThread & LoadLibrary.
6. Espera hasta que el hilo remoto termine (WaitForSingleObject).
7. Recuperar el código de salida del hilo remoto (GetExitCodeThread).
8. Liberar la memoria asignada (VirtualFreeEx).
9. Descargue la DLL del proceso remoto a través de CreateRemoteThread y FreeLibrary.
10. Espera hasta que el hilo termine (WaitForSingleObject).



Captura 70: Flujo DLL Injection

5.7.3. PROCESS HOLLOWING

Esta técnica consiste en la sustituir el código legítimo de un proceso por código malicioso. El malware la utiliza como técnica de evasión, ya que permite para inadvertido al estar sustituyendo el contenido de un proceso benigno.

Cuando un malware se inicia, llevará predefinidos el nombre de uno o varios procesos a inyectar (como `explorer.exe`) e iniciará una nueva instancia del mismo, en un estado suspendido, ya que de esta forma es seguro modificarlo. Seguidamente borrará totalmente el código del proceso lícito, ampliará la memoria si es necesario y copiará su propio código en su interior. A continuación, se reajustarán la dirección del punto de entrada y la dirección base. Finalmente reanudará el proceso que estaba en modo suspendido con su nuevo contenido malicioso.

Podremos encontrar esta técnica implementada tanto en binarios como en código de script contenido en documentos.

Los pasos a seguir para realizar el process hollowing son:

1. Se crea un proceso en estado suspendido mediante la API `CreateProcess` o alguna de sus variantes como `ZwCreateProcess` o `NtCreateProcess`. Se hará pasando el valor "Create_Suspended" en el parámetro `dwCreationFlags`. Una vez creado el proceso en estado suspendido crearemos una nueva sección ejecutable. No estará ligada a ningún proceso. Esto se puede hacer usando la función `ZwCreateSection`.
2. Obtenemos información sobre el proceso remoto. En primer lugar, se debe localizar la dirección base de la imagen de destino. Esto puede hacerse consultando el proceso con `NtQueryProcessInformation` para adquirir la dirección del entorno del proceso de bloque (PEB). El PEB se lee usando `ReadProcessMemory`. Adicionalmente obtenemos el contexto del proceso víctima y lo leemos mediante las llamadas a `GetThreadContext` y `ReadProcessMemory`. Es decir, si hemos creado el proceso suspendido `svchost.exe`, lo que hacemos es vaciar su contenido en memoria.
3. La memoria del proceso legítimo es des-mapeada (vacuada) mediante la función `NtUnmapViewOfSection` o `ZwUnmapViewOfSection`. Es decir, si hemos creado el proceso suspendido "svchost.exe", lo que hacemos es vaciar su contenido en memoria. Se le pasa el handler del proceso actual usando la función `GetCurrentProcess`.
4. Finalmente, sobre el proceso víctima reservamos espacio en memoria y escribimos el código malicioso mediante las funciones `VirtualAlloc` o `VirtualAllocEx` y `WriteProcessMemory` respectivamente.
5. Una vez copiado y reconstruido el proceso pasamos a indicar la nueva dirección OEP del proceso malicioso. Con `GetThreadContext` obtenemos esa dirección y la escribimos en el proceso víctima con `SetThreadContext`. Debemos tener en cuenta variantes como `Wow64SetThreadContext` para entornos de 64 bits.
6. Después de parchear el punto de entrada del proceso destino tenemos que reanudar el hilo. Esto se puede lograr usando la función `ResumeThread`.



Captura 70: Flujo Process Hollowing

Podremos encontrar algunas variantes para ser más sigiloso como la creación y mapeo de secciones para evitar el uso de `WriteProcessMemory`. Para evitar el uso de `SetThreadContext` se modificará el entry point.

La aproximación que utilizaremos para extraer el PE malicioso que sustituirá el código lícito del proceso que levantará el malware es:

1. La primera llamada a la API que se realizará a `CreateProcess`. El parámetro `CreationFlags` se pondrá en `CREATE_SUSPENDED`. El valor de retorno de esta llamada será un "handle" del proceso, que se pasará como parámetro a las siguientes llamadas. Aquí sería interesante poner un breakpoint.

2. El próximo Breackpoint lo pondremos en la llamada a WriteProcessMemory. Esto le permitirá ver de qué parte de la memoria del proceso padre se está copiando el código, lo que puede ser útil para obtener el payload desempaquetado o no. Es decir, no debemos esperar que cada parte del proceso se realice de forma atómica y sencilla, generalmente utilizarán varias llamadas dentro del proceso padre para terminar de desempaquetar y tener listo el código que será inyectado. En este punto ya podremos dumper el contenido del payload malicioso que se va a copiar al proceso objetivo. Para ellos una vez que estemos parados en la función WriteProcessMemory veremos que el tercer argumento es lpBuffer, podemos hacer un "follow in dump" en Olly para luego dumperlo. En ocasiones con esto será más que suficiente para seguir analizando el malware, pero en otras ocasiones el binario dropeado incluye Calls dinámicas y debemos poder encontrar una forma de seguir debuguando el proceso hijo que se ha creado.
3. Una vez que el código malicioso ha sustituido al del proceso víctima, es necesario obtener el nuevo punto de entrada. Esto se consigue usando las llamadas API GetThreadContext para leer la información en un buffer y poder modificarla y SetThreadContext. Para escribir en el nuevo proceso los cambios que realizamos en los datos que recibimos y manipulamos gracias a GetThreadContext. El parámetro pContext de SetThreadContext es un puntero al buffer que contiene la nueva estructura del Contexto y ahí encontramos el OEP del nuevo proceso malicioso.
4. El último Breackpoint en el que pararemos será a la llamada ResumeThread. Una vez hecha esta llamada, el nuevo proceso se reanudará desde el punto de entrada definido en la estructura de contexto anteriormente mencionada.

IMPORTANTE: No podrás atachearte al proceso con un debugger con Olly 1.01, porque se creó en modo suspendido, y hasta que se reanude su ejecución llamando a ResumeThread. Hay una técnica que nos ayuda a solventar este problema, y que implica varios pasos:

1. Encuentra el punto de entrada del ejecutable y toma nota de los dos primeros bytes en esa dirección. Para ello una vez paramos en el breakpoint de SetThreadContext, observamos el parámetro pContext de SetThreadContext, el cual es un puntero al buffer que contiene la nueva estructura del Contexto. Lo que haremos será un "follow in dump" del valor de pContext y en el dumpeo seleccionamos los dos primeros bytes que contienen el nuevo OEP del proceso víctima. Tomaremos nota de ellos.

2. Reemplazamos en memoria, utilizando el process hacker esos bytes con los **opcode EB FE**, que son la representación de 2 bytes de **jmp \$**. Se trata de un bucle infinito ya que la instrucción salta a sí misma. También podemos hacerlo con la interrupción INT3.
3. Reanudamos el proceso en Olly.
4. Ahora que el proceso está ejecutándose en un bucle infinito sin poder avanzar, podemos abrir una nueva instancia del debugger y atacharnos a él, suspenderlo pausando el Olly, restaurar los dos bytes originales y ejecutarlo nuevamente.
5. Debemos tener en cuenta que un proceso malicioso puede crear hilos adicionales en el proceso víctima, y esos pueden bypassar esta técnica si no hemos realizado el mismo proceso con esos otros hilos.
6. Hay herramientas como **PE_Sieve** que nos ayudarán a extraer el proceso malicioso de forma más automática, pero que no serán de tanta utilidad si es necesario un análisis dinámico del proceso víctima.

EJEMPLO

Análisis muestra: malware_process_injection_1.exe

1. Hash:
DC96FFF187E9AA4DA179A3B7B9F6A8FE3AB4E0A073B6B8D55E83C6D075BC957B
2. Tipo Fichero: PE 32 bits
3. No empacado
4. Nombre muestra: ProcessHollowing.exe
5. Ficheros obtenidos del análisis:
 - a. Payload extraído:
5F0AD6F81B73B78BB9AD29F357B53A60165101C11217FB495992A7BEC4CF1F60
6. Funcionalidad de la muestra:
 - a. Inyección en proceso mediante técnica process Hollowing
7. 55/70 motores de Virus Total detectan la muestra como maliciosa.

Puede ver el Vídeo Master_Process_HOLLOWING

En el siguiente enlace podemos ver otro ejemplo de la técnica de Process Hollowing realizada por un documento malicioso: [Enlace](#)

5.7.4. THREAD EXECUTION HIJACKING (A.K.A SUSPEND, INJECT, Y RESUME (SIR))

Se trata de una técnica parecida al process hollowing, la principal diferencia es que en este caso no se crea un proceso nuevo para inyectar el código malicioso, sino que se aprovechará un proceso ya abierto. Al no realizarse la llamada para la creación del nuevo proceso, esta técnica es más sigilosa que la de process hollowing. Por lo tanto, durante el análisis observaremos llamadas a funciones relacionadas con obtener información sobre un proceso y luego otra llamada para obtener el handler del proceso. Algunas de las llamadas que puedes encontrar son:

CreateToolhelp32Snapshot para obtener una instantánea de un proceso determinado y Thread32First que recupera información sobre el primer hilo de cualquier proceso encontrado en una instantánea del sistema. Seguidamente se utilizará OpenThread para abrir el hilo asociado al proceso objetivo.

Después de conseguir un handler para el hilo de destino, el malware sigue el mismo procedimiento que hemos analizado en Process Hollowing, poniendo el hilo en modo suspendido llamando a SuspendThread para realizar su inyección. El malware llama a VirtualAllocEx y WriteProcessMemory para asignar memoria y realizar la inyección de código.

La siguiente captura muestra un troyano genérico (Sha256: 787cbc8a6d1bc58ea169e51e1ad029a637f22560660cc129ab8a099a745bd50e) que utiliza esta técnica. Para secuestrar la ejecución del hilo, el malware modifica el registro EIP (un registro que contiene la dirección de la siguiente instrucción) del hilo objetivo llamando a SetThreadContext. Después, el malware reanuda el hilo para ejecutar el código de shell que ha escrito en el proceso del host. Desde la perspectiva del atacante, el enfoque SIR puede ser problemático porque suspender y reanudar un hilo en medio de una llamada al sistema puede hacer que el sistema se bloquee. Para evitar esto, un malware más sofisticado se reanudaría y volvería a intentarlo más tarde si el registro EIP está dentro del rango de NTDLL.dll.

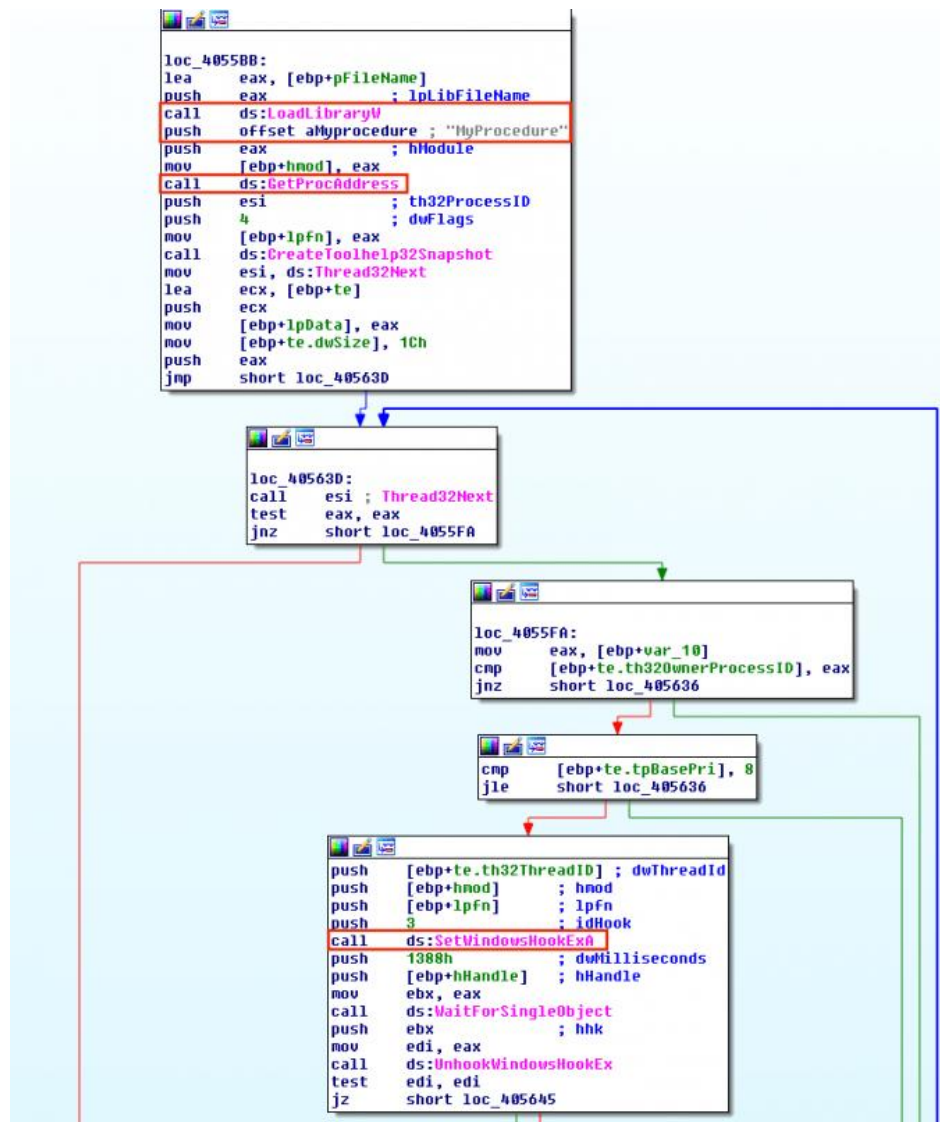


Captura 70: Troyano genérico realizando un Thread Execution Hijacking

5.7.5. HOOK INJECTION VIA SETWINDOWSHOOKEX

El Hooking es una técnica utilizada para interceptar llamadas de función. El malware puede aprovechar la funcionalidad de hooking para que su DLL maliciosa se cargue al activarse un evento en un hilo específico. Esto se hace normalmente llamando a SetWindowsHookEx para instalar una rutina de enganche en la cadena de Hook. La función SetWindowsHookEx toma cuatro argumentos. El primer argumento es el tipo de evento. Los eventos reflejan el rango de tipos de hooks, y varían desde presionar teclas en el teclado (WH_KEYBOARD) hasta entradas al ratón (WH_MOUSE), CBT, etc. El segundo argumento es un puntero a la función que el malware quiere invocar al ejecutarse el evento, y el tercer argumento es un módulo que contiene la función. Así, es muy común ver llamadas a LoadLibrary y GetProcAddress antes de llamar a SetWindowsHookEx. El último argumento de esta función es el hilo con el que se va a

asociar el procedimiento hook. Si este valor se establece en cero, todos los hilos realizan la acción cuando se dispara el evento. Sin embargo, el malware normalmente se dirige a un hilo para obtener menos ruido, por lo que también es posible ver las llamadas `CreateToolhelp32Snapshot` y `Thread32Next` antes de `SetWindowsHookEx` para encontrar y dirigirse a un solo hilo. Una vez que se inyecta la DLL, el malware ejecuta su código malicioso en nombre del proceso que su `threadId` fue pasado a la función `SetWindowsHookEx`. En la siguiente captura vemos un Locky Ransomware implementa esta técnica (Sha256: 5d6ddb8458ee5ab99f3e7d9a21490ff4e5bc9808e18b9e20b6dc2c5b27927ba1).



Captura 71: Locky Ransomware realizando un Hook Injection

5.7.6. INYECCIÓN Y PERSISTENCIA MEDIANTE REGISTRO (AooINIT_Dlls, AppCertDlls, IFEO)

Appinit_DLL, AppCertDlls, e IFEO (Image File Execution Options) son todas claves de registro que el malware utiliza tanto para la inyección como para la persistencia. Las entradas se encuentran en las siguientes ubicaciones:

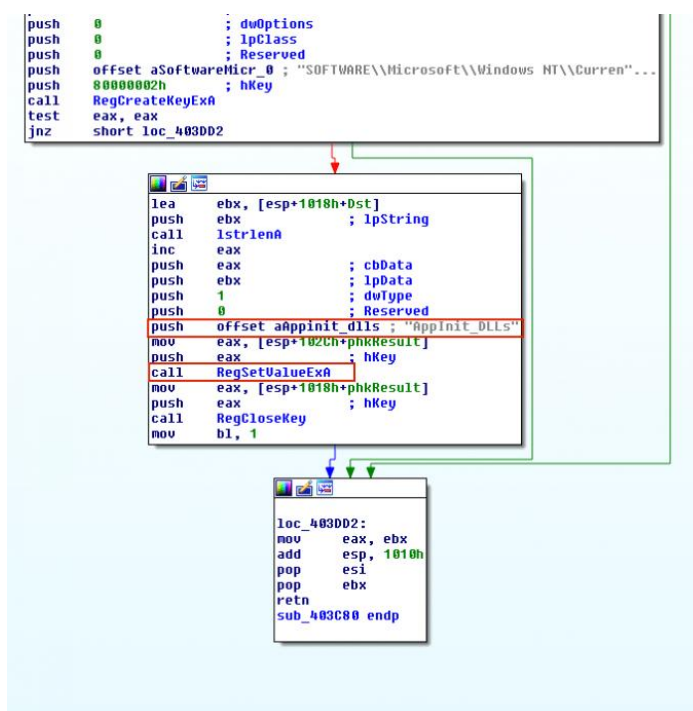
- HKLM\Software\Microsoft\NWindows
NT\NVersionActual\NWindows\NAppinit_Dlls
- HKLM\NSoftware\NWow6432Node\NMicrosoft\NWindows
NT\NVersionActual\NWindows_Appinit_Dlls HKLM

AppInit_Dlls

El malware puede insertar la ubicación de su biblioteca maliciosa bajo la clave de registro Appinit_Dlls para que otro proceso cargue su biblioteca. Cada biblioteca bajo esta clave de registro se carga en cada proceso que carga User32.dll. User32.dll es una biblioteca muy común utilizada para almacenar elementos gráficos como cuadros de diálogo. Así, cuando un malware modifica esta subclave, la mayoría de los procesos cargarán la biblioteca de malware. La siguiente captura muestra el troyano

Ginwui

(Sha256: 9f10ec2786a10971eddc919a5e87a927c652e1655ddbbae72d376856d30fa27c) que se basa en este enfoque para la inyección y la persistencia. Simplemente abre la clave de registro Appinit_Dlls llamando a RegCreateKeyEx, y modifica sus valores llamando a RegSetValueEx.



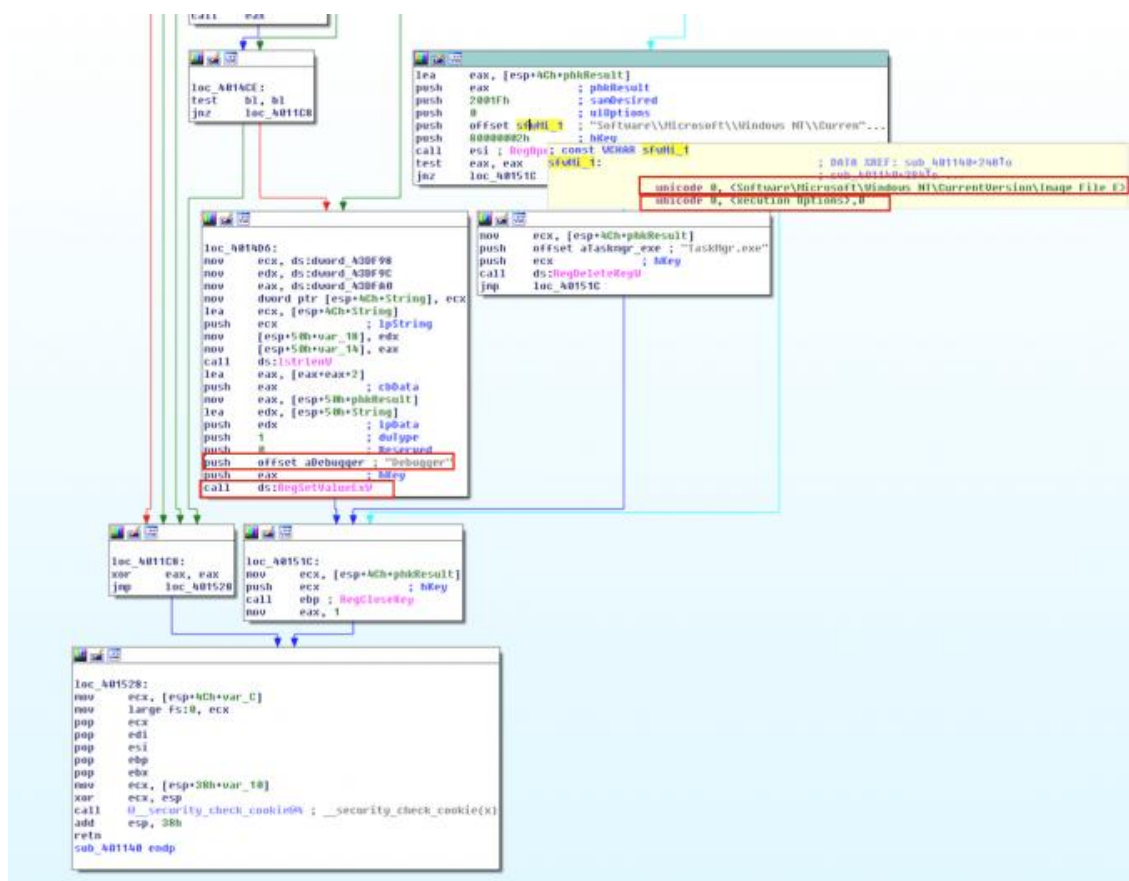
Captura 72: Ginwui modificando la clave de registro AppInit_DLLs

AppCertDlls

Este enfoque es muy similar al enfoque AppInit_DLLs, excepto que las DLL bajo esta clave de registro se cargan en cada proceso que llama a las funciones de la API de Win32 `CreateProcess`, `CreateProcessAsUser`, `CreateProcessWithLogonW`, `CreateProcessWithTokenW`, y `WinExec`.

Opciones de ejecución de archivos de imagen (IFEO)

El IFEO se utiliza típicamente con fines de depuración. Los desarrolladores pueden establecer el "Valor de depuración" bajo esta clave de registro para adjuntar un programa a otro ejecutable para la depuración. Por lo tanto, cada vez que se lance el ejecutable, se lanzará el programa que está conectado a él. Para utilizar esta característica, simplemente puede dar la ruta al depurador y adjuntarlo al ejecutable que desea analizar. El malware puede modificar esta clave de registro para inyectarse en el ejecutable de destino. En la siguiente captura, el troyano Ditzakun (Sha256: f0089056fc6a314713077273c5910f878813fa750f801dfca4ae7e9d7578a148) implementa esta técnica modificando el valor del depurador del Administrador de tareas.



Captura 73: Troyano Ditzakun modificando la clave de registro IFEO

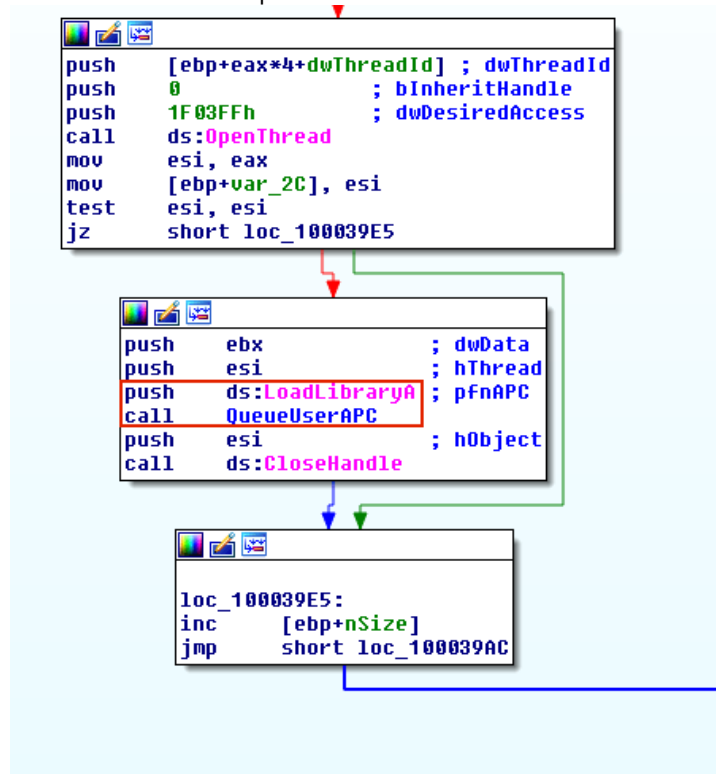
5.7.7. APC INJECTION Y ATOMBOMBING

La llamada a procedimiento asincrónica es una unidad de trabajo en una computadora. Por lo general, un programa funciona ejecutando una serie de llamadas a procedimientos síncronos en algún hilo. Pero si algunos datos no están listos (por ejemplo, un programa espera que el usuario responda), entonces mantener el hilo en estado de espera no es práctico, ya que un hilo asigna una cantidad considerable de memoria para la pila de procedimientos y esta memoria no se usa. Entonces, tal llamada a procedimiento se forma como un objeto con una pequeña cantidad de memoria para datos de entrada, y este objeto se pasa al servicio que recibe las entradas del usuario. Cuando se recibe la respuesta del usuario, el servicio la coloca en el objeto y pasa ese objeto a un servicio de ejecución. El servicio de ejecución consta de uno o más subprocesos de trabajo dedicados y una cola para tareas. Cada hilo de trabajo lee en una cola de tareas de bucle y, cuando se recupera una tarea, la ejecuta. Cuando no hay tareas, los subprocesos de trabajo están esperando y, por lo tanto, su memoria no se usa, pero el número de subprocesos de trabajo es lo suficientemente pequeño (no tiene sentido tener más subprocesos que procesadores en la máquina). [Llamada a procedimiento asincrónico](#).

El malware puede aprovechar las llamadas de procedimientos asíncronos (APC) para

forzar a otro hilo a ejecutar su código personalizado conectándolo a la cola APC del hilo objetivo. Cada hilo tiene una cola de APCs que esperan su ejecución cuando el hilo objetivo entra en estado alterable. Un hilo entra en un estado de alerta si llama a las funciones SleepEx, SignalObjectAndWait, MsgWaitForMultipleObjectsEx, WaitForMultipleObjectsEx, o WaitForSingleObjectEx. El malware suele buscar cualquier hilo que se encuentre en un estado alterable, y luego llama a OpenThread y a QueueUserAPC para poner en cola a un APC en un hilo. QueueUserAPC recibe tres argumentos: 1) un "handle" del hilo de destino; 2) un puntero a la función que el malware quiere ejecutar; 3) y el parámetro que se pasa al puntero de la función. En la siguiente captura, el malware de Amanahe (Sha256: f74399cc0be275376dad23151e3d0c2e2a1c966e6db6a695a05ec1a30551c0ad) primero llama a OpenThread para adquirir un "handle" de otro hilo, y luego llama a QueueUserAPC con LoadLibraryA como puntero de función para inyectar su DLL maliciosa en otro hilo.

El bombardeo atómico es una técnica que fue introducida por primera vez por la investigación de enSilo, y luego utilizada en Dridex V4. Como hemos discutido en detalle en un post anterior, la técnica también se basa en la inyección de APC. Sin embargo, utiliza tablas de átomos para escribir en la memoria de otro proceso.



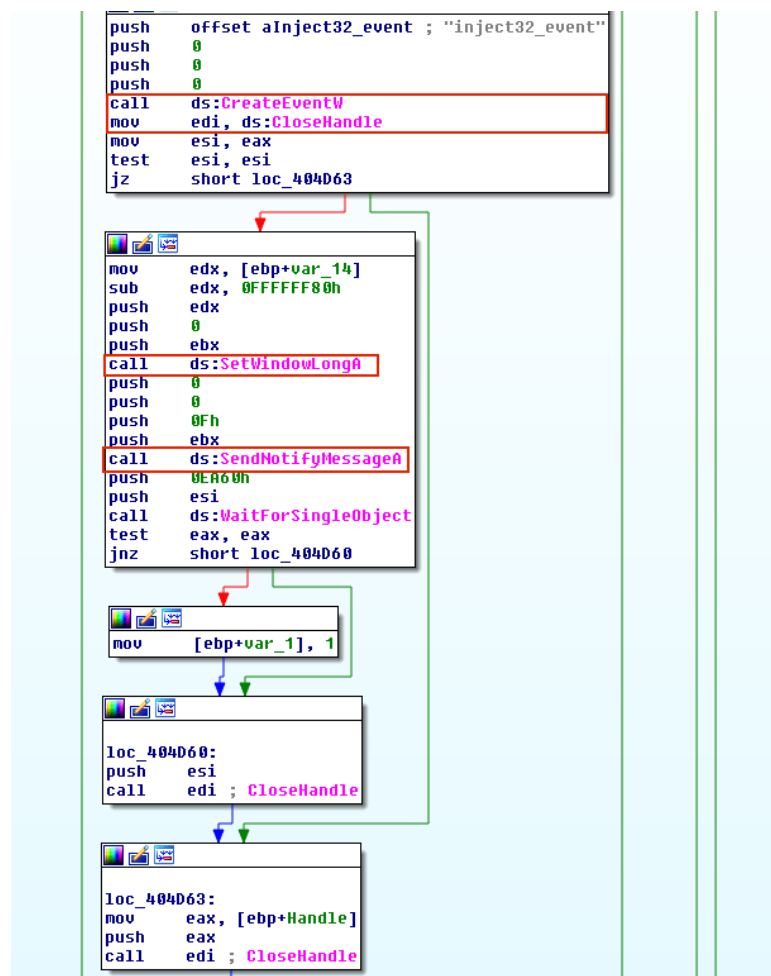
Captura 74: Almanahe realizando una inyección APC

5.7.8. EXTRA WINDOWS MEMORY INJECTION (EWMI) VIA SETWINDOWLONG

EWMI se basa en inyectar en la ventana de la bandeja del explorador la memoria de ventana extra, y se ha utilizado varias veces entre las familias de malware como Gapz y PowerLoader. Al registrar una clase de ventana, una aplicación puede especificar un número de bytes adicionales de memoria, llamado memoria de ventana extra (EWM). Sin embargo, no hay mucho espacio en EWM. Para sortear esta limitación, el malware escribe código en una sección compartida de explorer.exe, y utiliza SetWindowLong y SendNotifyMessage para tener un puntero de función que apunte al código de shell, y luego lo ejecuta.

El malware tiene dos opciones a la hora de escribir en una sección compartida. Puede crear una sección compartida y asignarla tanto a sí mismo como a otro proceso (por ejemplo, explorer.exe), o puede simplemente abrir una sección compartida que ya existe. La primera tiene la ventaja de asignar el espacio de acumulación y llamar a NTMapViewOfSection además de algunas otras llamadas a la API, por lo que el segundo enfoque se utiliza más a menudo. Después de que el malware escribe su código de shell en una sección compartida, utiliza GetWindowLong y SetWindowLong para acceder y modificar la memoria de ventana extra de "Shell_TrayWnd". GetWindowLong es una API que se utiliza para recuperar el valor de 32 bits con el desfase especificado en la memoria de ventana extra de un objeto de clase ventana, y SetWindowLong se utiliza para cambiar los valores con el desfase especificado. De este modo, el malware puede cambiar simplemente el offset de un puntero de función en la clase ventana, y apuntarlo al código de shell escrito en la sección compartida.

Al igual que la mayoría de las técnicas mencionadas anteriormente, el malware necesita activar el código que ha escrito. En las técnicas anteriormente mencionadas, el malware lo consigue llamando a APIs como CreateRemoteThread, QueueUserAPC o SetThreadContext. Con este enfoque, el malware desencadena el código inyectado llamando a SendNotifyMessage. Tras la ejecución de SendNotifyMessage, Shell_TrayWnd recibe y transfiere el control a la dirección apuntada por el valor previamente establecido por SetWindowLong. En la siguiente captura, un malware llamado PowerLoader (Sha256: 5e56a3c4d4c304ee6278df0b32afb62bd0dd01e2a9894ad007f4cc5f873ab5cf) utiliza esta técnica.



Captura 75: PowerLoader inyectando en la memoria extra de la ventana de la bandeja del shell

5.7.9. PROCESS INJECTION UTILIZANDO SHIMS

Microsoft proporciona Shims a los desarrolladores principalmente para la compatibilidad con versiones anteriores. Las Shims permiten a los desarrolladores aplicar correcciones a sus programas sin necesidad de reescribir el código. Aprovechando las shims, los desarrolladores pueden decirle al sistema operativo cómo manejar su aplicación. Los shims son esencialmente una forma de engancharse a las API y de dirigirse a ejecutables específicos. El malware puede aprovechar los shims para dirigirse a un ejecutable tanto para la persistencia como para la inyección. Windows ejecuta el motor Shim cuando carga un binario para comprobar si hay bases de datos shim para aplicar las correcciones adecuadas.

Hay muchas correcciones que se pueden aplicar, pero las favoritas del malware son las que están relacionadas con la seguridad (por ejemplo, DisableNX, DisableSEH, InjectDLL, etc.). Para instalar una base de datos de shimming, el malware puede desplegar varios enfoques. Por ejemplo, un enfoque común es simplemente ejecutar sdbinst.exe, y apuntarlo al archivo sdb malicioso. En la siguiente captura, un adware,

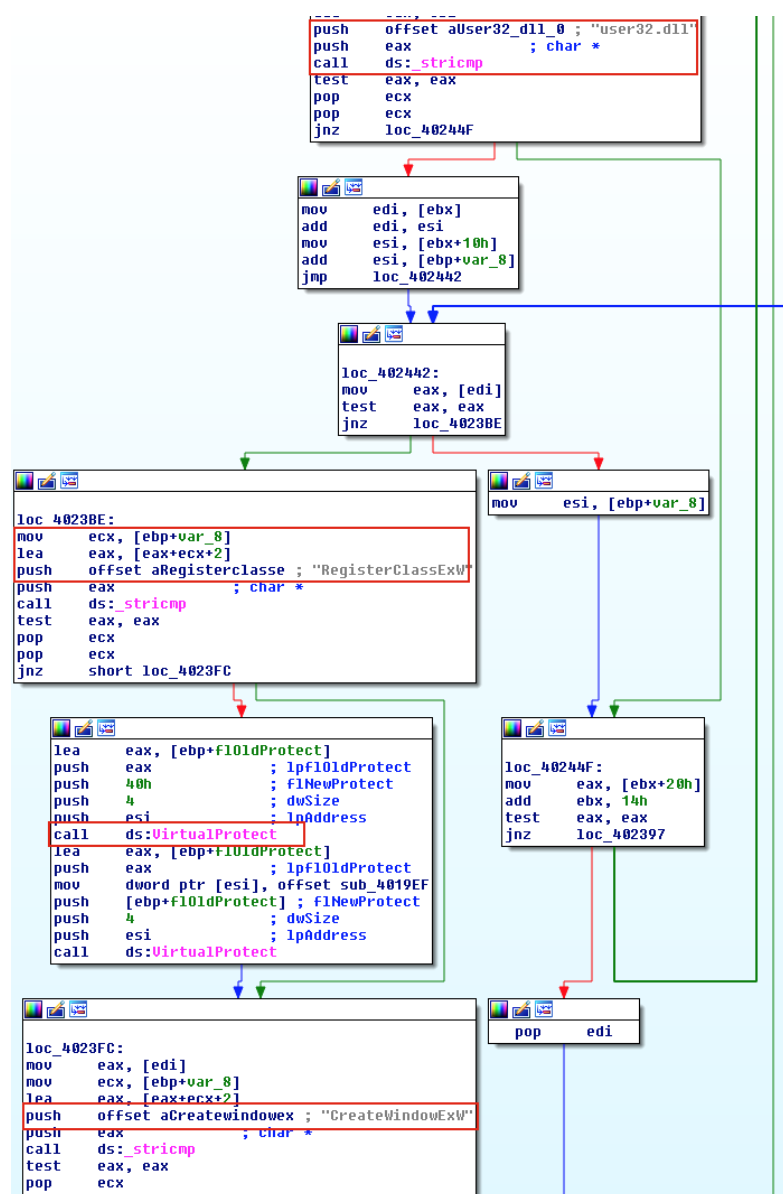
"Search Protect by Conduit" (Sha256: 6d5048baf2c3bba85adc9ac5ffd96b21c9a27d76003c4aa657157978d7437a20), utiliza un shim para la persistencia y la inyección. Realiza una calza "InjectDLL" en Google Chrome para cargar vc32loader.dll. Existen algunas herramientas para analizar los archivos sdb, pero para el análisis de los sdb listados a continuación, utilicé python-sdb.

```
<NAME type='stringref'>0x1ac</NAME>
<APP_NAME type='stringref'>0x1e6</APP_NAME>
<VENDOR type='stringref'>0x106</VENDOR>
<EXE_ID type='hex'>ce8affb6-1e8f-41f3-a1a3-cafd2e996ab8</EXE_ID>
<MATCHING_FILE>
  <NAME type='stringref'>0x120</NAME>
</MATCHING_FILE>
<LAYER>
  <NAME type='stringref'>0x30</NAME>
  <LAYER_TAGID type='integer'>0x19a</LAYER_TAGID>
</LAYER>
</EXE>
<EXE>
  <NAME type='stringref'>0x1f2</NAME>
  <APP_NAME type='stringref'>0x22e</APP_NAME>
  <VENDOR type='stringref'>0x106</VENDOR>
  <EXE_ID type='hex'>2b4c4b81-d5b5-4cb2-9436-ef2799a4630c</EXE_ID>
  <MATCHING_FILE>
    <NAME type='stringref'>0x120</NAME>
  </MATCHING_FILE>
  <LAYER>
    <NAME type='stringref'>0x30</NAME>
    <LAYER_TAGID type='integer'>0x19a</LAYER_TAGID>
  </LAYER>
</EXE>
</DATABASE>
<STRINGTABLE>
  <STRINGTABLE_ITEM type='string'>2.1.0.3</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>Apps32</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>VC32Ldr</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>InjectDll</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>\\.\globalroot\systemroot\apppatch\nbin\vc32loader.dll</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>chrome.exe</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>chc</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>&lt;Unknown&gt;</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>*</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>explorer.xxx</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>ex</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>firefox.exe</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>ff</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>iexplore.exe</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>ie</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>software_removal_tool.exe</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>sr</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>software_reporter_tool.exe</STRINGTABLE_ITEM>
  <STRINGTABLE_ITEM type='string'>sr2</STRINGTABLE_ITEM>
</STRINGTABLE>
```

Captura 76: SDB utilizado por Search Protect con fines de inyección

5.7.10. IAT HOOKING E INLINE HOOKING (A.K.A USERLAND ROOTKITS)

El enganche IAT y el enganche en línea se conocen generalmente como rootkits de usuario. El enganche IAT es una técnica que el malware utiliza para cambiar la tabla de direcciones de importación. Cuando una aplicación legítima llama a una API ubicada en una DLL, se ejecuta la función sustituida en lugar de la original. En cambio, con el enganche en línea, el malware modifica la propia función de la API. En la siguiente captura, el malware Fin Fisher (Sha256: f827c92fbe832db3f09f47fe0dcaafd89b40c7064ab90833a1f418f2d1e75e8e), realiza el enganche IAT modificando donde apunta el CreateWindowEx.



Captura 77: FinFisher realiza el hooking de IAT cambiando el lugar al que apunta CreateWindowEx

5.7.11. PROCESS DOPPELGANGING

Process Doppelganging es similar a Process Hollowing, es un enfoque diferente para lograr lo mismo, abusando de Windows NTFS Transactions y una implementación anticuada del cargador de procesos de Windows, que fue originalmente diseñado para Windows XP pero que se llevó a todas las versiones posteriores de Windows.

¿Cómo funciona el ataque?

Doppelganging funciona utilizando dos características distintas juntas para enmascarar la carga de un ejecutable modificado. Utilizando una característica

llamada NTFS Transaccional (TxF) en Windows para hacer cambios a un archivo ejecutable que nunca actuará. El malware debía escribirse en el disco o ejecutarse completamente desde la memoria, los productos de seguridad desarrollaron herramientas para luchar contra este tipo de malware. En caso de que el malware tuviera un archivo en el disco, el archivo podía ser escaneado.

El software que se ejecuta sin archivo es sospechoso y también podría ser detectado. Con Process Doppelganging, el software malicioso puede ejecutarse desde un archivo, pero este archivo será invisible para el software de seguridad. Los cambios realizados nunca se escriben en el disco, por lo tanto, es un ataque sin archivo que no puede ser rastreado por un software antivirus. El ejecutable modificado se carga entonces usando el mecanismo de carga de procesos de Windows. El proceso de malware puede seguir ejecutándose en tal caso. Si se abre, el archivo en el disco no tendrá ningún contenido sospechoso. Además, este archivo puede ser una aplicación conocida y firmada digitalmente.

Se crea un objeto de sección utilizando la API de NtCreateSection a partir del archivo modificado y se deshace la transacción con la API de la transacción Rollback. Una vez que esto se completa, se llama a NtCreateProcessEx con la sección maliciosa pasada como parámetro y luego se reanuda la ejecución en el proceso remoto.

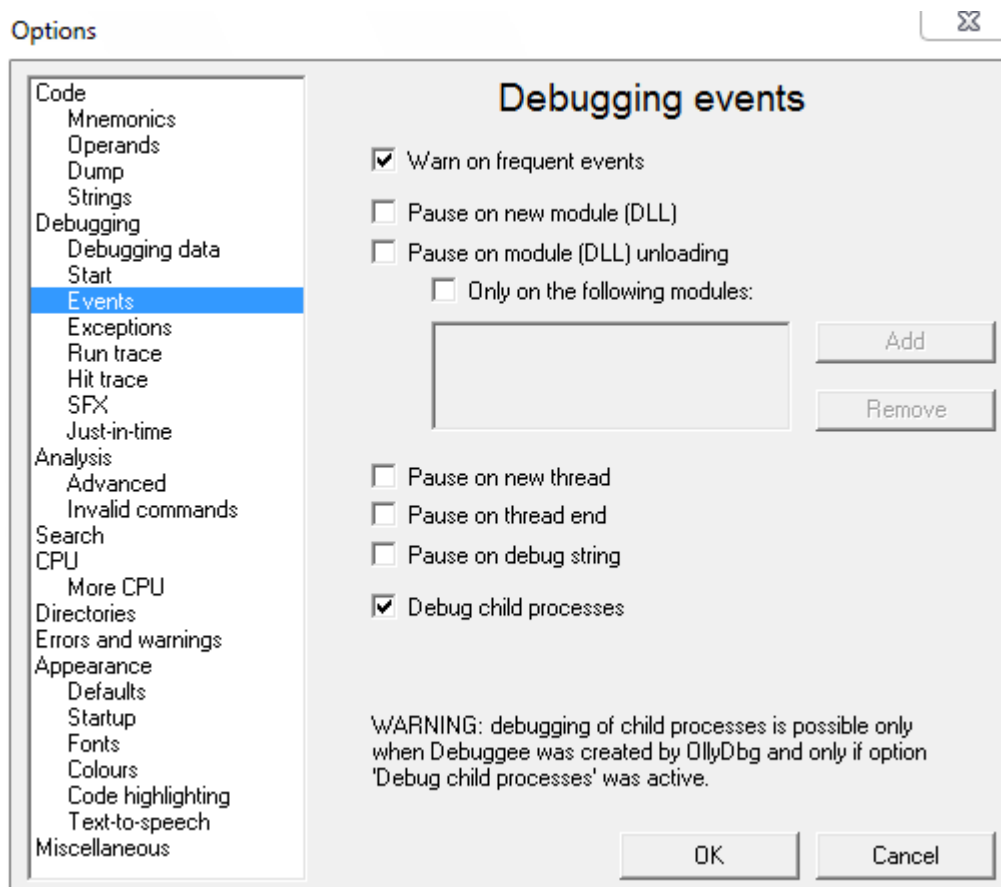
Llamar la ruta de la cadena de inyección es:

```
CreateTransaction -> CreateFileTransacted -> WriteFile -> CreateSection ->
NtCreateProcessEx -> RtlCreateProcessParametersEx -> VirtualAllocEx ->
WriteProcessMemory -> NtCreateThreadEx
```

Como podemos ver, el uso de APIs sospechosas como ReadProcessMemory/NtReadVirtualMemory, WriteProcessMemory, NtMapViewOfSection y SetThreadContext es menor, y la imagen es apilada por el cargador de PE de Windows en lugar de ser escrita en la memoria usando WPM, lo que da lugar a un procedimiento de aspecto considerablemente más auténtico.

En el siguiente enlace hay un artículo muy interesante sobre el análisis de un malware que utiliza esta técnica: [Enlace](#)

Otra aproximación para lidiar con procesos hijos es usar OllyDbg2.0 en lugar de OllyDbg1.10, ya que la versión 2 soporta la depuración de procesos hijos.



Captura 78: Configuración OllyDbg2.0 para análisis de procesos hijos

Cuando esta casilla está activada al comenzar a depurar un proceso, una nueva instancia de ollydbg se iniciará automáticamente siempre que el proceso depurado cree un nuevo proceso.

Sin embargo, ollydbg 1.0 tiene ciertas características que no tiene ollydbg 2.0. Habrá que valorar en cada situación la herramienta que mejor se ajusta a nuestras necesidades.

6. ANÁLISIS DE BINARIOS X64.

Para realizar reversing de aplicaciones de 64 bits, seguiremos la misma metodología aprendida durante este módulo. Por otra parte, algunas de las herramientas que hemos venido utilizando para malware desarrollado en 32 bits, deberán ser sustituidas por sus versiones de 64 bits como sucede con IDA. En otros casos las sustituiremos herramientas que ofrecen las mismas funcionalidades pero que están diseñadas para entornos de 64 bits como por ejemplo Olly, que no podremos utilizarlo y en su lugar utilizaremos XDB64.

Otros aspectos a tener en cuenta serán los registros del sistema, que utilizan otra nomenclatura diferente:

Los procesadores de la arquitectura x86-64 disponen de un banco de registros formado por registros de propósito general y registros de propósito específico.

Registros de propósito general hay 16 de 64 bits y de propósito específico hay 6 registros de segmento de 16 bits, también hay un registro de estado de 64 bits (RFLAGS) y un registro contador de programa también de 64 bits (RIP).

Registros de propósito general

Son 16 registros de datos de 64 bits (8 bytes): RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP y R8-R15.

Los 8 primeros registros se denominan de manera parecida a los 8 registros de propósito general de 32 bits disponibles en la arquitectura IA-32 (EAX, EBX, ECX, EDX, ESI, EDI, EBP y ESP).

Los registros son accesibles de cuatro maneras diferentes:

- 1) Como registros completos de 64 bits (quad word).
- 2) Como registros de 32 bits (double word), accediendo a los 32 bits de menos peso.
- 3) Como registros de 16 bits (word), accediendo a los 16 bits de menos peso.
- 4) Como registros de 8 bits (byte), permitiendo acceder individualmente a uno o dos de los bytes de menos peso según el registro.

En el siguiente enlace se proporciona documentación detallada relativa a la arquitectura de 64 bits: [Enlace](#)

En el siguiente enlace se ofrece información sobre el uso básico de x86_64: [Enlace](#)

7. ANÁLISIS DE MALWARE EN DISPOSITIVOS MÓVILES.

El malware móvil es un software malicioso diseñado específicamente para atacar dispositivos móviles, como smartphones y tabletas, con el objetivo de acceder a datos privados.

Aunque el malware para móviles no está actualmente tan extendido como el que ataca a las estaciones de trabajo tradicionales, es una amenaza creciente.

En un módulo anterior del master se estudió el proceso de análisis estático y dinámico sobre aplicaciones móviles. Habrá de tomarse este módulo de referencia para analizar aplicaciones móviles con el objetivo de determinar si realizan acciones maliciosas en base a su comportamiento. Seguiremos la misma metodología descrita a lo largo de este módulo en lo relativo a análisis estático y dinámico, lo que cambiará son únicamente las herramientas que utilizaremos.

Tipos de malware para móviles

Los ciberdelincuentes utilizan diversas tácticas para infectar los dispositivos móviles. Es importante conocer los diferentes tipos de amenazas de malware para móviles. A continuación, se presentan algunos de los tipos más comunes:

Las herramientas de acceso remoto (RAT) ofrecen un amplio acceso a los datos de los dispositivos de las víctimas infectadas y suelen utilizarse para la recopilación de información. Las RAT suelen acceder a información como las aplicaciones instaladas, el historial de llamadas, las libretas de direcciones, el historial de navegación web y los datos de sms. Las RAT también pueden utilizarse para enviar mensajes SMS, habilitar las cámaras de los dispositivos y registrar los datos del GPS.

Los troyanos bancarios suelen disfrazarse de aplicaciones legítimas y buscan comprometer a los usuarios que realizan sus operaciones bancarias -incluidas las transferencias de dinero y el pago de facturas- desde sus dispositivos móviles. Este tipo de troyanos tiene como objetivo robar los datos de acceso financiero y las contraseñas.

El ransomware es un tipo de malware que se utiliza para bloquear a un usuario en su dispositivo y exigir el pago de un "rescate", normalmente en Bitcoin imposible de rastrear. Una vez que la víctima paga el rescate, se le proporcionan códigos de acceso que le permiten desbloquear su dispositivo móvil.

El malware de Criptominería permite a los atacantes ejecutar minado de criptomoneda de forma encubierta en el dispositivo de la víctima, lo que les permite generar criptomonedas. La criptominería se realiza a menudo a través de un código troyano que se oculta en aplicaciones de aspecto legítimo.

El fraude de clics publicitarios es un tipo de malware que permite a un atacante secuestrar un dispositivo para generar ingresos a través de clics publicitarios falsos.

Métodos de distribución de malware para móviles

Hay algunas formas comunes en las que se basan los atacantes para distribuir su código malicioso:

1. Phishing y Spoofing móvil: El phishing es la práctica de engañar a alguien para que proporcione su valiosa cuenta o información personal, a menudo mediante la suplantación de identidad. El spoofing es la práctica de disfrazar las comunicaciones electrónicas o los sitios web como una entidad de confianza de la víctima. Aunque el spoofing y el phishing suelen ir de la mano, el spoofing puede utilizarse para otros objetivos más allá del phishing. Por ejemplo, un correo electrónico suplantado puede intentar convencer al destinatario de que haga clic en un mensaje malicioso.

Mientras que el phishing se ha centrado tradicionalmente en la adquisición de credenciales a través del correo electrónico, el phishing a través de mensajes sms (smshishing) y aplicaciones de mensajería se ha vuelto mucho más frecuente.

Uno de los métodos más populares para engañar a las víctimas para que instalen malware es enviarles enlaces a través de un SMS falso a archivos de paquetes Android (APK) alojados en sitios web controlados por los atacantes. Por ejemplo, se puede pedir a las víctimas que hagan clic en un enlace SMS a un sitio bancario falso diseñado para parecer fiable y convencer a la víctima de que "actualice su aplicación bancaria". La actualización instalaría entonces el código malicioso, permitiendo así al atacante obtener acceso y recoger las credenciales.

2. Dispositivos rooteados o con jailbreak

Rootear o hacer jailbreak a un dispositivo significa simplemente que se han saltado las protecciones internas y se tiene un control ilimitado del sistema operativo. Aquellos que hacen jailbreak a sus teléfonos a menudo lo hacen para descargar aplicaciones de terceros que no están aprobadas por su sistema operativo, o hacer personalizaciones a sus teléfonos que no son posibles con las protecciones por defecto.

Aunque el jailbreaking y el rooting pueden abrir un mundo de libertades y personalizaciones, también exponen los dispositivos a un mayor riesgo de ataque malicioso.

En los siguientes enlaces pueden encontrarse ejemplos de análisis de malware en dispositivos móviles: [Enlace 1](#) [Enlace 2](#) [Enlace 3](#)

8. TÉCNICAS ANTI-DEBUGGIN.

Podemos definir las técnicas antidebugging, como los mecanismos que implementan los desarrolladores de malware para dificultar el análisis de la muestra lo máximo posible. Los programas de depuración del código permiten analizar malware ejecutando el código malicioso paso a paso, ofreciendo diferentes funcionalidades como introducir cambios en el espacio de memoria, Añadir puntos de interrupción, observar e incluso modificar los valores de variables y registros entre otras muchas funcionalidades. Si la depuración se realiza de forma correcta y sin dificultad, facilitará la comprensión del comportamiento del malware. Por lo tanto, los desarrolladores de malware se afanan tratando de evitar el proceso de depuración.

Los motivos por los que invierten tiempo, imaginación y esfuerzo en desarrollar nuevas técnicas pueden resumirse en los siguientes puntos:

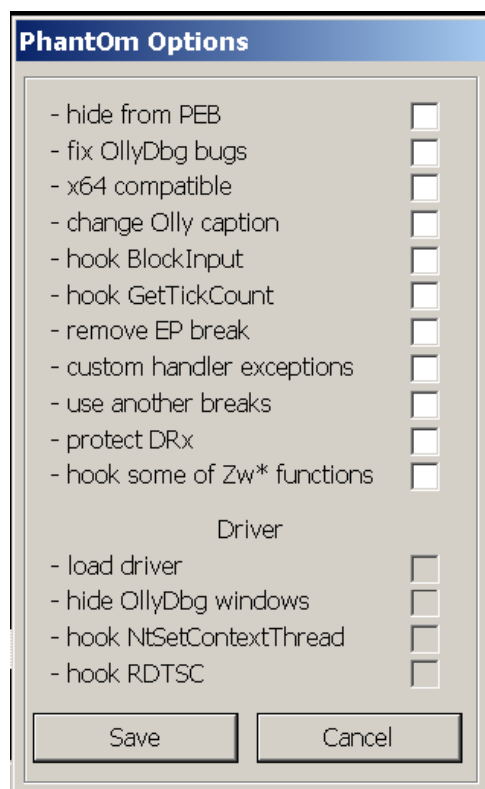
- Si entendemos el ciclo de vida del malware, desde su desarrollo y hasta que ya es detectado por todos los motores antivirus, comprenderemos que desde que un malware se lanza por primera vez, hasta que ya es totalmente detectado, transcurre un tiempo determinado, este tiempo será mayor

cuantas más técnicas implemente la muestra tanto varía ser detectada como para ser analizada en profundidad.

- Todo malware suele estar soportado por una infraestructura de comunicaciones, con diferentes objetivos pero que podemos resumir en todo lo que se puede hacer desde un C2C. Adicionalmente existen otros IoC y particularidades en el propio código de la muestra que tratan siempre de ocultarse, para evitar ser detectados gracias al análisis profundo de la amenaza.
- Las técnicas de anti-depuración tratan que un programa no se ejecute bajo un depurador y en caso de que lo haga, cambiar su comportamiento, producir un error o no ejecutarse.

Las técnicas que los desarrolladores de malware utilizan, son complejas y creativas, van cambiando a lo largo del tiempo y siempre suponen un reto para el analista. Las técnicas más comunes, podremos bypassearlas con sencillez gracias a los plugings que hay disponibles para los diferentes entornos de depuración. Sin embargo, todas aquellas técnicas que se apoyan en la propia complejidad del código y los flujos que se utilizan, los cuales pueden incluir inyecciones en diferentes procesos de formas muy variadas como hemos podido ver anteriormente, utilización de ficheros, diferentes capas de empaquetado, etc serán sin duda el reto al que el analista deberá enfrentarse al analizar una muestra.

En definitiva, abordaremos este problema, utilizando plugins que nos ayudarán en gran medida, y seguiremos la metodología explicada para tratar de entender en el mayor grado posible el flujo, funcionalidades y técnicas que implementa, así como realizar la obtención de los IoC y particularidades encontrados. En la mayoría de los casos, el proceso de Anti-Debugging ralentizará el proceso de ingeniería inversa, pero no lo hará imposible.



Captura 79: Configuración plugin Phamthom

Analicemos ahora en detalle que tipo de técnicas antidebuging podremos sortear con el uso de este plugging en Olly.

Veamos primero “HideFrom PEB”:

Antes de comenzar debemos saber que el Bloque del Entorno de Procesos (PEB) es una estructura de datos que reside en el espacio de usuario de cada proceso y contiene datos sobre el proceso relacionado. Puede ser accedido por los módulos WinAPI y directamente desde la memoria.

BeingDebugged: La técnica más directa es comprobar el valor del campo BeingDebugged en la estructura del PEB. Hay muchas llamadas a la API documentadas y no documentadas para comprobar este valor:

- CheckRemoteDebuggerPresent
- CreateToolhelp32Snapshot
- DbgSetDebugFilterState
- IsDebuggerPresent
- NtQueryInformationProcess
- RtlQueryProcessHeapInformation
- RtlQueryProcessDebugInformation

También se puede obtener el valor de BeingDebugged sin una llamada a API de sistema. Podemos realizarlo mediante la lectura del registro de segmento fs en fs:[30h]

el cual apunta a TIB (Thread Information Block) para arquitecturas x86.

ProcessHeap: Una flag del PEB muestra si el primer espacio de memoria heap del proceso fue creado en modo de depuración.

El Heap contiene dos flags que están inicializadas en conjunción con la NtGlobalFlag. Los valores en esos campos se ven afectados por la presencia de un depurador, también dependen de la versión de Windows. Se puede acceder al flag con un offset determinado en PEB.

Rastros en el sistema:

Como cualquier otro software, los programas de depuración dejan rastros en el sistema, el malware trata de buscar esos rastros.

Claves de registro:

- Las entradas de registro que los depuradores escriben tras su instalación.
- Existe una clave de registro que se establece en el programa depurador que se ejecuta cuando se produce un error ("HKLM\Software\Microsoft\Windows NT\Versión Actual\AeDebug").

FindWindow: Función WinAPI que puede reconocer la presencia de un programa depurador (por ejemplo "ollydbg.exe"). El mismo resultado se puede alcanzar con OpenProcess para los procesos o con FindFirstFile y FindNextFile para las rutas del sistema de archivos (por ejemplo, en la carpeta %ProgramFiles%).

Identificación de los puntos de ruptura: Un Breakpoint es probablemente la característica más importante de un depurador; permite al investigador detener la ejecución de un programa en un determinado punto del código.

Escaneo de instrucciones - El punto de ruptura de un depurador se implementa insertando "0xCC" ("INT 3" en ensamblador) lo que significa que el software interrumpe con el parámetro '3'. Un malware puede escanearse a sí mismo para encontrar esta instrucción en ensamblador (por ejemplo, con la instrucción en ensamblador "repne scasb").

Calcular la suma de comprobación o el hash del código: el malware puede calcular una suma de comprobación o un hash de su código en tiempo de ejecución para determinar si se ha parcheado o si se ha insertado un punto de interrupción (por ejemplo, un valor que se describió anteriormente: "0xCC"), y por lo tanto esto también puede utilizarse para identificar depuradores.

El entorno del depurador:

Hay muchas técnicas para identificar el contexto del depurador en funcionamiento.

OutputDebugString se utiliza para enviar una cadena a un depurador. Si falla, se produce un error. El malware puede utilizar SetLastError con un valor definido, luego ejecutar OutputDebugString (si falla, sobrescribirá el último valor de error), y luego comprobar el último error con GetLastError.

Comprobaciones de tiempo: cuando se depura en un modo de un solo paso / establecimiento de un punto de interrupción, se produce un retraso mientras se ejecuta el ejecutable. El malware puede comprobar si hay una marca de tiempo y compararla con otra después de unas pocas instrucciones maliciosas, con el fin de comprobar si hubo un retraso. La instrucción rdtsc se utiliza para obtener el número de ticks desde el último reinicio del sistema y se reinicia después de 49,7 días (se colocará como valor de 64 bits en los registros EDX:EAX de 32 bits). Hay funciones WinAPI equivalentes para esta prueba (GetTickCount / QueryPerformanceCounter).

Depurador de vulnerabilidades: un depurador es un programa, y como cualquier otro programa, puede tener vulnerabilidades. Por ejemplo, una famosa sería: colapsar OllyDbg 1.1 llamando a OutputDebugString con el valor "%s".

Opciones de ejecución de archivos de imagen (IFEO): IFEO es una clave de registro que está destinada a ejecutar un programa particular bajo un depurador local.

Clave: "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion - Opciones de ejecución de archivos de imagen - Programa particular"

Valor: Depurador: <camino completo a un depurador local> (REG_SZ)

Un malware puede comprobar si hay algún depurador configurado manualmente en la máquina, aunque la depuración y la anti-depuración no es el único uso de esta clave de registro. También se implementa fácilmente como técnica para lanzar malware, ya que Windows no realiza ninguna comprobación de que el "depurador" sea realmente un depurador. Por esta razón, un autor de malware puede abusar de él para ejecutar malware, por ejemplo, simplemente adjuntándolo a otro proceso que se carga de forma predeterminada con el inicio.

Retro llamada TLS: TLS (Thread Local Storage) es una región de memoria que es local a un hilo. Se puede abusar de ella para ejecutar una función antes del punto de entrada del PE (es decir, en la inicialización de un hilo) - esta técnica se llama TLS Callback. En la mayoría de los casos, los depuradores comienzan a depurar desde el punto de entrada. Esta técnica puede identificarse de las siguientes maneras:

- Estáticamente por la existencia de la sección .tls
- IDA Pro soporta correr desde todos los puntos de entrada (incluyendo TLS Callback)
- Los depuradores pueden ser configurados para establecer un punto de ruptura antes de ejecutar el TLS Callback.

- Todas las funciones de devolución de llamada TLS están etiquetadas con "TlsCallback" preparado...

En resumen, la Anti-depuración puede ser alcanzada por las características esenciales del sistema operativo o por el comportamiento natural del propio depurador. Los autores de malware siguen desafiando a los investigadores con nuevas técnicas para retrasar el proceso de ingeniería inversa.

El siguiente enlace contiene un documento muy completo sobre técnicas anti debugin: [Enlace](#)

Ejercicios propuestos:

Realice un análisis básico y avanzado estático y dinámico para crear un pequeño informe resultado como los mostrados durante el módulo de los malware analizados durante el presente módulo.

1. **Malware.exe**
(49A48D4FF1B7973E55D5838F20107620ED808851231256BB94C85F6C80B8EBFC)
2. **9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66.xlsm**
(9a6b21529e1dce6bc494ef0cc67e983cc822226d2579be1e588bd8c483e95d66)
3. **5e3cb24d52ef617a4dca51ece6721a0745a17909f1b7fd4a54a59ceba7ea32b6.ppt**
(5e3cb24d52ef617a4dca51ece6721a0745a17909f1b7fd4a54a59ceba7ea32b6)
4. **0b3b3b22c8a6e3474150ea1cb8ab494413d3a641d475916114b8c4a94393f753.pdf**
(0b3b3b22c8a6e3474150ea1cb8ab494413d3a641d475916114b8c4a94393f753)
5. **9da7d051bdf010d86a461d407c30d40f57d495fb6a5d22735ff792724bc3831e.rtf**
(9da7d051bdf010d86a461d407c30d40f57d495fb6a5d22735ff792724bc3831e)
6. **2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf**
(2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762)
7. **Análisis muestra: NET_MALWARE_master_1.exe**
(6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7)
8. **malware_process_injection_1.exe**
(7e7d0557cc95e3f509f71a72aad9b8ab85d6a681df4a46e1648e928a4be5f4be)